50 LLM questions

interviewers expect you to know.

8 of them decide senior offers.

Swipe for the ones that get asked most.

SECTION 1: FOUNDATIONS

* 1. What are Large Language Models (LLMs)?

LLMs are deep neural networks, almost always transformer-based, trained on very large text corpora with one deceptively simple objective:

> predict the next token.

→ Scale that objective across billions of parameters and trillions of tokens and the model **stops** being just a text predictor.

→ It starts to carry usable representations of grammar, facts, reasoning patterns and style, which is why one model can summarise, translate, write code and answer questions without being separately trained for each.

Remember

* LLMs are not magic. They are extremely good next-token predictors.

*** Key Idea**

One objective.
Many abilities.
Emergence comes from scale.

Interview edge: name the objective explicitly. Saying

* "it is trained to predict the next token, and everything else is an emergent side effect of doing that at scale" instantly signals you understand the mechanism, not the marketing.



Section 1: Foundations

* 2. How are LLMs different from traditional language models?

Traditional language models (n-gram models, and later RNNs and LSTMs) were narrow and mostly task-specific. They had a short effective memory, were trained on modest datasets, and you generally trained a fresh model per task.

Remember

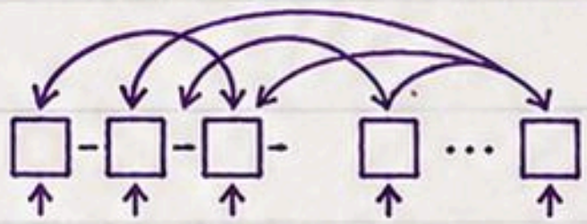
* LLMs are different across multiple axes, not just bigger versions of old models.

→ LLMs differ on four axes:

① Scale of parameters and data.



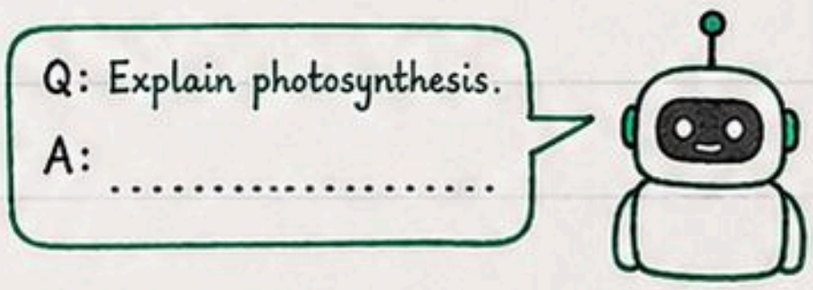
② Transformer architecture with self-attention instead of recurrence.



③ Self-supervised pretraining followed by transfer to many tasks.



④ In-context learning, which means you can change the model's behaviour with a prompt instead of retraining it.



Interview edge: that last point is the real break in the timeline.

Before LLMs, adapting a model meant a training job.

Now it means writing a better prompt.

* That shift is why your role exists.



Section 1: Foundations

* 3. What are Foundation Models?

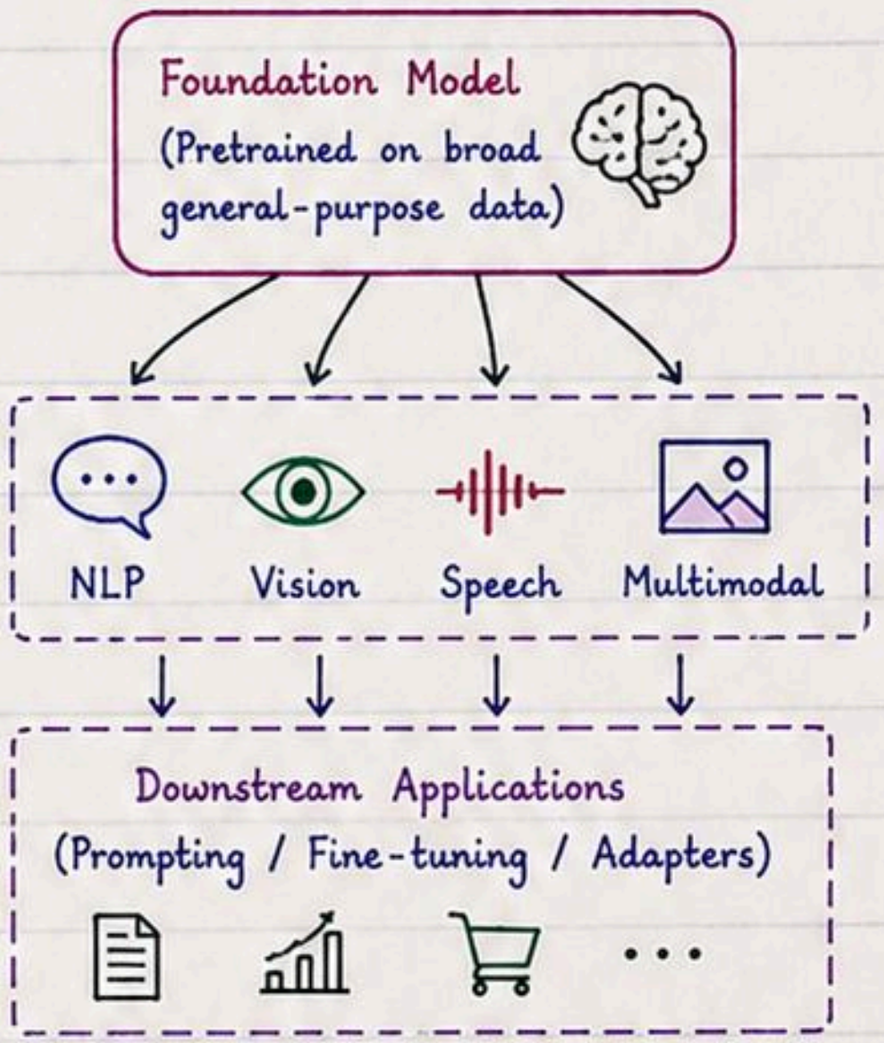
A foundation model is a large model pretrained on broad, general-purpose data that then serves as the shared base for many downstream applications, through prompting, fine-tuning or adapters.

Remember

* Pretraining is expensive.

Building on top of it is cheap.

→ LLMs are the text-side example, but the category is wider: it includes vision models, speech models and multimodal models.



→ The economic point is that the expensive part, pretraining, happens once, and thousands of teams build cheaply on top of it.

Interview edge: if you are interviewing for an enterprise role, add the sourcing angle. "Foundation model" is the layer where the build-versus-buy decision actually gets made,

* which is why most Indian enterprises consume foundation models through an API rather than pretraining their own.



Section 1: Foundations

* 4. How is GPT-4 different from GPT-3 in capabilities and applications?

The practical differences are multimodality, context length, and reliability of reasoning. GPT-3 was text-in, text-out with a short context window. GPT-4 accepts images alongside text, handles far longer inputs, follows complex multi-step instructions much more consistently, and hallucinates less on factual and professional-exam style tasks.

Remember

The core upgrades:

* Multimodality
* Longer context
* Stronger reasoning
* Lower hallucination

→ In application terms, GPT-3 was good for generation and completion; GPT-4 opened up document understanding, multi-step agentic workflows and use cases where being wrong actually costs money.

Aspect	GPT-3	GPT-4
Input	Text only	Text + Images
Context Window	Short	Much longer
Reasoning & Instructions	Basic to moderate	Much stronger, more reliable
Hallucination	Higher	Lower
Typical Use Cases	Generation, completion	Docs, analysis, agents, high-stakes workflows

Interview edge: avoid quoting parameter counts, because OpenAI never published GPT-4's.

* Candidates who confidently state a number lose credibility.

Talk about capability deltas instead.



Section 1: Foundations

* 5. What is tokenization, and why is it important in LLMs?

Tokenization is the step that converts raw text into the discrete units a model can actually process, then maps each unit to an integer ID. Modern LLMs use subword tokenization, so common words are single tokens while rare words split into fragments.

Remember

* Tokens are the currency of LLMs.
* Same meaning, different token count across languages.

→ It matters for three very practical reasons:

① Your context window is measured in tokens.

② Your API bill is measured in tokens.

③ Tokenizer efficiency varies wildly across languages.

How tokenization works (example)

Text: Unbelievable!

Tokens: [Un | believable | !]

Token IDs: [5021 | 21345 | 999]

↑ subword fragment | ↑ subword fragment | ↑ punctuation token

Same sentence, different token counts

Sentence meaning: The future belongs to those who learn.	English	Hindi	Tamil	Kannada
	The future belongs to those who learn.	भविष्य उन लोगों का है जो सीखते हैं।	கற்றவருக்கே எதிர்காலம் சொந்தம்.	ಕಲಿತವರಿಗೇ ಭವಿಷ್ಯ.
	Tokens: 7	Tokens: 16–20	Tokens: 22–28	Tokens: 20–26

→ Indic scripts typically consume 2x–4x more tokens than English.

Interview edge: the language point is a strong differentiator in Indian interviews.

* The same sentence in Hindi, Tamil or Kannada often consumes two to four times the tokens of its English equivalent, because these scripts are under-represented in tokenizer training.

→ If you are building anything multilingual, that is a direct cost and context-limit problem, and mentioning it shows you have shipped rather than studied.



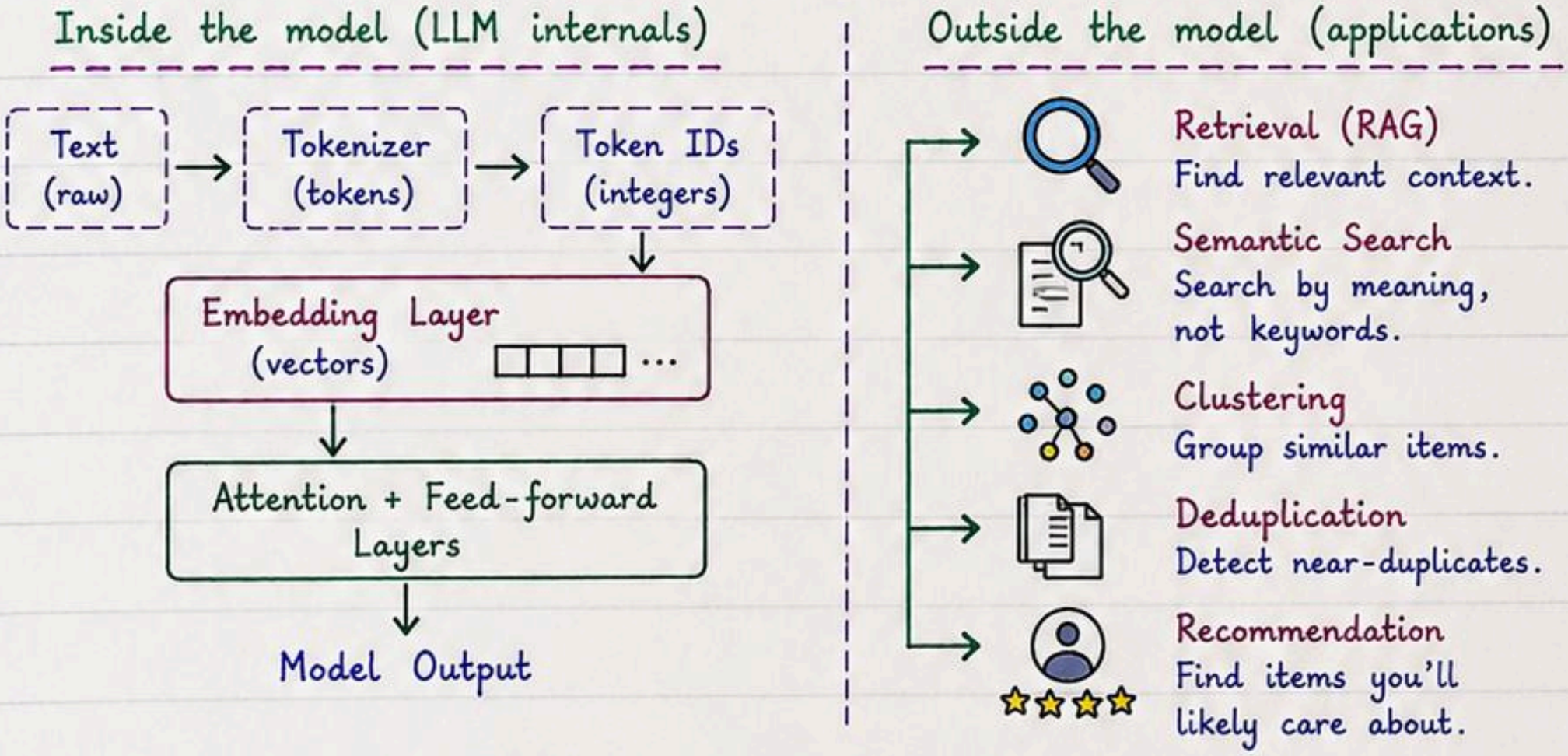
Section 1: Foundations

* 6. What role do embeddings play in LLMs?

Embeddings are dense vectors that place tokens, sentences or documents in a geometric space where distance approximates semantic similarity. Inside the model, the embedding layer is the entry point: token IDs become vectors that every subsequent attention and feed-forward layer transforms. Outside the model, embeddings power the retrieval half of RAG, semantic search, clustering, deduplication and recommendation.

Remember

* Embeddings turn meaning into math.
* Distance in vector space ≈ semantic similarity.



Interview edge:

1. Internal token embeddings are learned during pretraining and are part of the model.
2. Retrieval embeddings come from a separate embedding model you choose and can swap.

* Mixing these up is one of the most common ways candidates reveal they have only read about RAG.



Section 1: Foundations

* 7. How do LLMs handle out-of-vocabulary (OOV) words?

They largely sidestep the problem. Classical models had a fixed word vocabulary and mapped anything unseen to a single UNK token, destroying information. Modern LLMs use subword algorithms such as Byte Pair Encoding, WordPiece or SentencePiece, so an unfamiliar word is decomposed into known sub-units. Byte-level BPE goes further and falls back to raw bytes, which means nothing is ever truly out of vocabulary.

Remember

* No single UNK token.
* Use subword units.
* Byte-level BPE means no true out-of-vocabulary.

Classical models (word-level)

Fixed vocabulary

Unseen word → [UNK]

Information lost! ✗

Modern LLMs (subword / byte-level)

Open-ended vocabulary

Unfamiliar word → known sub-units

Information preserved! ✓

→ Examples of how words are tokenized

Input Word	WordPiece	BPE	Byte-level BPE
Learnbay	learn ##bay	Learn bay	Learn bay
Shrivastava	Shr ##iv ##ast ##ava	Shriv ast ava	Shriv ast ava
naïveté	naï ##vet ##é	naï vet é	6E 61 C3 AF 76 65 74 C3 A9
🚀 RocketX	🚀 Rock ##et ##X	🚀 Rocket X	F0 9F 9A 80 52 6F 63 6B 65 74 58

Interview edge: Give a concrete example.

A product name like "Learnbay" or a surname the tokenizer has never seen simply splits into pieces the model can still reason over.



* The cost is not failure, it is token inflation and slightly weaker representations for rare terms.

Section 1: Foundations

* 8. What are Sequence-to-Sequence (Seq2Seq) Models?

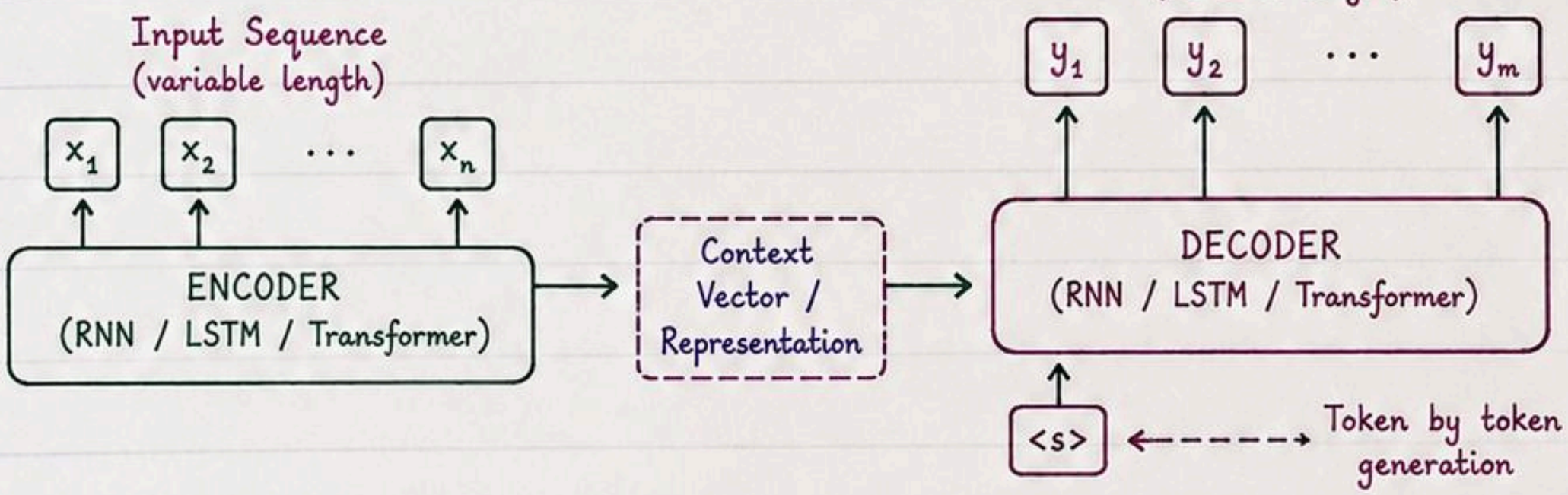
Seq2Seq is an architecture pattern for tasks where a variable-length input maps to a variable-length output: translation, summarisation, question answering. An encoder reads the full input and compresses it into a representation, and a decoder generates the output token by token conditioned on that representation.

It originated with RNN and LSTM pairs and now survives in transformer form in models like T5 and BART.

Remember

* Variable-length input to variable-length output.
* Encoder compresses. Decoder generates.
* Used in translation, summaries, QA, etc.

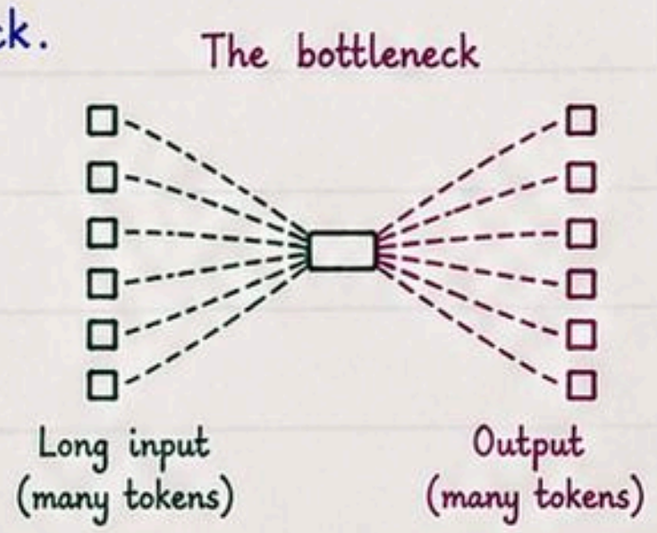
How Seq2Seq works



Interview edge: name the original bottleneck.

* Early Seq2Seq squeezed an entire sentence into one fixed-size vector, which broke down on long inputs.

→ Attention was invented specifically to relieve that bottleneck.



Section 1: Foundations

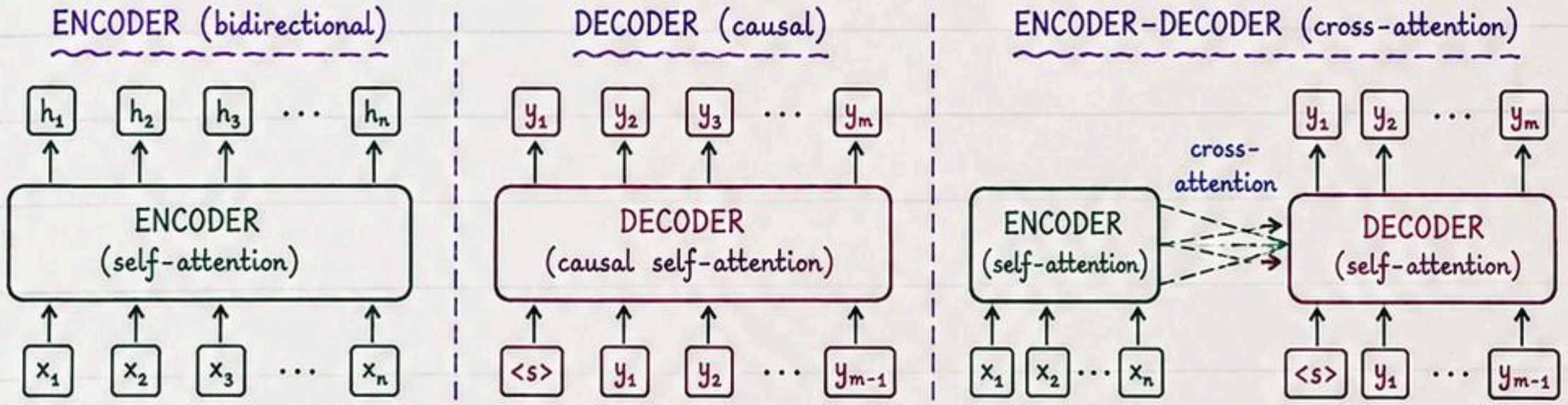
* 9. What is the difference between an encoder and a decoder?

An encoder processes the whole input at once with bidirectional attention, so every position sees both left and right context. Its job is to build a rich representation, which makes it ideal for classification, extraction and embedding tasks. BERT is encoder-only.

A decoder attends causally, meaning each position sees only previous positions, because it has to generate the next token without peeking at the answer. GPT-style models are decoder-only. Encoder-decoder models keep both, with cross-attention connecting them.

Remember

* Encoder = understand the whole.
* Decoder = generate step by step.
* Encoder-Decoder = transform.



- Sees both left and right context.
- Used for understanding tasks (classification, extraction, embedding).

Example: BERT

- Sees only previous positions.
- Used for generation tasks (next token prediction).

Example: GPT

- Encoder builds representation.
- Decoder generates using cross-attention.
- Used for transformation tasks (translation, summarisation, QA).

Examples: T5, BART

Interview edge: map each family to a job.

* Encoder-only → understanding (read, classify, extract, embed).
* Decoder-only → generation (write, chat, code, create).
* Encoder-Decoder → transformation (translate, summarise, convert).

Then note that the industry consolidated on decoder-only for general-purpose LLMs because generation is the more general interface.



Section 1: Foundations

* 10. How do autoregressive models differ from masked models?

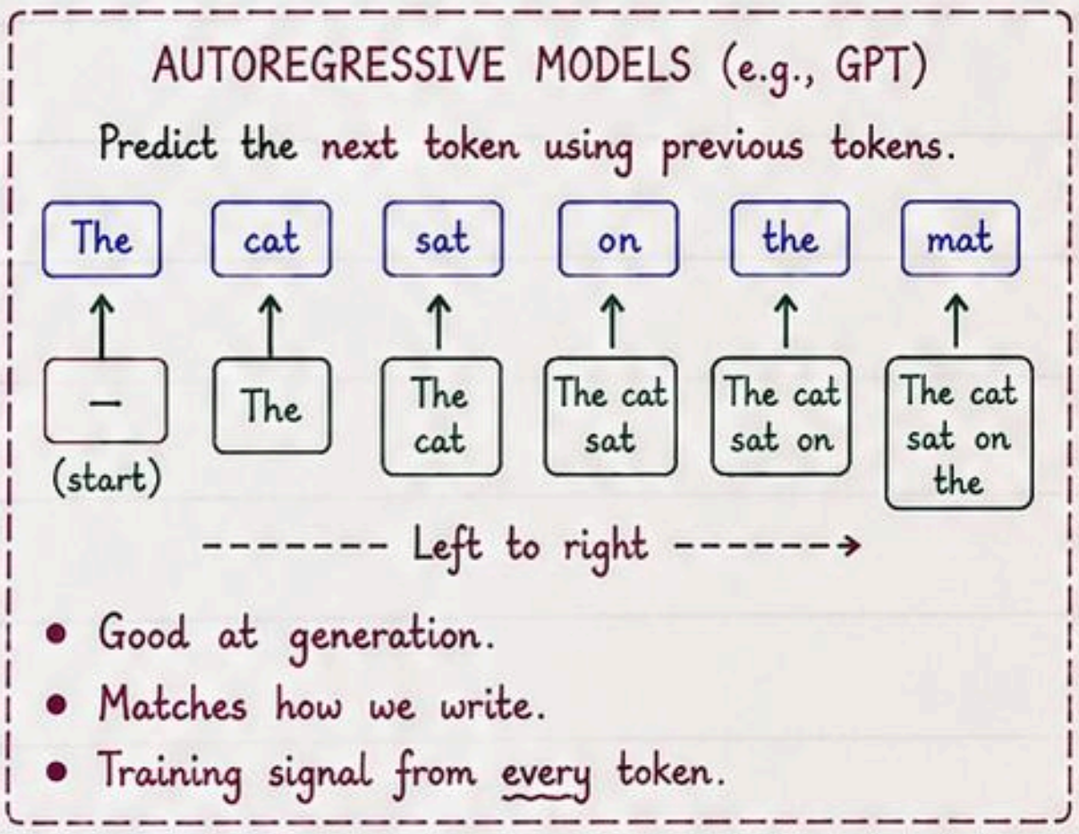
Autoregressive models factorise the sequence probability left to right and predict the next token given all previous ones. That objective matches generation exactly, so these models write fluent text.

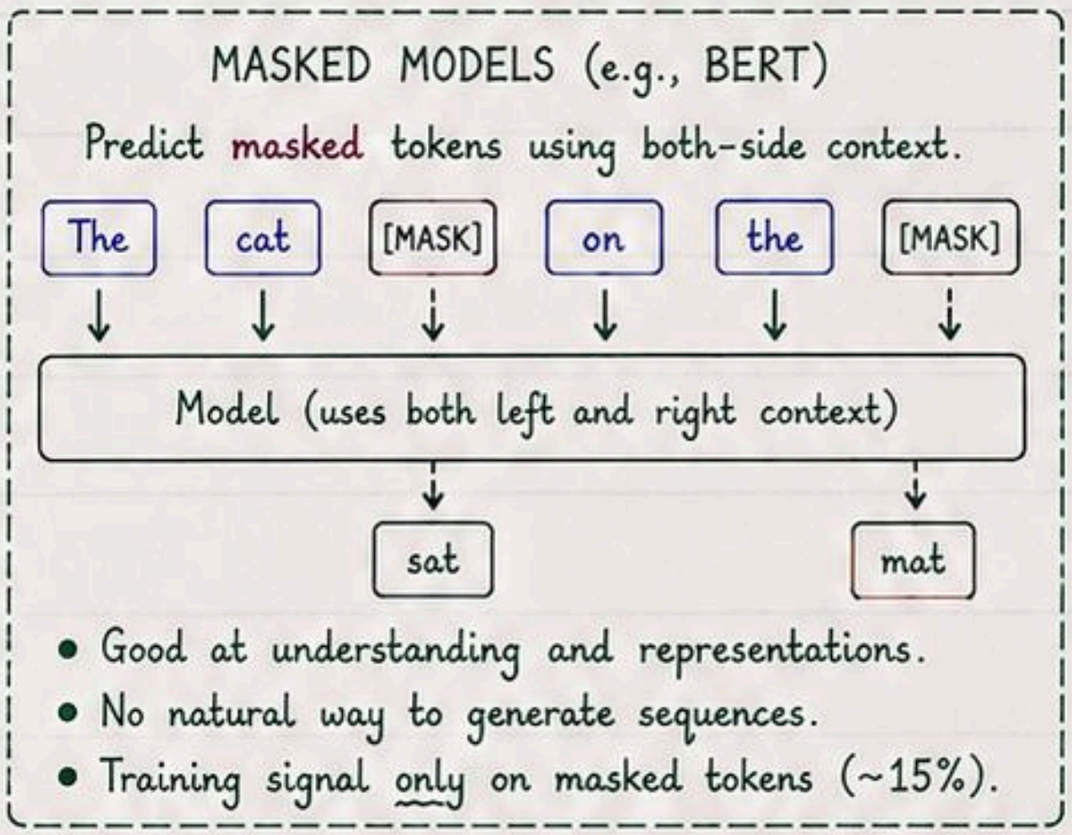
Masked language models hide a fraction of tokens and predict them using context from both directions, which produces stronger representations for understanding but no natural way to generate.

In short: autoregressive optimises for writing, masked optimises for reading.

Remember

* Autoregressive = predict next token using past context.
* Masked = predict missing token using both-side context.
* Different objectives, different strengths.



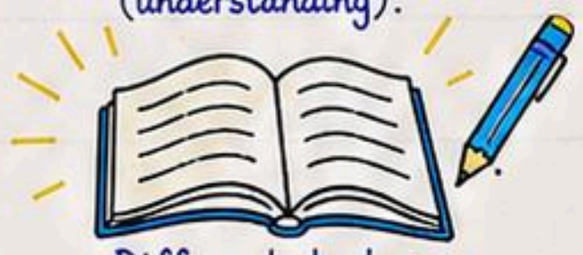


→ Interview edge: training efficiency

Aspect	Autoregressive Models	Masked Models
Objective	Predict next token (left-to-right)	Predict masked token (using both sides)
Learning signal	From every token (100% of tokens)	Only from masked tokens (~15% of tokens)
Training efficiency	High	Lower
Why it matters	More learning per token → scales better, needs less data for same amount of learning.	Less learning per token → needs more data/compute for same learning.

Bottom line

Autoregressive = writing (generation).
Masked = reading (understanding).



Different tools.
Different jobs.
Both are useful.

Section 2: Pretraining and Architecture

* 11. What is masked language modeling, and how does it help pretraining?

Masked language modeling (MLM) randomly hides a portion of input tokens, typically around 15%, and trains the model to reconstruct them from surrounding context.

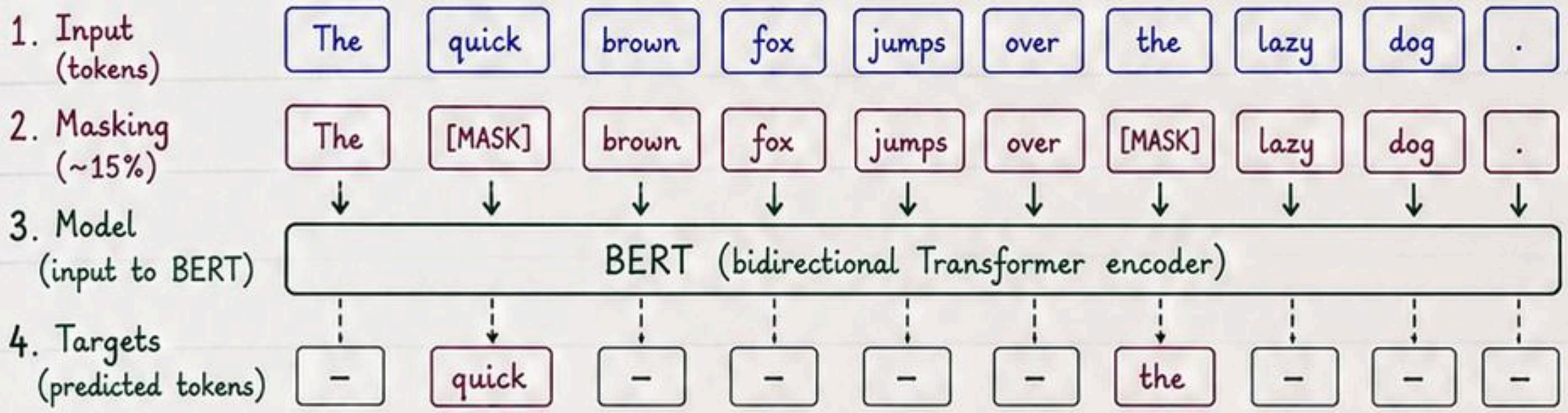
Because the model can look both left and right, it is forced to build genuinely bidirectional representations of meaning rather than just predicting forward.

This gives you a general-purpose language understanding backbone that can be fine-tuned on classification, named entity recognition or retrieval with relatively little labelled data.

Remember

* Hide ~15% tokens.
* Use both left and right context.
* Builds general understanding (representations).

How Masked Language Modeling works



BERT's masking strategy (for each selected token)

80% Replace with [MASK] token

Helps the model learn to use context and rely less on the actual token.

Example: The [MASK] brown

10% Replace with random token

Prevents the model from over-relying on [MASK] and improves robustness.

Example: The cat brown

10% Keep the token unchanged

Further reduces the mismatch between pretraining and fine-tuning.

Example: The quick brown

Interview edge:

* BERT does not simply replace every selected token with [MASK].
* It uses the 80/10/10 rule: [MASK] / random token / unchanged.
* This design reduces the gap between pretraining and fine-tuning, where [MASK] tokens never appear, leading to better performance at downstream tasks.



Section 2: Pretraining and Architecture

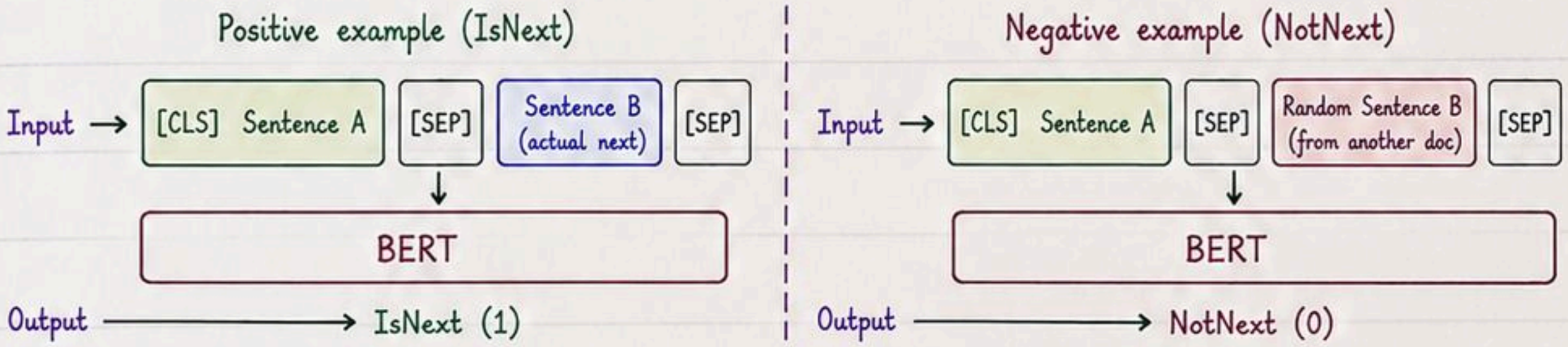
* 12. What is Next Sentence Prediction (NSP)?

Next Sentence Prediction (NSP) was BERT's second pretraining objective. Given two segments, the model classifies whether the second sentence genuinely follows the first in the source document (IsNext) or was randomly sampled from another document (NotNext). The intent was to teach discourse-level and inter-sentence relationships, which were expected to help downstream tasks such as Natural Language Inference (NLI) and Question Answering (QA).

Remember

* NSP = binary classification (IsNext vs NotNext).
* Trains the model to understand relationships between sentences.
* Aimed at improving NLI, QA and other tasks that need discourse understanding.

How NSP works (during pretraining)



What happened to NSP?

- RoBERTa (Liu et al., 2019) ran ablations and found that removing NSP and training on longer, contiguous sequences consistently improved performance.
- The extra objective made training slower and didn't provide meaningful gains.
- Most later models (RoBERTa, DeBERTa, ELECTRA, DistilBERT, ALBERT, etc.) dropped NSP.
- Some models replaced it with harder or more effective objectives like Sentence Order Prediction, Token Distance Prediction, or Contrastive objectives.

Timeline

- **BERT (2018)** — Introduced NSP.
- **RoBERTa (2019)** — Ablations show NSP not useful. Removed NSP. Trained on longer sequences.
- **After 2019** — NSP largely abandoned or replaced by stronger objectives.

Interview edge:

* The well-read answer is not just the definition, but the story: NSP was a good idea, tested, found not to help, and then removed. This shows you follow the literature, not just the first tutorial you saw.



Section 2: Pretraining and Architecture

* 13. How does the Transformer architecture overcome challenges of traditional Seq2Seq models?

The Transformer (Vaswani et al., 2017) replaces recurrence with self-attention and fixes three fundamental problems of traditional Seq2Seq models.

(1) Sequential computation bottleneck (no parallelism)
RNNs process tokens one by one, so training cannot parallelise across the sequence.
→ Self-attention processes all positions simultaneously.

✓ Enables massive parallelism → the key reason large-scale pretraining became possible and economical on GPUs / TPUs.

(2) Long-range dependency degradation (vanishing gradients)
RNNs must carry information through many sequential steps, so useful signals fade.
→ Attention gives any position direct access to any other in a single hop.

✓ Captures long-range dependencies much more effectively.

(3) Fixed-size context vector bottleneck
Seq2Seq encoders compress the entire input into one fixed-size vector.
→ Transformer decoder attends over all encoder states (not a single summary vector).

✓ No information bottleneck; the model can leverage the full set of representations.

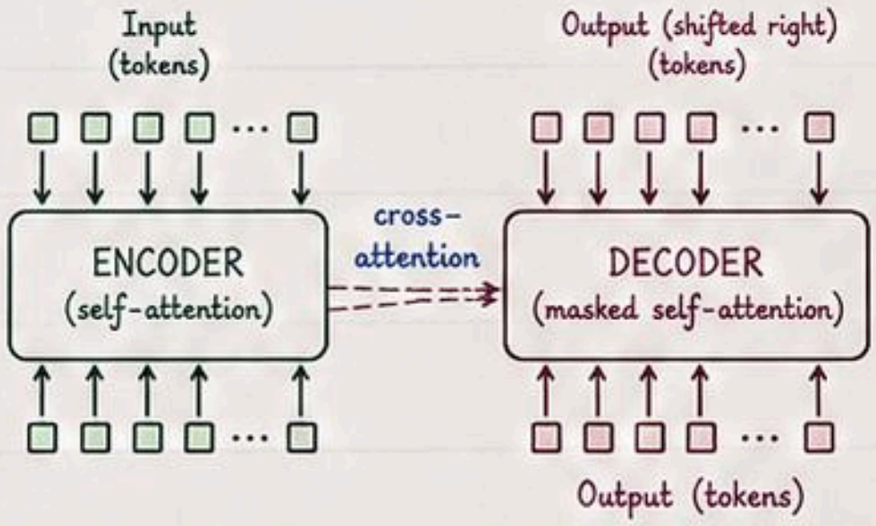
Remember

* Transformers eliminate recurrence and replace it with attention.
* Parallel, deep, and directly connected anywhere.
* That's the architectural shift that changed everything.

RNN / Seq2Seq vs Transformer

Aspect	Traditional Seq2Seq (RNN/LSTM)	Transformer
Computation	Sequential (token-by-token)	Fully parallel across sequence (self-attention)
Long-range dependencies	Hard to capture; vanishing gradients	Direct path between any two positions (one-hop access)
Context representation	Single fixed-size vector (bottleneck)	Attention over all encoder states (no bottleneck)
Training speed	Slow (cannot parallelise)	Much faster (parallelisable)
Scalability	Limited	Highly scalable (enables large models)

Transformer Encoder-Decoder



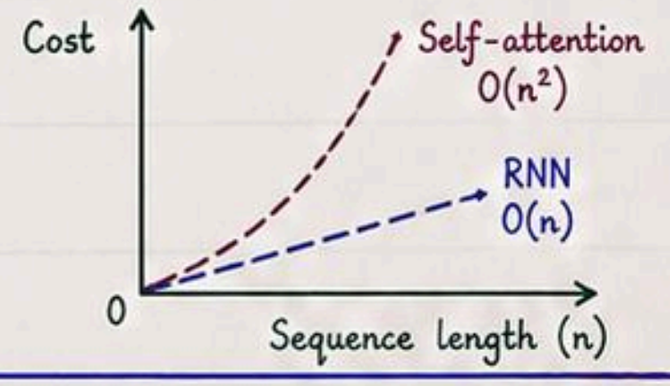
Interview edge:

The Transformer's strength comes with a cost.

* Self-attention is quadratic in sequence length: $O(n^2)$

This is why long-context work focuses on:

- FlashAttention
- Sparse / Sliding-window / Block attention
- KV cache optimisation for inference



Section 2: Pretraining and Architecture

* 14. What are positional encodings in LLMs?

Self-attention is permutation-invariant: if you shuffle the input tokens, the raw attention computation produces the same set of representations. Since word order carries meaning, position must be injected explicitly.

The original Transformer added fixed sinusoidal signals to input embeddings. Later models used learned absolute positions, and current models overwhelmingly use relative schemes such as RoPE (Rotary Position Embedding), which rotates query and key vectors by position, or ALiBi (Attention with Linear Biases), which biases attention scores by distance.

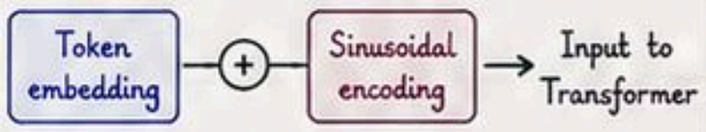
Remember

* Self-attention alone has no notion of order.
* Positional encodings inject order information.
* Different choices = different extrapolation and generalisation behaviour.

Ways to encode position

1. Sinusoidal (original Transformer)

Fixed functions of position added to token embeddings.



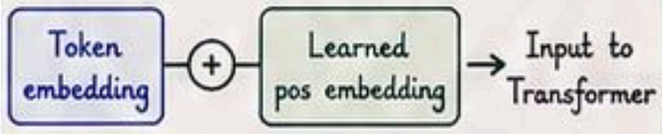
Formula (for dimension 2i and 2i+1):

$$PE_{pos,2i} = \sin\left(pos / 10000^{2i/d_{model}}\right)$$

$$PE_{pos,2i+1} = \cos\left(pos / 10000^{2i/d_{model}}\right)$$

2. Learned Absolute Position

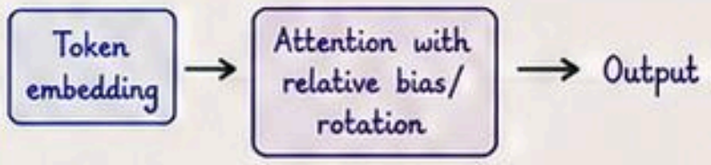
Each position has a learned vector that is added to token embeddings.



- Simple and effective.
- Does not extrapolate well beyond training length.

3. Relative Position (modern default)

Encode position as a relative property between tokens.



- RoPE: rotates Q and K vectors by position-dependent angles.
- ALiBi: adds a linear bias to attention scores based on distance.

Intuition: why relative works better

Relative methods depend on distance $(i - j)$, not the absolute position index.



⇒ Relative encodings generalise to longer sequences.

Connection to context length

- Learned absolute positions struggle to extrapolate.
- RoPE extrapolates gracefully and preserves quality.
- That's how models jump from 8k to 32k to 128k+.

Method	Extrapolation	Typical use today
Sinusoidal	Moderate	Rarely used now
Learned Absolute	Poor	Older models only
RoPE (relative)	Good	Default in GPT-4o, Llama 3, Mistral, etc.
ALiBi (relative)	Good	Used in some long-context models

RoPE scaling (e.g., NTK scaling, YaRN) extends usable context (e.g., 8k → 128k) without full retraining.

Interview edge:

* Positional encodings are how models know word order.
* Relative schemes (RoPE / ALiBi) are the reason long-context LLMs are possible.
* The choice impacts extrapolation, efficiency, and model quality at scale.



Section 2: Pretraining and Architecture

* 15. What is attention in Transformer models?

Attention is a learned, content-based lookup. For every token, the model produces a **query**, and every token also produces a **key** and a **value**.

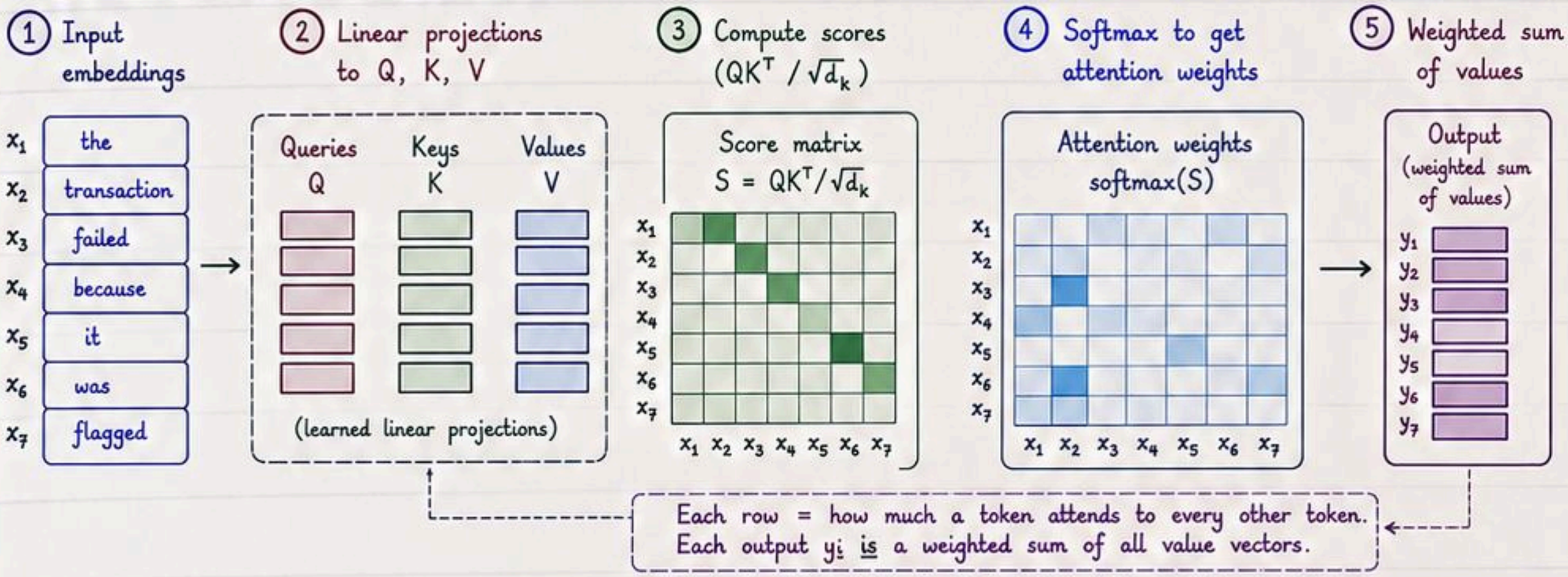
The query is compared against all keys to score relevance, those scores are normalised into **weights**, and the output is the weighted sum of **values**.

The effect is that each token builds its representation by pulling selectively from whichever other tokens are relevant to it, rather than from a fixed window.

Remember

* Attention lets each token "look" at all other tokens.
* Relevance is learned, not based on distance.
* Different heads learn different kinds of relationships.

How attention works (single head)



Why attention matters - a resolution example

Sentence:

"the **transaction** failed because **it** was flagged."

↑ what the event is ↑ needs a referent

The model must figure out what "it" refers to.
Attention lets the representation of "it" place high weight on "transaction" (and related tokens), so the model links them correctly.

Attention from token "it" (x_5)

the	transaction	failed	because	it	was	flagged
x_1	x_2	x_3	x_4	x_5	x_6	x_7

Attention weights from "it" to all tokens →

0.02	0.68	0.06	0.05	0.03	0.06	0.10

↑ Highest weight (most relevant)

↑ Some attention to other useful words

Interview edge:

* Attention is the core mechanism that enables long-range dependencies.
* It is content-based, fully differentiable, and learned end-to-end.
* Multi-head attention = multiple independent attention heads running in parallel, allowing the model to capture diverse relationships.



Section 2: Pretraining and Architecture

* 16. What is Multi-head attention?

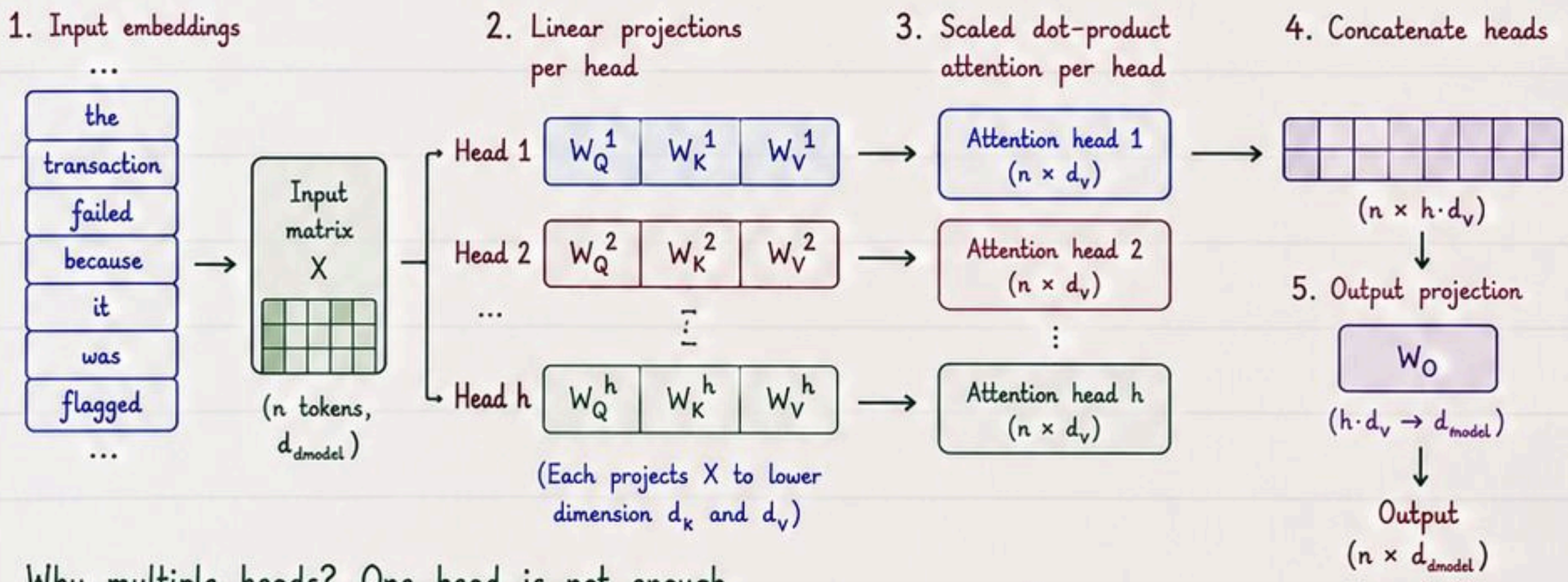
Multi-head attention (MHA) allows the model to attend to information from different representation subspaces at the same time.

Each head has its own set of parameters that produce independent queries, keys and values. Heads run attention in parallel, the results are concatenated and linearly projected back to the original dimension.

Remember

* Split → attend in parallel → concatenate → project.
* Heads specialise in different types of relationships.
* Enables the model to capture multiple patterns simultaneously.

How Multi-head attention works



Why multiple heads? One head is not enough.

A single head produces one attention distribution per token. It must choose one pattern.

Token: it (x_5)

the x_1	transaction x_2	failed x_3	because x_4	it x_5	was x_6	flagged x_7

One head must split its probability mass.

0.08	0.35	0.05	0.22	0.05	0.10	0.15

Compromise: cannot strongly focus on multiple important tokens at once.

Multiple heads allow different attention patterns in parallel. Each head specialises.

Head A (coreference)

0.02	0.85	0.03	0.03	0.02	0.02	0.03

Head B (syntax)

0.02	0.03	0.85	0.03	0.02	0.02	0.03

Head C (locality / punctuation)

0.10	0.10	0.10	0.10	0.10	0.10	0.40

Together, the model captures coreference (transaction), syntax (failed), and punctuation / local context (,) simultaneously.

Interview edge:

* One softmax distribution per token forces a single relevance pattern.
* Multiple heads let the model attend to the subject and the verb and the punctuation at the same time.
* Empirical studies have found heads that track syntax, coreference, positional information, and even "dead" heads that contribute little.
* This diversity is a major reason Transformers are so powerful.



Section 2: Pretraining and Architecture

* 17. How are attention scores computed in transformers?

Attention takes a query (what I'm looking for) and compares it against all keys (what others offer). The similarity scores become weights that decide how much information to pull from each value (what information is passed).

Remember

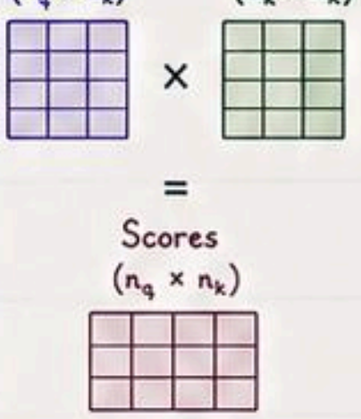
- Q = Queries
- K = Keys
- V = Values
- d_k = dimension of keys

Step-by-step computation (scaled dot-product attention)

① Compute scores (QK^T)

Each query with each key.

Q $(n_q \times d_k)$ × K^T $(d_k \times n_k)$ = Scores $(n_q \times n_k)$

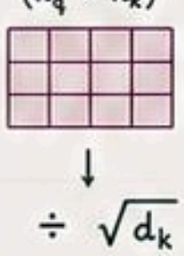


Raw similarity scores.

② Scale

Divide by $\sqrt{d_k}$ to control variance.

Scores $(n_q \times n_k)$ → ÷ $\sqrt{d_k}$



Keeps values in a healthy range.

③ Mask (if needed)

In decoder, prevent attention to future positions.

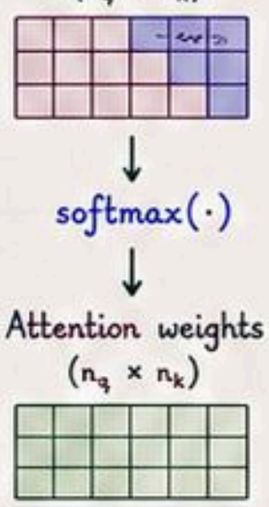
Scaled scores $(n_q \times n_k)$ → + Causal mask (future = $-\infty$) → Masked scores $(n_q \times n_k)$



④ Softmax

Apply softmax across the key axis (rows).

Masked scores $(n_q \times n_k)$ → softmax(·) → Attention weights $(n_q \times n_k)$

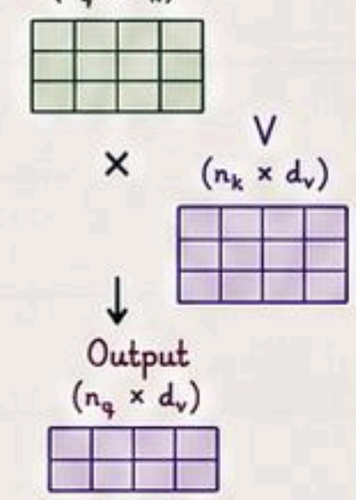


Each row sums to 1. (distribution over keys)

⑤ Weighted sum with values

Multiply weights by values and sum.

Attention weights $(n_q \times n_k)$ × V $(n_k \times d_v)$ → Output $(n_q \times d_v)$



Final attended representations.

The whole formula (in matrix form)

$$\underbrace{\text{Attention}(Q, K, V)}_{(n_q \times d_v)} = \underbrace{\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)}_{(n_q \times n_k)} \underbrace{V}_{(n_q \times d_v)}$$

Where:

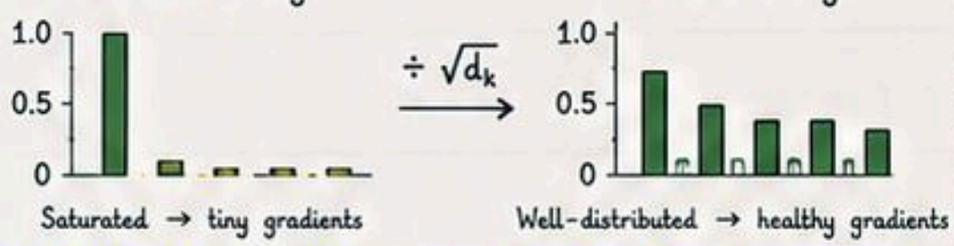
- $Q \in \mathbb{R}^{n_q \times d_k}$ (queries)
- $K \in \mathbb{R}^{n_k \times d_k}$ (keys)
- $V \in \mathbb{R}^{n_k \times d_v}$ (values)
- M is the mask (0 for allowed, $-\infty$ for masked positions)

For encoder: $M = 0$ (no mask)

For decoder: M = causal mask

Why do we divide by $\sqrt{d_k}$?

As dimension d_k grows, the dot products QK^T have larger variance. This makes softmax push towards extreme values (one weight ≈ 1, others ≈ 0).



* Scaling keeps softmax in a learning-friendly regime.

Causal mask example (sequence length = 6)

Mask (add to scores)

	1	2	3	4	5	6
1	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$
2	0	0	$-\infty$	$-\infty$	$-\infty$	$-\infty$
3	0	0	0	$-\infty$	$-\infty$	$-\infty$
4	0	0	0	0	$-\infty$	$-\infty$
5	0	0	0	0	0	$-\infty$
6	0	0	0	0	0	0

0 — allowed (can attend)

$-\infty$ — masked (cannot attend)

After softmax, all $-\infty$ become 0 weight.

Interview edge:

* Attention = softmax $\left(QK^T)/\sqrt{d_k}$ mask$\right)$ V.
* Scaling by $\sqrt{d_k}$ prevents saturation and keeps gradients alive.
* The causal mask is what makes decodor attention autoregressive.



Section 2: Pretraining and Architecture

* 18. What is the softmax function and why is it used in attention?

Softmax converts a vector of real numbers (scores) into a probability distribution: every value is positive and the whole vector sums to one.

In attention, softmax turns raw relevance scores into weights. These weights tell us "how much to pay attention" to each token when building the output as a weighted average of values.

Softmax is smooth and differentiable everywhere, so gradients flow through it during training.

Remember

* Softmax outputs are in (0, 1) and sum to 1.
* This makes attention weights interpretable as probabilities.
* Weighted sum with these weights = expected value over all tokens.

Defintion

For a score vector $s = [s_1, s_2, \ldots, s_n]$,

$$\text{softmax}(s_i) = \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} \quad \text{for } i = 1, \ldots, n$$

- e^{s_i} makes all outputs positive.
- Dividing by the sum ensures $\sum_i \text{softmax}(s_i) = 1$.

Example

Scores: $s = [2.0, 1.0, 0.1, -1.0]$

Exponentials: $e^{s_i} = [7.39, 2.72, 1.11, 0.37]$

Sum = 11.59

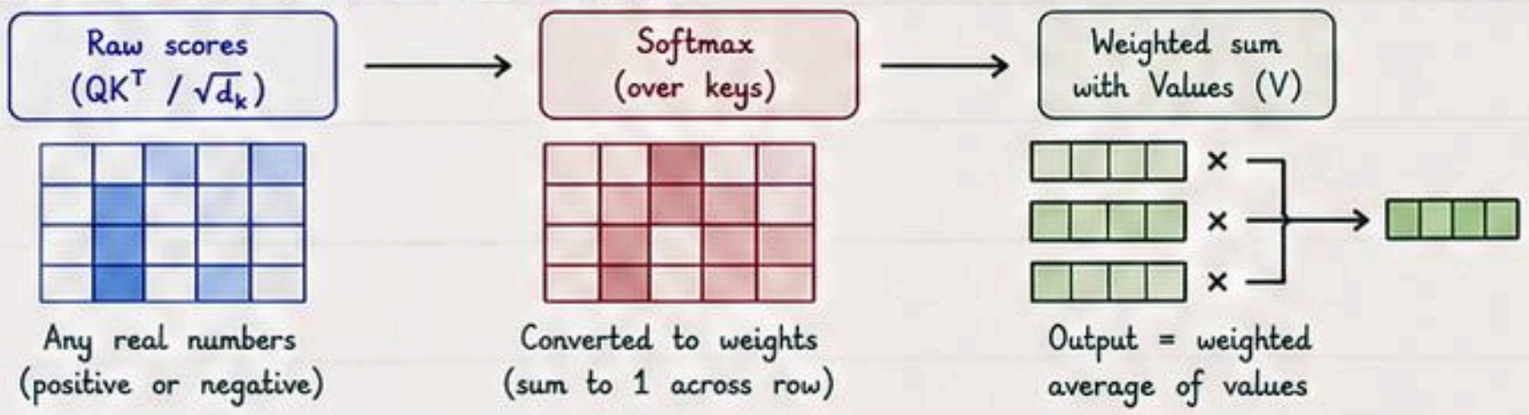
Softmax(s)	0.637	0.235	0.096	0.032

Highest score → gets most weight

Lowest score → gets small weight

All positive and sum to 1.00

Where softmax sits in attention



Interpretation

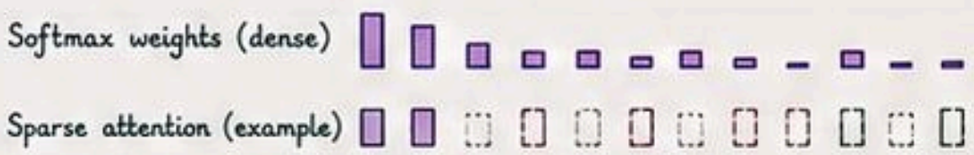
Attention output for a token is a weighted average over all tokens' value vectors, with softmax weights deciding how much to use each one.

Two key failure modes (interview edge)

① Softmax never assigns exactly zero

Every token gets at least a tiny bit of attention, even if irrelevant. This can waste capacity.

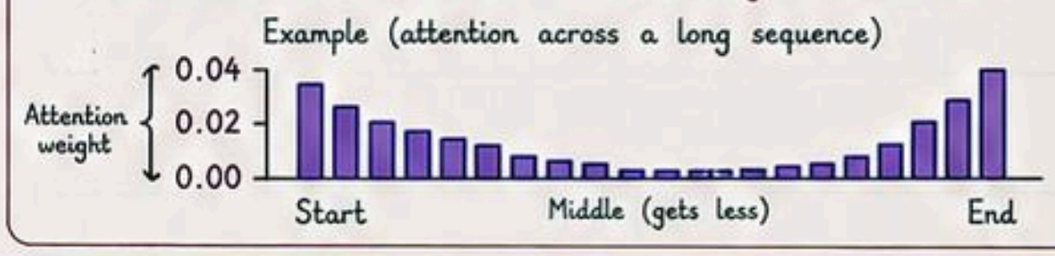
→ Motivation for sparse attention variants (Top-k, Sparsemax, Entmax, Reformer, etc.) that can produce exact zeros.



② Distribution can flatten in very long sequences

As sequence length grows, attention can spread thinly across too many tokens, diluting focus.

→ Contributes to "lost in the middle" in long-context models.



Interview edge:

* Softmax makes attention a probability distribution → clear interpretation.
* It is smooth and differentiable → gradients flow.
* But softmax attention is always dense and can flatten in long sequences.
* These limitations drive research into sparse and long-context attention methods.



Section 2: Pretraining and Architecture

* 19. How is dot product used in self-attention?

In self-attention, every token produces a query (Q), a key (K) and a value (V) vector. The dot product between a query and every key measures how similar they are.

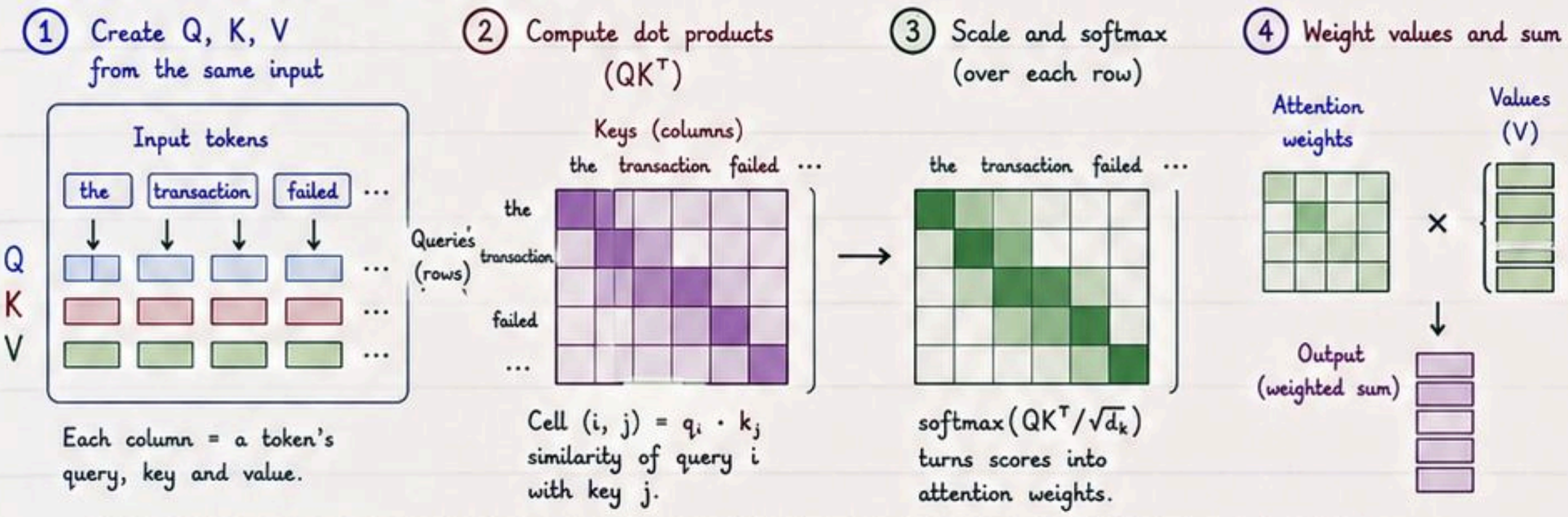
If two vectors point in similar directions, the dot product is large (high similarity). If they point in different directions, it is small or negative (low or no match).

Because Q, K and V all come from the same sequence, each token compares itself with all other tokens in that sequence — attending to its own neighbours.

Remember

* Dot product = similarity.
* Large positive → strong match (attend more).
* Around zero → neutral.
* Negative → not relevant (attend less).

How dot product drives self-attention

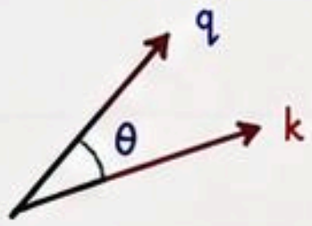


Each output is a weighted sum of value vectors, where weights come from dot-product similarities.

Why dot product?

Dot product is:

- High when directions align → high similarity.
- Low/negative when directions differ → low similarity.



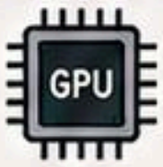
$$q \cdot k = \|q\| \, \|k\| \cos(\theta)$$

$\theta \approx 0°$	$\rightarrow \cos(\theta) \approx 1$	→ high
$\theta \approx 90°$	$\rightarrow \cos(\theta) \approx 0$	→ neutral
$\theta \approx 180°$	$\rightarrow \cos(\theta) \approx -1$	→ negative

Dot product asks: "How well does what I'm looking for (query) match what you offer (key)?"

Why did dot product win? (interview edge)

Dot product attention (scaled) won in practice because:



1. It is just matrix multiplication.
   Perfect for GPUs/TPUs and Tensor Cores.
   → Extremely fast and highly parallel.



2. Additive attention (Bahdanau style) needs a small neural network for every pair (i, j).
   → Far more compute and memory.



3. Quality was competitive, but efficiency decided the architecture.

Interview edge:

* Dot product = fast, simple, hardware-friendly similarity.
* It scales to long sequences and large models.
* Additive attention is more expressive but far slower.
* Engineering efficiency is why Transformers became the standard.



Section 2: Pretraining and Architecture

* 20. What is beam search, and how does it differ from greedy decoding?

Greedy decoding picks the single highest-probability token at every step and never looks back, so an early **mistake** is unrecoverable.

Beam search keeps the k most promising partial sequences alive at each step, expands all of them, and prunes back to k.

This increases the chance of finding a sequence with higher total probability than greedy would reach.

It costs roughly k times the compute and memory.

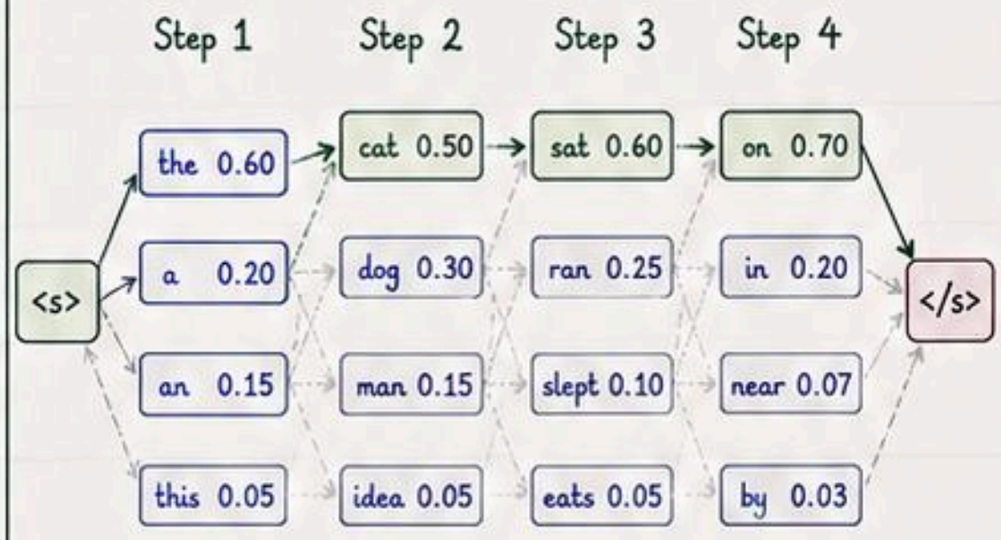
Remember

* Greedy: keep 1 best at each step.
* Beam search: keep top k candidates at each step.
* Beam search finds sequences with higher total probability.
* Cost ≈ k × (compute & memory).

Greedy decoding (k = 1)

Always picks the highest probability token.

Early mistake → cannot recover.

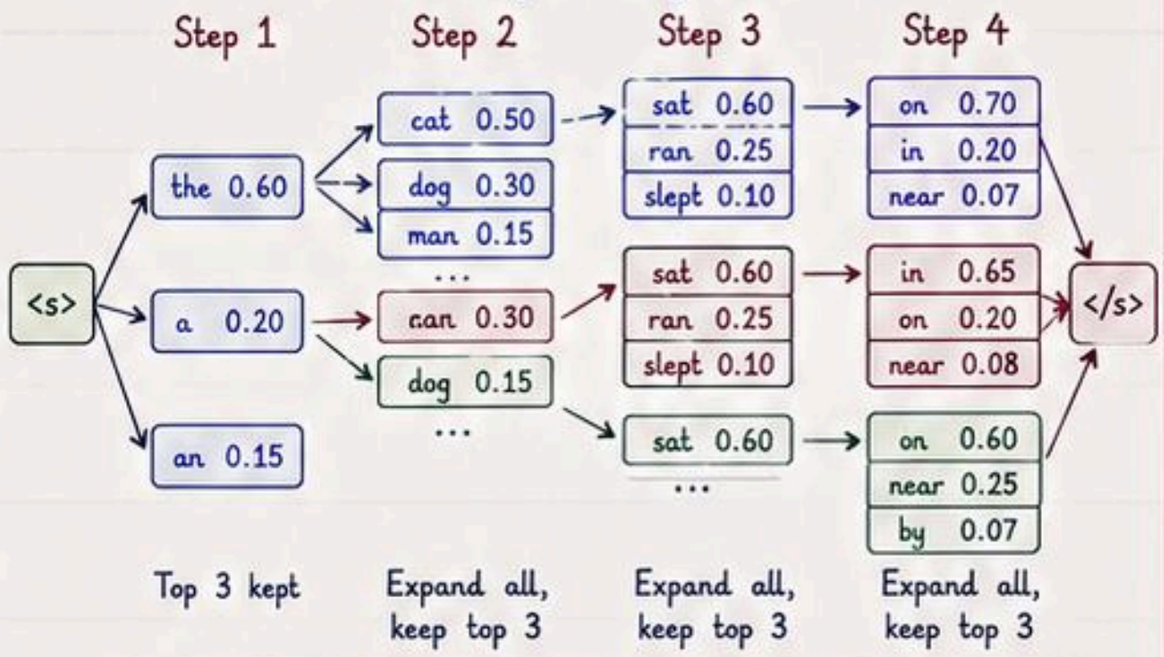


Output: the cat sat on </s>

Beam search (k = 3)

Keeps top k partial sequences at each step.

Can recover from early mistakes.



Best final output (example): the cat sat on </s>

Found because beam search explored multiple paths.

Where each belongs

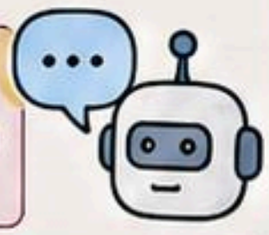
Beam search is standard when there is one broadly correct output.

* Machine translation — 猫 → cat
* Summarisation — Summary: ...
* Speech recognition / ASR — hello world
* OCR post-processing — Hello → Hello

Beam search is the wrong default for open-ended chat or creative generation.

* Produces bland, repetitive, over-generic text.
* Reduces diversity.
* Traps the model in locally safe paths.

What we use instead for chatbots:
Sampling with temperature + top-p (nucleus) sampling balances coherence with diversity.



Interview edge:

* Beam search = systematic exploration with a fixed-width frontier (size k).
* Greedy = frontier size 1.
* Beam search gives better quality for tasks with a single right answer.
* For open-ended chat, sampling > beam search for engaging, diverse responses.



Section 3: Decoding, Loss and the Math Layer

* 21. Explain the concept of temperature in LLM text generation.

Temperature (T) is a scaling factor applied to the logits just before softmax.

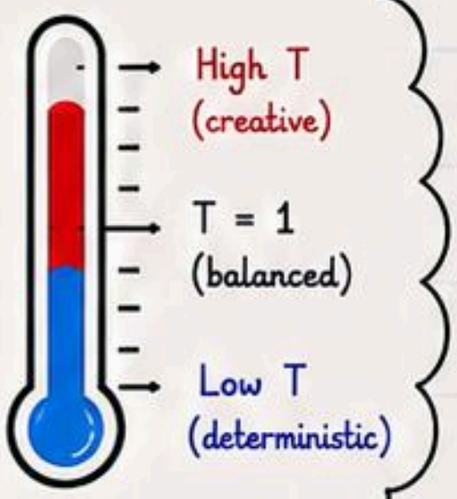
Given logits $z = [z_1, z_2, \ldots, z_n]$

$$p_i = \text{softmax}\left(\frac{z_i}{T}\right) = \frac{e^{z_i/T}}{\sum_{j=1}^{n} e^{z_j/T}}$$

- $T < 1$ → sharpen distribution (more confident)
- $T = 1$ → no change (default)
- $T > 1$ → flatten distribution (more random)
- $T \to 0$ → approaches greedy decoding (pick argmax)

Remember

* Applied to logits, not probabilities.
* Lower T concentrates probability mass on top tokens.
* Higher T spreads mass to tail tokens.
* T does not change what the model "knows" → only how bold or creative it is.

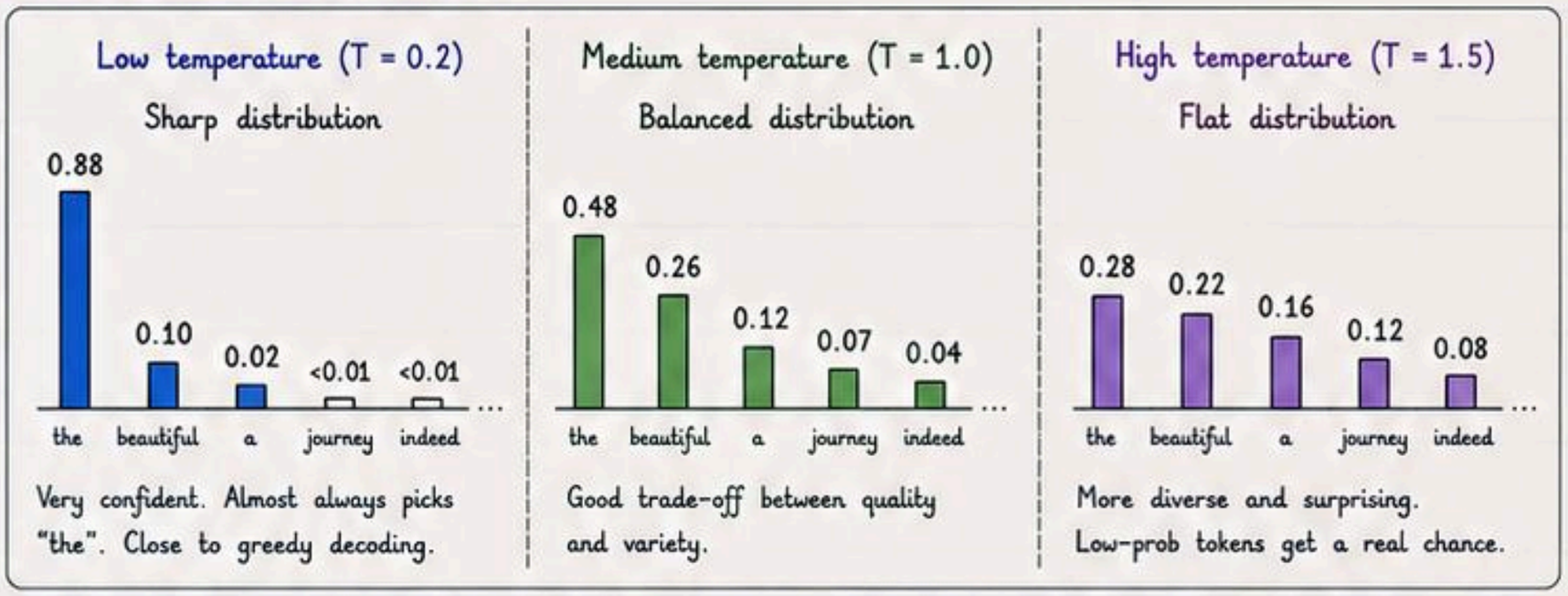


What temperature does to the output distribution

Example next-token logits (after the model)

the	3.20
beautiful	2.10
a	1.20
journey	0.30
indeed	−0.20
...	

÷ T then softmax →



Temperature changes the shape of the probability distribution, which changes which tokens are likely to be sampled.

Production guidance: pick T for the job

Use low T (0 – 0.3) for...

- Structured extraction (JSON, tables)
- SQL generation
- Classification / routing
- Code generation for execution
- Any output feeding a downstream system

Goal: correctness, consistency, determinism

Use higher T (0.7 – 1.0+) for...

- Ideation and brainstorming
- Marketing copy / creative writing
- Open-ended conversation
- Storytelling

Goal: diversity, creativity, exploration

The trap: T = 0 is not fully deterministic

Setting temperature to zero (or near zero) removes randomness from sampling, but outputs can still differ across runs due to:

- Floating-point non-determinism on GPUs
- Different batch sizes or padding
- Variations in kernel implementations
- Model parallelism / hardware differences

So T = 0 reduces variance but does not guarantee identical outputs.

Interview edge:

* Temperature is a simple but powerful control over creativity vs. reliability.
* Lower T → sharper, safer, more deterministic.
* Higher T → flatter, more diverse, more creative.
* Choose T based on the task and the downstream cost of mistakes.

Section 3: Decoding, Loss and the Math Layer

* 22. Difference between top-k sampling and top-p sampling?

Top-k keeps the k most likely tokens and samples only from them. Top-p (nucleus sampling) keeps the smallest set of top tokens whose cumulative probability reaches p, and samples from that set.

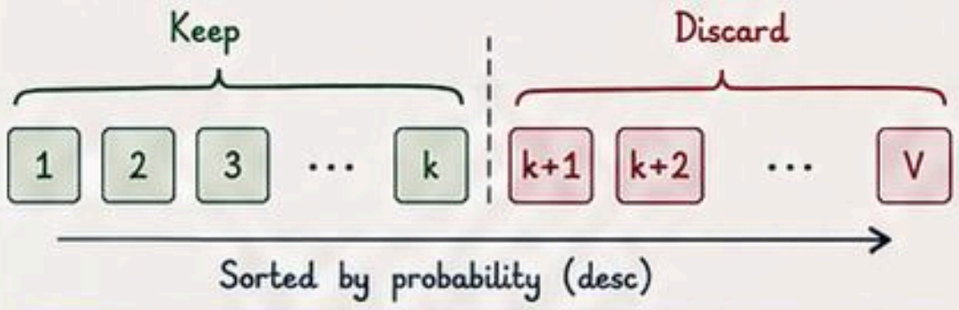
Top-k has a fixed candidate count. Top-p adapts to how confident the model is.

Remember

* Top-k: fixed size (k tokens), regardless of probabilities.
* Top-p: variable size, depends on cumulative probability threshold p.
* Both are sampling methods (not greedy).
* Usually used with temperature (T).

Top-k sampling

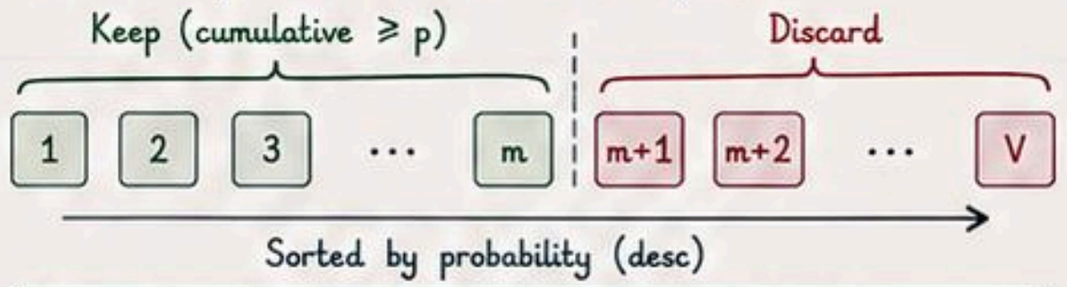
Keep the k highest-probability tokens.
Candidate pool size = k (always fixed).



Samples only from these k tokens.
Same number of options even if model is very confident.

Top-p (nucleus) sampling

Keep the smallest set of top tokens whose cumulative probability ≥ p.
Candidate pool size = variable (adapts).



Samples from the nucleus (smallest set reaching p).
Size m changes with confidence.

Example: prompt ends with "the capital of India is."

Well-trained model puts almost all probability on one token ("New").

Rank	Token	Probability	Cumulative Probability
1	New	0.94	0.94
2	Delhi	0.04	0.98
3	India	0.01	0.99
4	Mumbai	0.003	0.993
5	Bharat	0.002	0.995
...	...	...	...
50	(others)	<0.001 each	~1.00

→ **Top-k (k = 50)**
Forces 50 candidates into the pool.
Only 1 good token, 49 wrong ones.
You're sampling from a lot of noise.



→ **Top-p (p = 0.9)**
Cumulative probability hits 0.94 at the first token.
Pool size = 1 (just "New").



Top-p adapts: confident → small pool; unsure → larger pool.

When to use what? (production guidance)

Top-p (nucleus) + temperature (recommended default)

- ✅ General chat, creative writing, ideation
- ✅ Open-ended generation
- ✅ Better quality, more natural diversity
- ✅ Adapts to model confidence

Most modern LLM systems use Top-p + T.

Top-k (use with care)

- ❌ Useful when you need a hard cap on choices
- ❌ Or for very small models / constrained vocab
- ❌ Less adaptive → often less natural
- ❌ Usually worse than Top-p for open-ended tasks

Interview edge:

* Top-k: fixed number of candidates (k).
* Top-p: smallest set whose cumulative probability ≥ p (adaptive).
* Top-p is default and pairs with temperature.
* Example: "the capital of India is." → Top-p narrows to one correct token.



Section 3: Decoding, Loss and the Math Layer

* 23. How does Adaptive Softmax speed up LLMs?

The final softmax over a vocabulary of 50k–250k tokens is very expensive: every decoding step needs a score for every token.

Adaptive Softmax (Grave et al., 2017) exploits the Zipfian distribution of word frequency. It splits the vocabulary into a small head of frequent tokens and one or more tails of rare tokens (clusters).

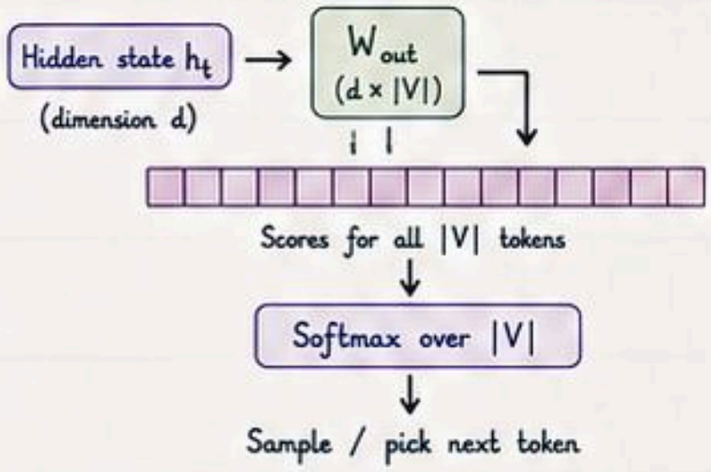
The head is computed at full dimensionality on every step. Tail clusters use reduced-dimension projections and are only evaluated when selected, cutting computation substantially with minimal accuracy loss.

Remember

* Word frequency ~ Zipfian (few words are very common, most are rare).
* Adaptive Softmax = hierarchical softmax with clusters and shortlists.
* Computes the head every step; computes only one tail cluster when needed.
* Big speedup with tiny quality loss.

Standard (full) softmax

Compute a score for every token at every step.



Cost per step ~ $O(d \times |V|)$

Example setup

Vocabulary size	$\|V\|$	= 100,000
Model dim	d	= 4096
Head size	H	= 4,000
Cluster count		= 10
Avg cluster size	$\|C_i\|$	≈ 9,600
Cluster dim	d_i	= 256

Standard softmax cost: $d \times |V|$ ≈ 409M ops/step

Adaptive softmax cost (head only):

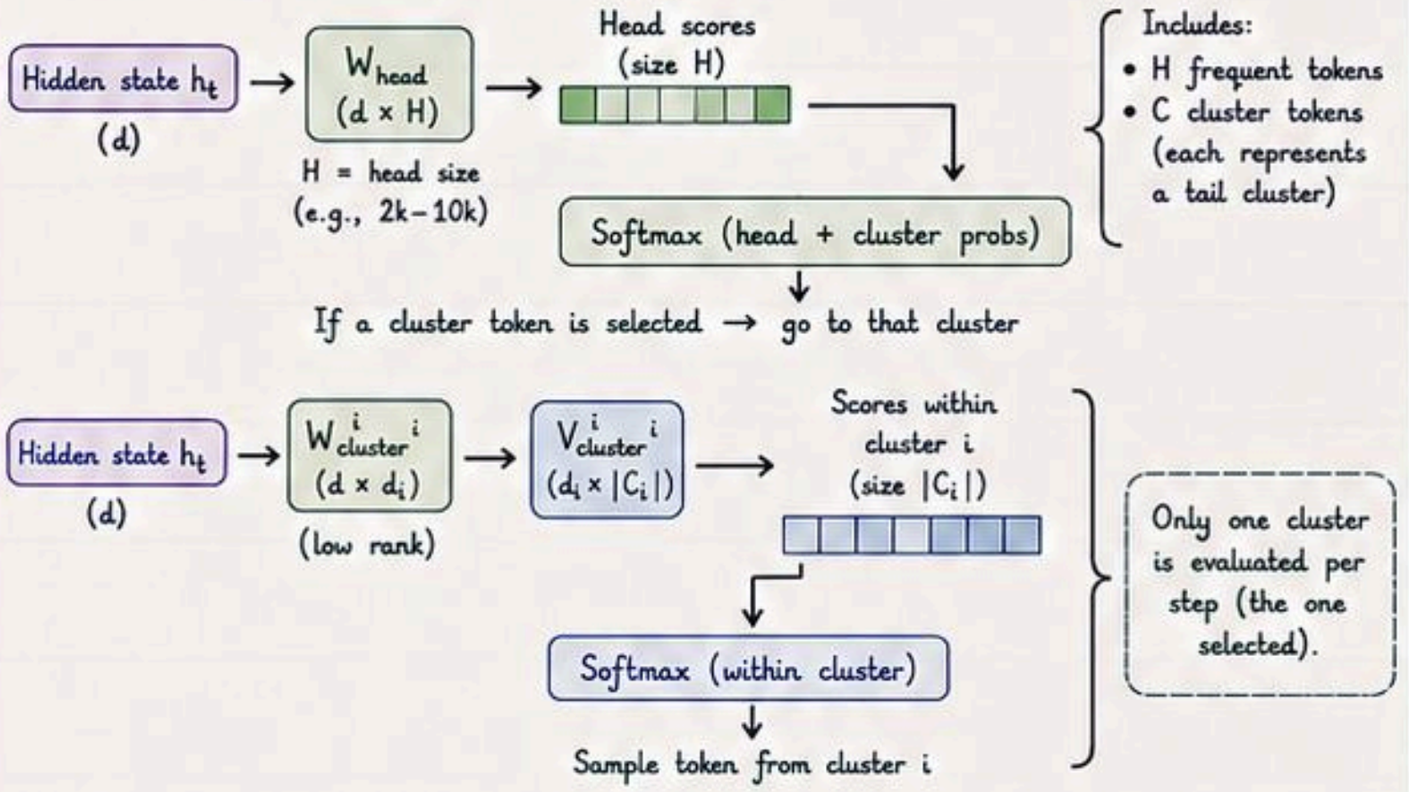
$d \times H$ ≈ 16M ops/step → ~25x cheaper

If a cluster is selected:

$d \times d_i + d_i \times |C_i|$ ≈ 3.8M ops (extra)

Adaptive Softmax (hierarchical)

1) Compute head (frequent tokens) every step.
2) If a tail cluster is chosen, compute scores only within that cluster using a low-dim projection.



Cost per step ~ $O(d \times H) + O(d \times d_i + d_i \times |C_i|)$ (only if a cluster is selected)

Where it fits and what replaced it

When Adaptive Softmax mattered

- Pre-transformer and early-transformer era (RNN/LSTM and early MT models).
- Large vocabularies (100k+) made full softmax a major bottleneck.
- Gave big training and inference speedups with negligible perplexity change.

What we use today instead

- ✓ Tied input-output embeddings (weight tying) (reduces parameter count and improves cache locality).
- ✓ Quantised output layers (INT8/FP8) to cut memory bandwidth and compute.
- ✓ Speculative decoding / draft models to reduce the number of steps.
- ✓ Better hardware (matrix cores, large parallelism) makes full softmax affordable.

Interview edge:

* Adaptive Softmax speeds up by avoiding full-vocab computation every step.
* It leverages Zipfian frequency: frequent words in a head, rare words in clusters.
* Only one cluster is evaluated when needed using low-rank projections.
* It was crucial earlier; today we prefer weight tying, quantisation and speculative decoding.



Section 3: Decoding, Loss and the Math Layer

* 24. What is cross-entropy loss and why is it used in language models?

Cross-entropy measures the distance between the model's predicted probability distribution and the true distribution. In next-token prediction, the true distribution is a one-hot vector on the correct token.

Practically, cross-entropy reduces to the negative log probability the model assigned to the right answer, averaged over all tokens.

Minimising cross-entropy is equivalent to maximising the likelihood of the training data, which is exactly the objective we want.

Remember

* Used for multiclass classification (many tokens).
* Measures how well predicted distribution matches the true distribution.
* Lower loss → better predictions.
* Differentiable and works great with gradient descent.

1. Definition

For a single training example, with true distribution y and predicted distribution $\hat{y}$ over V tokens:

$$H(y, \hat{y}) = -\sum_{i=1}^{V} y_i \log \hat{y}_i$$

where:

y_i = true probability of token i (one-hot: 1 for correct token, 0 else)
$\hat{y}_i$ = predicted probability of token i
V = vocabulary size

2. Simplifies to negative log probability

If the correct token is k, then $y_k = 1$ and $y_i = 0$ for $i \neq k$.

So,

$$H(y, \hat{y}) = -\log \hat{y}_k$$

i.e., the loss is just the negative log probability assigned to the correct token.

3. Sequence loss (what we actually optimize)

For a sequence of T tokens with ground truth tokens $w_1, \ldots, w_T$ and model predictions $\hat{y}_1, \ldots, \hat{y}_T$,

$$\mathcal{L}_{CE} = -\frac{1}{T}\sum_{t=1}^{T} \log P(w_t \mid w_{<t})$$

Average negative log likelihood per token.
Minimising this is equivalent to maximising the likelihood of the entire training corpus.

Tiny example

Vocabulary: [the, cat, sat, on] ($|V| = 4$)
Correct next token: "cat"

Token	Model probability $\hat{y}_i$	y_i (one-hot)	$y_i \log \hat{y}_i$
the	0.10	0	0
cat	0.70	1	$\log 0.70 = -0.357$
sat	0.15	0	0
on	0.05	0	0

Cross-entropy loss $= -\sum y_i \log \hat{y}_i = 0.357$

4. Connection to Perplexity (the go-to follow-up)

Perplexity is the exponential of average cross-entropy:

$$\text{Perplexity} = \exp(\mathcal{L}_{CE})$$

Interpretation: the effective number of tokens the model is choosing between at each sten.

What does perplexity mean?

Perplexity	Interpretation
1	Perfect certainty (always picks the right token)
8	As uncertain as picking uniformly among 8 options
50	As uncertain as picking among 50 options
100	More uncertain (higher is worse)

Why it's useful

- Scale-free and easy to compare across models and datasets.
- Standard intrinsic metric for pretraining and evaluation.
- Lower perplexity → model assigns higher probability to the true data → better model.

Interview edge:

* Cross-entropy is the workhorse loss for next-token prediction.
* It equals negative log likelihood of the correct token.
* Perplexity = exp(average cross-entropy): lower is better.
* Interpretable: "How many choices is the model effectively considering?"



Section 3: Decoding, Loss and the Math Layer

* 25. How are gradients computed with respect to embeddings?

An embedding lookup is mathematically a multiplication of a one-hot vector by the embedding matrix, which means it is differentiable like any other linear layer.

During backpropagation, the gradient arriving at a token's vector is written back to that token's row in the embedding matrix.

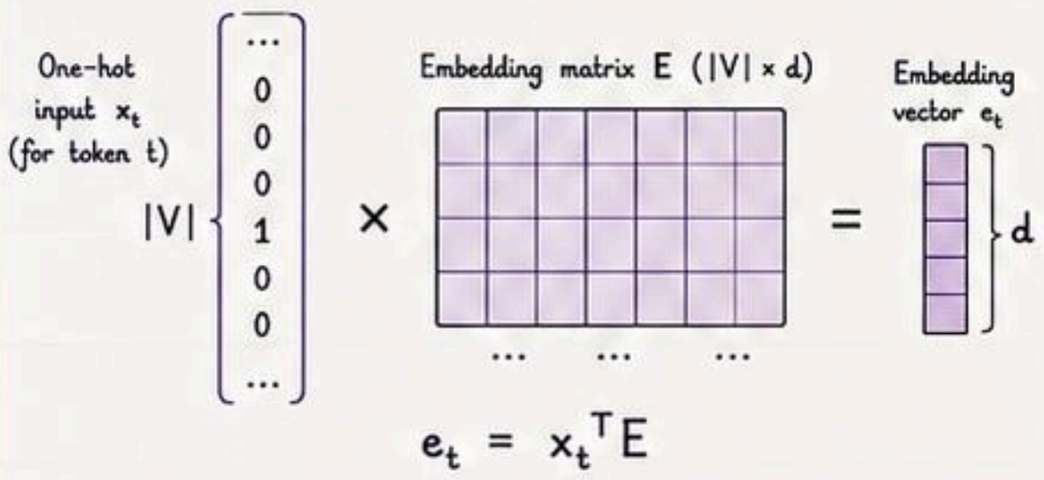
Because only the tokens present in the batch receive gradient, embedding updates are sparse, and frameworks exploit that with sparse gradient accumulation rather than updating the full matrix.

Remember

* Embedding lookup = one-hot vector × matrix.
* Only rows for tokens in the batch get updated.
* This makes embedding updates sparse.
* Efficient frameworks use sparse gradient accumulation.
* Consequence: rare tokens get updated rarely.

1. Forward pass: lookup as matrix multiplication

Vocabulary size = $|V|$, embedding dimension = d

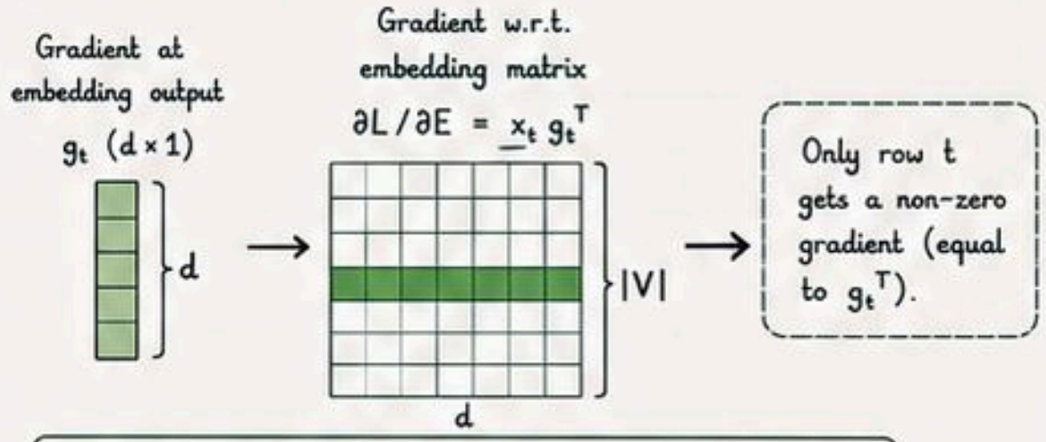


$$e_t = x_t^T E$$

Each row i in E is the embedding for token i.

2. Backward pass: writing gradients back to rows

Let $g_t = \partial L / \partial e_t$ be the gradient arriving at the embedding vector.



Update: $E[t, :] \leftarrow E[t, :] - \eta \cdot g_t^T$

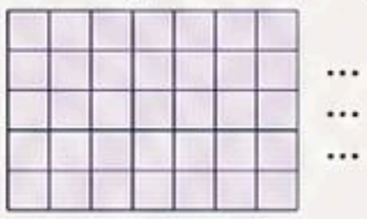
All other rows have zero gradient for this step.

3. Example with a tiny vocabulary

Vocabulary ($|V|$ = 6), embedding dimension (d = 4)

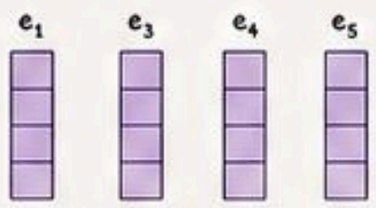
Index	Token
0	the
1	cat
2	sat
3	on
4	mat
5	.

Embedding matrix E (6 × 4)



Batch tokens (as indices): [1, 3, 4, 5]

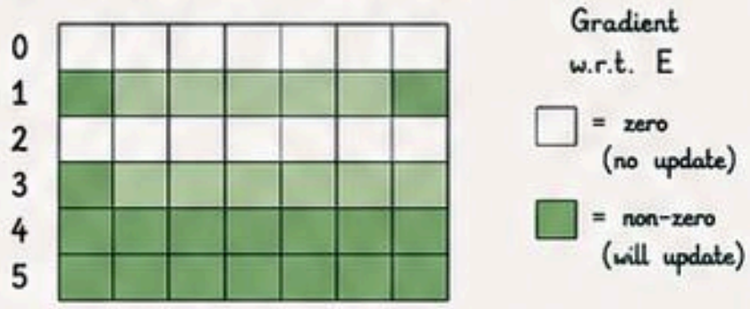
Looked up embeddings:



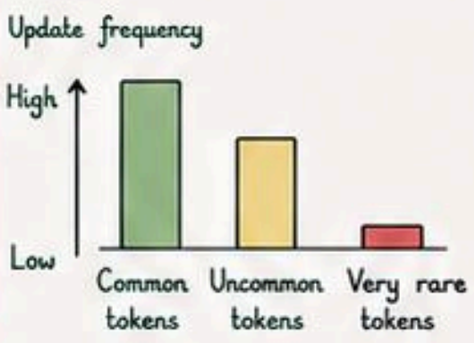
(each is d = 4 dim)

During backprop, gradients come back for these 4 tokens.

Only rows 1, 3, 4, 5 in E get updated.



4. Why this matters (the consequence)



- Rare tokens receive gradient updates very infrequently.
- Their embeddings stay closer to initialization and learn less.
- Leads to weaker representations for rare technical terms, low-resource languages, and unusual proper nouns.
- Connects back to tokenization: more fragmentation → rarer tokens → poorer learning.

5. Implementation note

- Embedding gradients are typically accumulated in sparse format (e.g., COO / CSR).
- Only the rows present in the batch are stored and updated.
- Saves memory bandwidth and compute.
- After accumulation over micro-batches, the optimizer updates only touched rows.

Interview edge:

* The math is simple, the consequence is not.
* Sparse updates are why rare tokens underperform.
* Better tokenization + more data = more frequent updates = better embeddings.
* This is a key reason behind failures on rare terms, code, and low-resource languages.



Section 3: Decoding, Loss and the Math Layer

* 26. What is the role of the Jacobian matrix in backpropagation?

When a layer maps a vector input to a vector output, its derivative is not a single number but a matrix of all partial derivatives of every output with respect to every input. That matrix is the Jacobian.

Backpropagation through a deep network is, formally, a chain of Jacobian multiplications from the loss backwards to the parameters.

Remember

* Jacobian $J_f(x)$ of $f: \mathbb{R}^n \to \mathbb{R}^m$ is an $m \times n$ matrix where $[J_f(x)]_{ij} = \frac{\partial f_i}{\partial x_j}$.
* For scalars, Jacobian reduces to the gradient.
* It generalizes gradients to vector-valued functions.
* Backprop is repeated application of the chain rule using Jacobians.

1. Definition

For $y = f(x)$ where $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$, the Jacobian is:

$$J_f(x) = \frac{\partial y}{\partial x} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \frac{\partial y_m}{\partial x_2} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

Row i: how output y_i changes with all inputs.
Column j: how all outputs change with input x_j.

2. Chain rule with Jacobians

For a composition $z = g(y)$ and $y = f(x)$:

$$J_z(x) = J_g(y)\, J_f(x) \quad \text{(matrix multiplication)}$$

$$x \xrightarrow[(n)]{} \underset{\mathbb{R}^n \to \mathbb{R}^m}{f} \xrightarrow[(m)]{y} \underset{\mathbb{R}^m \to \mathbb{R}^p}{g} \xrightarrow[(p)]{z}$$

In a network with layers $1, 2, \ldots, L$ and loss $\mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial x} = J_{f_1}(x)^T\, J_{f_2}(h_1)^T \cdots J_{f_L}(h_{L-1})^T\, \frac{\partial \mathcal{L}}{\partial h_L}$$

(backprop is the chain rule in reverse)

3. Backprop as vector-Jacobian products (what frameworks actually do)

We rarely form full Jacobians (they are huge!).

Reverse-mode AD computes vector-Jacobian products (VJPs) directly.

Upstream gradient (row vector) $\bar{y}^T$, $1 \times m$ → Multiply by $J_f(x)^T$, $(m \times n)$ → Downstream gradient (row vector) $\bar{x}^T = \bar{y}^T J_f(x)$, $1 \times n$

Why this is key

- ✓ Cost depends on operations, not on $m \times n$ size.
- ✓ Memory is constant in output size.
- ✓ This is reverse-mode autodiff.

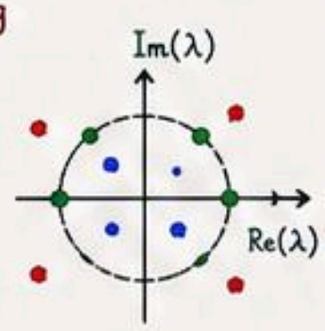
Example: $y = Wx + b$ (linear layer)	Jacobian:	VJP with upstream gradient $\bar{y}^T \in \mathbb{R}^{1 \times m}$:	
$W \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$	$J_f(x) = [W \;\; I_m]$ $(m \times (n+m))$	$\bar{x}^T = \bar{y}^T W$ $(1 \times n)$	$\bar{b}^T = \bar{y}^T$ $(1 \times m)$

No need to build the full Jacobian! We only do the multiplications we need.

4. Spectral view: why Jacobians matter for training

Let J be the Jacobian of a layer (or block). Its eigenvalues λ_i control how gradients change in that direction.

- $|\lambda_i| > 1$ → gradient grows (risk of explosion)
- $|\lambda_i| < 1$ → gradient shrinks (risk of vanishing)
- $|\lambda_i| = 1$ → gradient preserved



- $|\lambda| > 1$ (explode)
- $|\lambda| < 1$ (vanish)
- $|\lambda| = 1$ (preserve)

5. Implementation reality

- ✓ Frameworks never materialize Jacobian matrices.
- ✓ They compute VJPs using local derivative rules for each operation.
- ✓ This makes backprop feasible for billion-parameter models.
- ✓ Efficient memory use via checkpointing, recomputation and in-place ops.

Interview edge:

* The Jacobian tells us how outputs move with inputs.
* Backprop is a chain of Jacobian transposes multiplying the loss gradient.
* We compute vector-Jacobian products, not full Jacobians.
* Jacobian eigenvalues explain vanishing and exploding gradients across depth.



Section 3: Decoding, Loss and the Math Layer

* 27. What is the chain rule and how is it used in deep learning?

The chain rule states that the derivative of a composition of functions is the product of the derivatives of its parts.

A neural network is precisely a composition of functions, so the gradient of the loss with respect to any early-layer weight is the product of local derivatives along the path from that weight to the loss.

> Backpropagation is nothing more than the chain rule applied systematically, right to left.

Remember

* Chain rule (two functions):
$$\frac{dL}{dx} = \frac{dL}{dy}\ \frac{dy}{dx}$$
* For many functions: multiply the local derivatives along the path.
* This is why gradients flow backward through the network.
* Enables training of very deep networks.

1. The chain rule (intuition first)

If $y = f(x)$ and $z = g(y)$, then:

$$\frac{dz}{dx} = \frac{dz}{dy}\ \frac{dy}{dx}$$

Extend to many functions:

If $y_1 = f_1(x),\ y_2 = f_2(y_1),\ \dots,\ y_n = f_n(y_{n-1})$, then

$$\frac{dy_n}{dx} = \frac{dy_n}{dy_{n-1}}\ \frac{dy_{n-1}}{dy_{n-2}}\ \cdots\ \frac{dy_1}{dx} = \prod_{i=1}^{n} \frac{dy_i}{dy_{i-1}}$$

2. A simple neural network as a chain of functions

Input x → $z^{[1]} = W^{[1]}x + b^{[1]}$ (affine) → $a^{[1]} = \sigma(z^{[1]})$ (activation) → $\cdots$ → $z^{[L]} = W^{[L]}a^{[L-1]} + b^{[L]}$ (affine) → $\hat{\sigma} = g(z^{[L]})$ (output) → $\mathcal{L} = \mathcal{L}(\hat{\sigma}, y)$ (loss)

The gradient to an early weight $W^{[l]}$ is:

$$\frac{dL}{dW^{[l]}} = \underbrace{\frac{dL}{d\hat{\sigma}}\ \frac{d\hat{\sigma}}{dz^{[L]}}\ \frac{dz^{[L]}}{da^{[L-1]}}\ \frac{da^{[L-1]}}{dz^{[L-1]}}\ \cdots}_{\text{backpropagated from the right}}\ \underbrace{\frac{da^{[l]}}{dz^{[l]}}\ \frac{dz^{[l]}}{dW^{[l]}}}_{\text{local derivative at this layer}}$$

3. Backpropagation = efficient application of the chain rule

Naively computing the gradient for each parameter would recompute the same sub-paths many times.

Example: a 4-layer chain $x \to f_1 \to f_2 \to f_3 \to f_4 \to \mathcal{L}$

Naive (inefficient)
Recompute for each parameter

$\frac{dL}{d\theta_1}$: $x \to f_1 \to f_2 \to f_3 \to f_4 \to L$

$\frac{dL}{d\theta_2}$: $x \to f_1 \to f_2 \to f_3 \to f_4 \to L$

$\frac{dL}{d\theta_3}$: $x \to f_1 \to f_2 \to f_3 \to f_4 \to L$

$\frac{dL}{d\theta_4}$: $x \to f_1 \to f_2 \to f_3 \to f_4 \to L$

Lots of repeated work!

Backprop (efficient)
Cache once, reuse many times

1. Forward pass: compute and cache intermediates
   $x \to f_1 \to f_2 \to f_3 \to f_4 \to L$
2. Backward pass: propagate gradients right to left
   $L \leftarrow f_4 \leftarrow f_3 \leftarrow f_2 \leftarrow f_1 \leftarrow x$
3. Each local derivative is computed once and reused to update all relevant parameters.

Why this matters

- Avoids exponential blow-up of computation.
- Turns the chain rule into a dynamic program.
- Makes training billion-parameter models possible.
- This is the algorithmic heart of modern deep learning.

Interview edge:

* Caching (storing activations) is what makes the chain rule tractable.
* Backprop = dynamic programming over the chain rule.
* Without it, we would recompute the same derivatives for every parameter.
* This efficiency is the reason deep learning scales.



Section 3: Decoding, Loss and the Math Layer

* 28. What is ReLU and why is it important?

ReLU (Rectified Linear Unit) is an activation function that outputs the input when positive and zero otherwise.

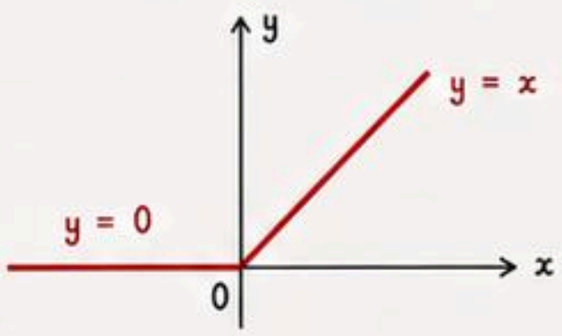
$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x, & \text{if } x > 0 \\ 0, & \text{otherwise} \end{cases}$$

It introduces non-linearity, is computationally cheap, and helps gradients flow better than sigmoid/tanh. Its downside is "dying ReLU", where units stuck in the negative region stop learning.

Remember

* Non-linearity is essential; without it, deep networks reduce to a single linear transform.
* ReLU gradient: $\frac{d\text{ReLU}(x)}{dx} = 1$ $(x > 0)$, 0 $(x < 0)$, undefined at $x = 0$ (set to 0 or 1).
* Cheap to compute: just a threshold.
* Risk: dying ReLU → zero gradient forever.

1. ReLU function and derivative



x	ReLU(x)	dReLU/dx
$x < 0$	0	0
$x = 0$	0	0 or 1 (implementation choice)
$x > 0$	x	1

2. Why ReLU mattered

- Introduces non-linearity with zero extra parameters.
- Mitigates vanishing gradients: gradient is 1 for positive inputs (unlike sigmoid/tanh).
- Sparse activations: many zeros → some compute and memory savings.
- Fast: one comparison and a multiply.

Compared to sigmoid/tanh

Activation	Gradient in saturated region
Sigmoid	≈ 0 (vanishing)
Tanh	≈ 0 (vanishing)
ReLU	1 (passes gradient)

3. The downside: dying ReLU

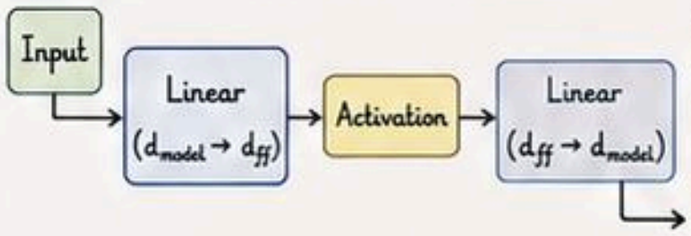
If a unit's inputs are always negative, its output is always 0 and gradient is 0 → it never updates.

Common causes
- Large learning rates
- Poor initialization
- Too much L2 regularization

Mitigations
- He initialization
- Lower learning rate
- Leaky ReLU / PReLU / ELU
- Normalization (LayerNorm, RMSNorm)

4. What modern LLMs actually use (ReLU is not the default)

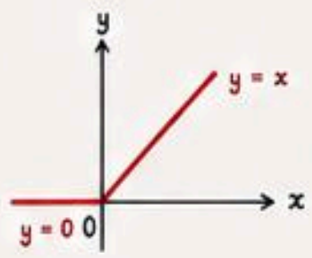
Typical Transformer FFN block



Activation choice is where models differ.

Common activation functions in LLMs

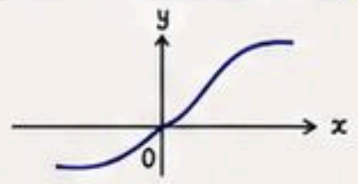
- **ReLU** (legacy)



Simple but not used in most modern LLMs.

- **GELU** (used in GPT-family)

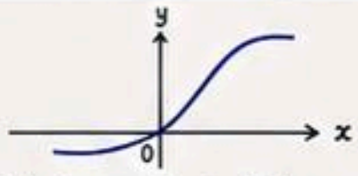
$$\text{GELU}(x) = x\Phi(x)$$
$$\Phi(x) = \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$



Smoother than ReLU, better empirical performance.

- **SwiGLU** (used in LLaMA, PaLM 2, Gemma, Mistral, etc.)

$$\text{SwiGLU}(x) = \text{Swish}(x) \odot (W_2 x)$$
$$\text{Swish}(x) = x \cdot \sigma(x)$$



Gated + smooth → higher quality at similar cost.

* SwiGLU (gated linear units) are now the de facto standard in state-of-the-art open models.

Interview edge:

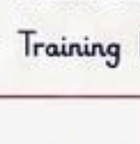
* ReLU was a breakthrough for deep nets by fixing gradient flow.
* But modern LLMs prefer smoother or gated activations (GELU, SwiGLU).
* Know the history and the current practice.
* Always connect activation choice to optimization and final model quality.



Section 3: Decoding, Loss and the Math Layer

* 29. What is the vanishing gradient problem, and how do Transformers solve it?

When gradients propagate backwards through many layers or many time steps, repeated multiplication by small local derivatives shrinks them towards zero. Early layers barely learn and long-range dependencies never get captured.



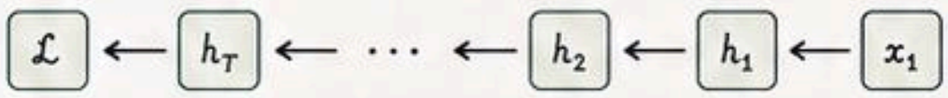
RNNs suffered acutely because depth scaled with sequence length (hundreds or thousands of time steps).

Remember

* Gradients are products of local derivatives along the path from loss back to parameters.
* If most derivatives have magnitude < 1, the product → 0 (vanishing gradient).
* If > 1, the product → ∞ (exploding gradient).
* Vanishing → no learning signal for early layers or long-range information.

1. The vanishing gradient problem

Simple view: gradient is a product of many terms.



Gradient to early state h_1:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

If $\left|\frac{\partial h_t}{\partial h_{t-1}}\right| < 1$ on average, the product → 0 as T grows.

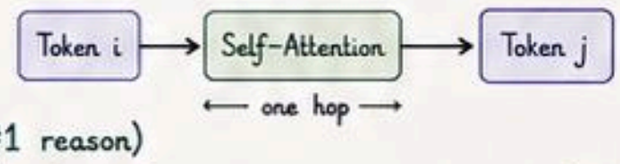
Consequences

- Early layers get tiny gradients → barely update.
- Model can't capture long-term dependencies.
- Training becomes unstable for long sequences or deep stacks.

2. How Transformers mitigate it (structurally)

① No recurrence, short paths

Any two tokens connect in one attention hop (O(1) path length) instead of O(n) time steps.

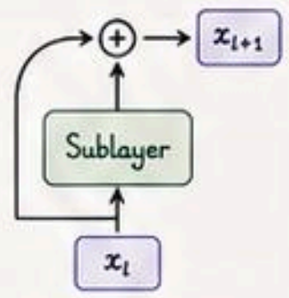


② Residual connections (the #1 reason)

Each sublayer output is added to its input:

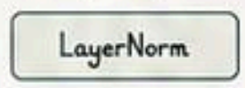
$$x_{l+1} = x_l + \text{Sublayer}(x_l)$$

Gradients have an identity path straight through every block (with derivative = 1). Signal never has to go through many multiplicative terms.



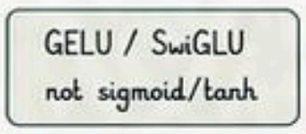
③ Layer Normalization (LN)

Keeps activations in a stable range → derivatives don't shrink or explode.



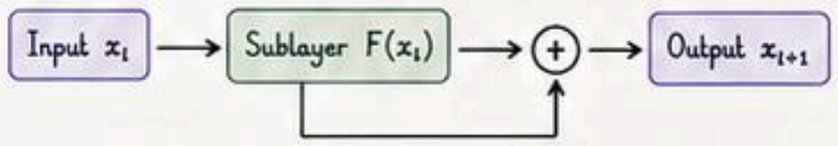
④ Non-saturating activations

ReLU/GELU/SwiGLU stay in regions with healthy gradients (unlike sigmoid/tanh saturation).



3. Residual connections give an identity gradient path

Consider one residual block:



Gradient w.r.t. input:

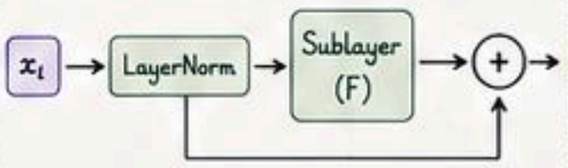
$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_{l+1}} \left(I + \frac{\partial F(x_l)}{\partial x_l}\right)$$

Even if $\frac{\partial F}{\partial x_l}$ is small, the identity term I passes the gradient through unchanged.

* This is why very deep Transformers (100s of layers) train successfully.

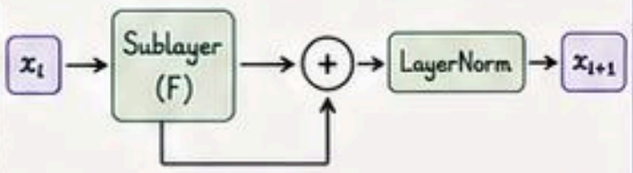
4. Pre-Norm (now standard) vs Post-Norm

Pre-Norm (used in modern LLMs)



- ✓ More stable gradients
- ✓ Easier to train deep models
- ✓ Preferred for scale (e.g., LLaMA)

Post-Norm (original Transformer)



- ! Less stable at large depth
- ! More sensitive to LR and init
- ! Harder to train very deep stacks

5. Why this combination works



Short paths via attention
O(1) path length between tokens



Residual connections
Identity path keeps gradients alive



Layer normalization
Stable scales → stable gradients



Good activation functions
Avoid flat regions (saturation)

→ **Result:** deep, long-context models that learn effectively across hundreds of layers and thousands of tokens.

Interview edge:

* If you must pick one, name residual connections first.
* Mention pre-norm vs post-norm, and that pre-norm is the modern standard.
* This structural design—short paths, residuals, LN, and good activations—solved the vanishing gradient problem for Transformers at scale.



Section 3: Decoding, Loss and the Math Layer

* 30. What are eigenvalues and eigenvectors (in dimensionality reduction)?

For a square matrix $A \in \mathbb{R}^{n \times n}$:

- An eigenvector $v \neq 0$ is a non-zero vector that only gets stretched or compressed by A (its direction does not change).
- The eigenvalue λ is the factor of that scaling.

$$Av = \lambda v$$

In PCA (Principal Component Analysis):

1. Compute the mean-centered data matrix $X \in \mathbb{R}^{n \times d}$ (n samples, d dims).
2. Covariance matrix: $\Sigma = \frac{1}{n-1} X^T X \in \mathbb{R}^{d \times d}$
3. Eigen-decompose: $\Sigma = Q \Lambda Q^T$

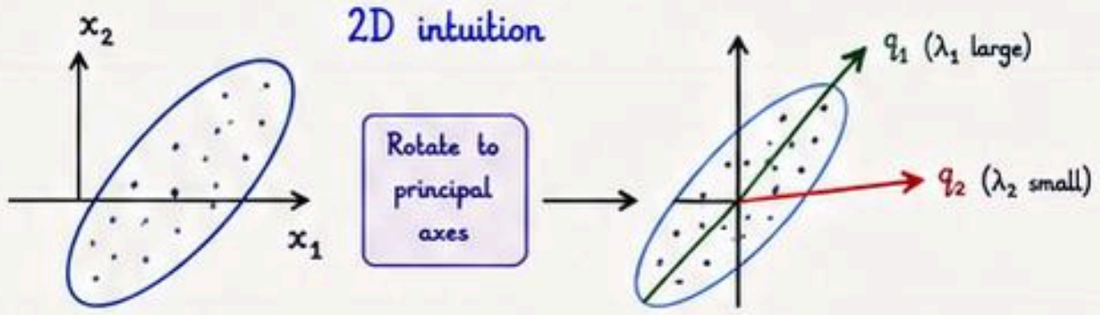
$Q = [q_1, q_2, \ldots, q_d]$ (orthonormal eigenvectors)

$\Lambda = \text{diag}(\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_d)$ (eigenvalues ≥ 0)

PCA finds orthogonal directions of maximum variance. Keep the top k → reduce dimensionality while preserving most of the data's structure (information).

Remember

- Eigenvectors form an orthogonal basis.
- Eigenvalues can be thought of as "variance explained" along those directions (for covariance matrix).
- Larger λ → more variance captured.
- Keep top k eigenvectors (columns of Q_k). Project data: $Z = XQ_k$ (compressed to k dims).
- Reconstruction (optional): $\hat{X} = ZQ_k^T$



How much variance is kept?

Explained variance ratio for component i: $EVR_i = \frac{\lambda_i}{\sum_{j=1}^{d} \lambda_j}$

Cumulative EVR after top k components: $\sum_{i=1}^{k} EVR_i$

We choose the smallest k such that cumulative EVR ≥ target (e.g., 95%).

Why this matters in the GenAI stack (same spectral logic, different clothes)

1. Compressing embeddings for cheaper vector search

Embeddings are typically 768–3072 dims. Most variance is concentrated in a lower-dimensional subspace.

Original embedding (3072 dims) → PCA / SVD, Keep top k components → Compressed embedding (128–512 dims)

- Lower dimension → less memory.
- Faster ANN search (FAISS, ScaNN, etc.).
- Minimal loss in retrieval quality.

2. Matryoshka (nested) embeddings

Matryoshka embeddings are trained so that the first k dimensions capture the most important information.

Full embedding (1024 dims)

Truncate to 512 dims

Truncate to 128 dims

Still keeps semantics reasonably well!

- This is effectively learning a good ordering of the principal components.

3. LoRA and low-rank updates

LoRA assumes weight updates live in a low-rank subspace.

Full weight update ΔW ($d_{out} \times d_{in}$)

Approximate with rank-r factors:

$$\Delta W \approx BA$$
$$B \in \mathbb{R}^{d_{out} \times r},\ A \in \mathbb{R}^{r \times d_{in}}$$

- $r \ll \min(d_{out}, d_{in})$
- Far fewer trainable parameters.
- Same intuition: most useful variation lies in a low-dimensional subspace.

4. Weight compression & quantization

SVD/PCA is used to find low-rank approximations of weight matrices (or their updates).

Weight matrix W ($m \times n$) → SVD → $U \Sigma V^T$

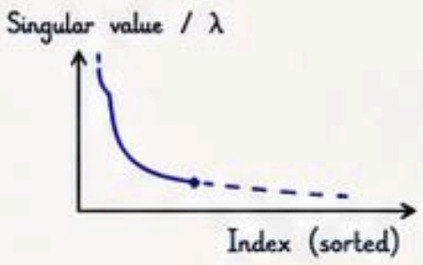
Keep top r singular values →

$$W_r = U_r \Sigma_r V_r^T$$

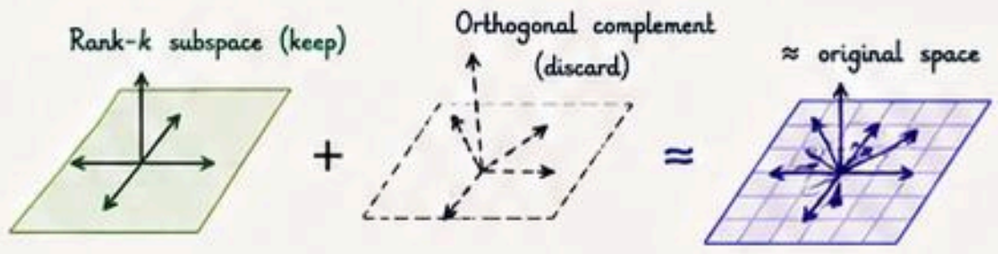
Smaller, faster, cheaper to serve.

The unifying picture

In all these cases, the data/updates are dominated by a few directions.



We keep the top directions (eigenvectors / singular vectors) and discard the rest.



Key takeaways

- Eigenvectors = directions.
- Eigenvalues = importance (variance).
- Keep the top few → compress without losing much.
- This idea underlies embeddings, search, LoRA, and compression across the GenAI stack.

Interview edge:

- Don't just define eigenvalues—explain why they matter.
- Connect PCA to real GenAI applications (embeddings, LoRA, compression).
- State the practical impact: lower memory, faster search, cheaper training/inference.
- It's the same math showing up everywhere in the stack.

Section 4: Fine-Tuning, Efficiency and Prompting

* 31. How is KL divergence used in evaluating LLMs?

KL divergence (Kullback–Leibler divergence) measures how much one probability distribution differs from another. It is asymmetric, so direction matters.

Definition

$$D_{KL}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

Read as: how much information is lost when Q is used to approximate P.

Remember

* $D_{KL}(P \parallel Q) \geq 0$ and equals 0 iff P = Q.
* Not symmetric: $D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$.
* Penalizes Q for assigning low probability to events that P considers likely.
* Sensitive to the tail: big penalty if Q misses probability mass that P puts on.

KL divergence shows up in three key places in LLM work

1. Knowledge Distillation (teacher → student)



The student is trained to match the teacher's output distribution over all tokens, not just the argmax token.

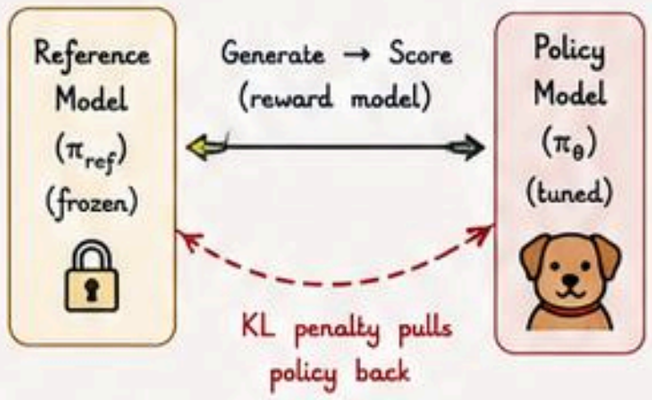
Loss: $D_{KL}(P_T \parallel P_S)$

Where P_T = teacher distribution,
P_S = student distribution.

- ✓ Transfers dark knowledge (similarities between non-top tokens).
- ✓ Improves generalization.
- ✓ Common in distilling large models into smaller ones.

2. Alignment (RLHF / DPO / PPO)

The KL "leash"



Objective (typical RLHF / PPO form):

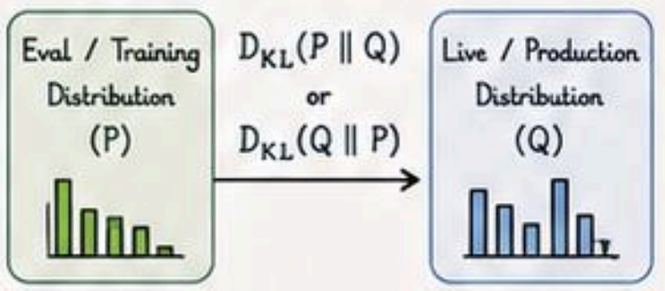
$$\mathcal{L} = E_{x \sim D, y \sim \pi_\theta}[R(x, y)] - \beta D_{KL}(\pi_\theta(\cdot|x) \parallel \pi_{ref}(\cdot|x))$$

$R(x, y)$ = reward for response y given prompt x
β = strength of the KL penalty (the leash)

- ✓ Without the KL term, the model can exploit the reward model (reward hacking).
- ✓ KL keeps the tuned policy close to the reference, so outputs remain on-distribution and safe.
- ✓ Think of it as: optimize for helpfulness, with a leash that keeps it recognizably itself.

3. Monitoring & Evaluation (Drift Detection)

Compare distributions between two sources (e.g., eval set vs. live traffic).



High KL → the distributions are different → potential drift.

What we compare

- Token distributions (at next-token level).
- Embedding distributions (e.g., CLS / mean-pool).
- Feature / activation distributions inside layers.

Why it matters

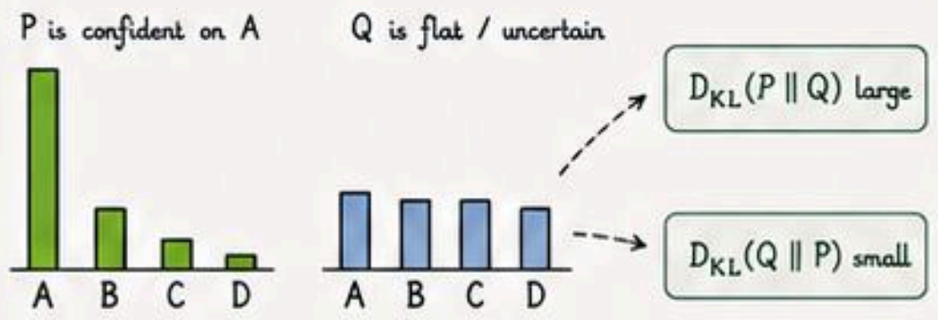
- Detect data drift and user behavior change.
- Catch prompt/template drift.
- Trigger re-evaluation or re-training when needed.

Why KL and why direction matters

KL is asymmetric.

Example (intuitive):

If P puts high probability on event A but Q ignores A → $D_{KL}(P \parallel Q)$ is large. But if Q spreads probability broadly → $D_{KL}(Q \parallel P)$ can be much smaller.



Rule of thumb

Use $D_{KL}(\pi_{policy} \parallel \pi_{ref})$ in alignment: penalize the policy for moving away from the reference. This is the standard "keep me close to who I was" direction.

Interview edge:

* Lead with alignment: KL is the leash that keeps the model from going off the rails.
* Without it, reward optimization finds weird, high-reward outputs humans hate.
* Know the other two uses: distillation (training) and drift detection (monitoring).
* Always mention direction and why it matters.



Section 4: Fine-Tuning, Efficiency and Prompting

* 32. What is LoRA and QLoRA?

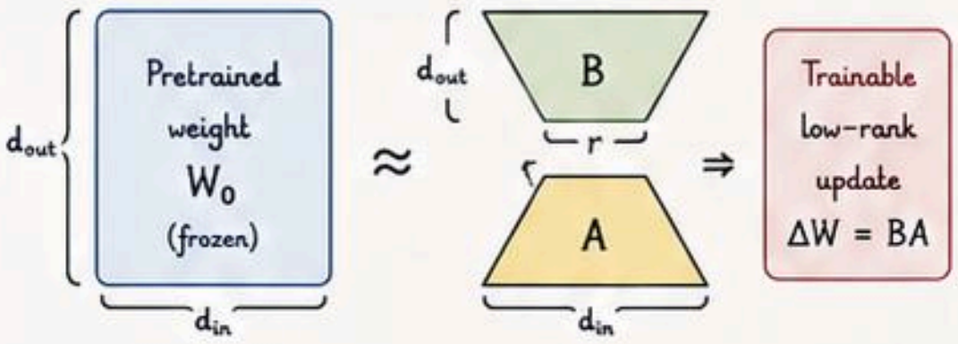
LoRA (Low-Rank Adaptation) and QLoRA are parameter-efficient fine-tuning methods that drastically cut compute and memory while preserving quality.

Remember

* LoRA: add low-rank trainable updates, keep base model frozen.
* QLoRA: quantize the base model to 4-bit (NF4) and train LoRA adapters in higher precision.
* Big win: tiny number of trainable params, small memory, fast fine-tunes, easy to deploy.
* Result: fine-tune large models on a single GPU.

1. LoRA: Low-Rank Adaptation

Instead of updating a full weight matrix $W \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA learns a low-rank update $\Delta W = BA$ where $B \in \mathbb{R}^{d_{out} \times r}$, $A \in \mathbb{R}^{r \times d_{in}}$, $r \ll \min(d_{in}, d_{out})$.

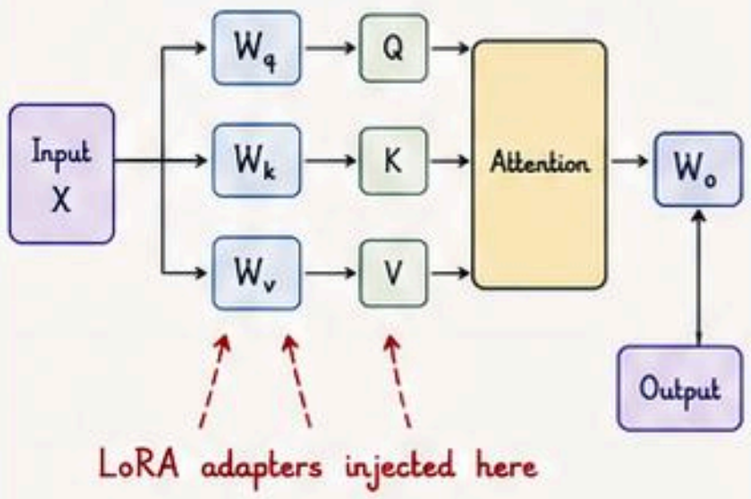


At inference: $W = W_0 + \alpha \cdot BA$
where α is a scaling factor.

Only A and B are updated. W_0 stays frozen.

Where LoRA is typically applied

Most commonly on attention projection matrices: W_q, W_k, W_v, and W_o (sometimes MLP layers too).

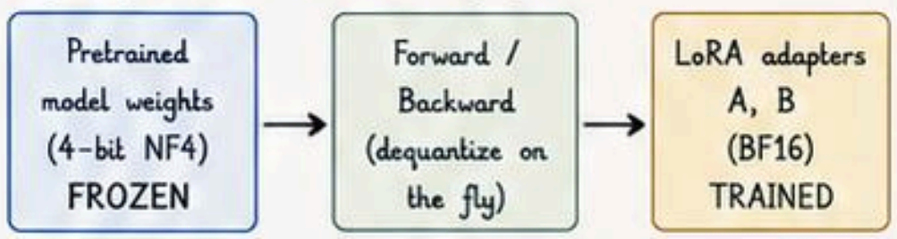


Why it works

- Fine-tuning updates often lie in a low-dimensional subspace.
- Low-rank captures most task-specific signal.
- Matches full fine-tuning quality on many tasks.

2. QLoRA: LoRA + 4-bit Quantization

QLoRA stores the frozen base model in 4-bit NF4 precision and trains LoRA adapters in higher precision (e.g., BF16). Paged optimizers handle memory spikes.



- Dequantize weights to BF16 for compute.
- Gradients flow only through LoRA adapters.

NF4 quantization (NormalFloat-4)

- 4-bit data type with non-uniform bins, optimized for normal weight distributions.
- Better accuracy than naive FP4.
- Typical range: ~ -6 to 6 (learned per-block scale).

Paged optimizers

- Optimizer states are stored in CPU RAM and paged to GPU only when needed.
- Prevents OOM during backprop on long sequences or large batch sizes.

Why QLoRA is a game changer

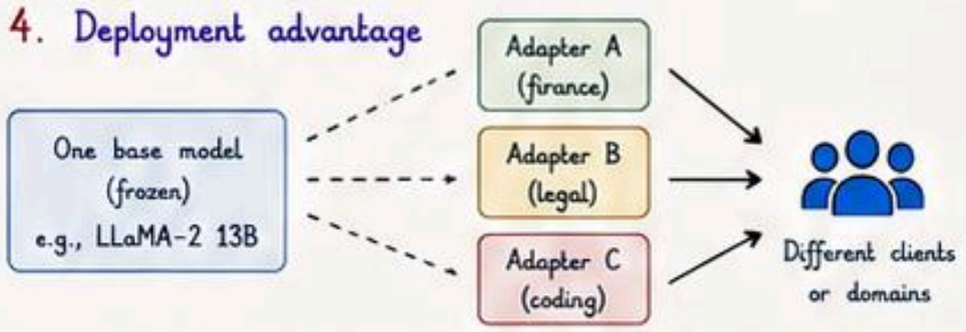
- Dramatically lower memory (4-bit weights).
- Tiny number of trainable parameters (LoRA).
- Stable training with paged optimizers.
- Enables 7B-13B model fine-tuning on a single GPU!

3. Cost & size comparison (example: LLaMA-2 7B)

Method	Trainable Params	GPU Memory (approx.)*	Storage for model	Quality
Full Fine-Tuning (FP16)	~ 7 Billion	~ 160-200 GB	~ 28 GB (FP16)	★★★★★
LoRA (r = 16)	~ 16-20 Million	~ 18-24 GB	~ 28 GB (FP16)	★★★★☆
QLoRA (r = 16, NF4)	~ 16-20 Million	~ 8-11 GB	~ 3.6 GB (4-bit)	★★★★☆

*Values vary by sequence length, batch size, and implementation.

4. Deployment advantage



Adapters are tiny (tens to hundreds of MB) → easy to store, swap, share, and serve without duplicating the base model.

Interview edge:

* LoRA = parameter-efficient fine-tuning via low-rank updates.
* QLoRA = LoRA + 4-bit NF4 quantization + paged optimizers.
* Turns fine-tuning of 7B-13B models into a single-GPU job.
* Adapters are small → multi-tenant serving and fast iteration.
* Lower cost, faster experiments, same (or close) quality.

Rule of thumb

If compute or memory is limited, use QLoRA. If you have huge compute and need the last 1-2% quality, full FT might win.

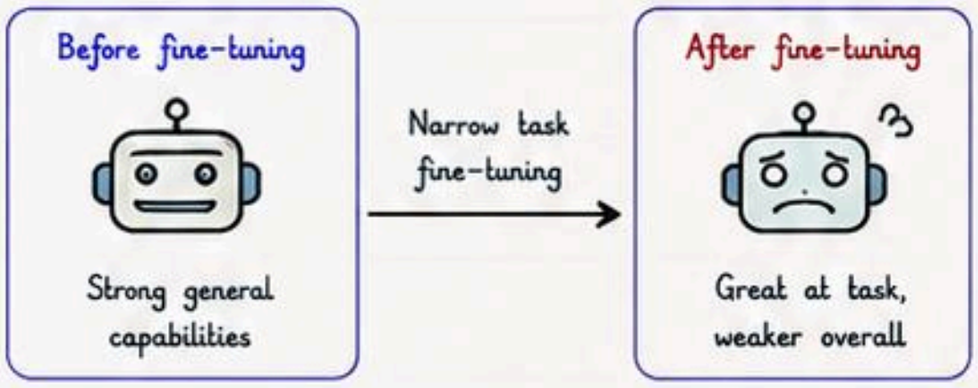


Section 4: Fine-Tuning, Efficiency and Prompting

* 33. How can catastrophic forgetting be mitigated in LLMs?

Catastrophic forgetting is when fine-tuning on a narrow task overwrites general capability.

Symptom: Great at your task → worse at everything else.



Remember

* You can't update weights without risk.
* Goal: learn the new task, keep what the model already knows.
* No single trick works everywhere—use a combination.
* Better yet, question whether fine-tuning is needed at all (see bottom).

Practical ways to mitigate catastrophic forgetting

1. Parameter-efficient fine-tuning (PEFT)

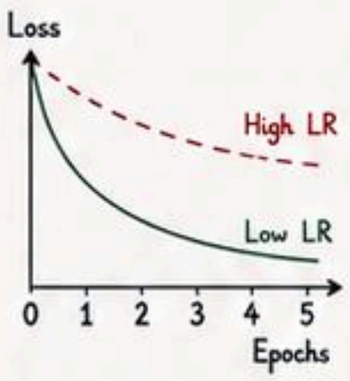
Prefer methods like LoRA, QLoRA, adapters.



- Base knowledge stays intact.
- Only small adapters change.
- Often the best first choice.

2. Low LR and few epochs

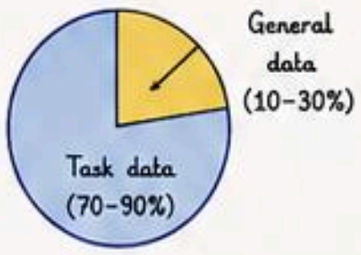
Small, careful updates change less.



- Lower learning rate.
- Train for fewer epochs.
- Early stop on validation.

3. Replay with general instruction data

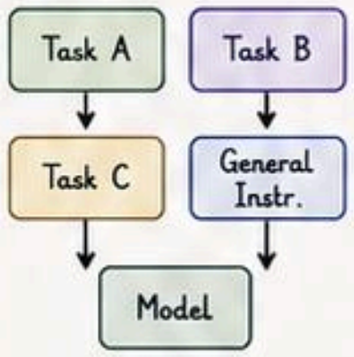
Mix in a slice of general data to rehearse old knowledge.



- Keeps the model anchored to broad capability.
- Small % goes a long way.

4. Multi-task training instead of sequential

Train on multiple tasks together.



- Avoids overwriting from task-to-task.
- Encourages shared representations.

5. Regularization (e.g., EWC)

Penalize movement in important parameters.

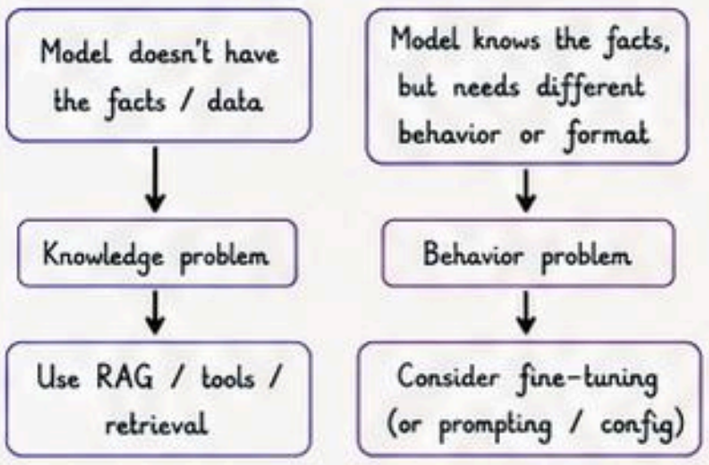
$$\mathcal{L} = \mathcal{L}_{task} + \lambda \sum_i F_i(\theta_i - \theta_i^*)^2$$

F_i = importance of parameter i
θ_i^* = original (pre-ft) value
λ = regularization strength

- Elastic Weight Consoildation (EWC) is a classic method.
- Keeps important weights close to where they started.

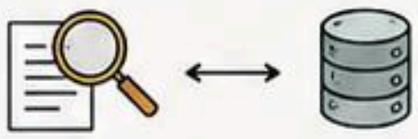
Interview edge: challenge the premise before fine-tuning

Step 1: Is this a knowledge problem or a behavior problem?



Why prefer RAG for knowledge?

- ✓ Updatable without retraining
- ✓ No risk of forgetting
- ✓ Uses latest, domain-specific sources
- ✓ Cheaper and faster to iterate
- ✓ Transparent citations / provenance



When fine-tune?

- ✓ You care about output format, tone, or style
- ✓ Domain vocabulary and phrasing
- ✓ Task-specific behavior / reasoning patterns
- ✓ Structured outputs, tool use, guardrails
- ✓ Consistent brand / policy adherence



Key takeaways

* Fine-tuning can cause forgetting. Mitigate it deliberately.
* PEFT (LoRA/QLoRA) + low LR + replay + multi-task + regularization.
* Question if you need to fine-tune—RAG often solves knowledge needs better.
* The goal is balance: specialize without losing what makes the model useful.



Section 4: Fine-Tuning, Efficiency and Prompting

* 34. How does Parameter-Efficient Fine-Tuning (PEFT) prevent catastrophic forgetting?

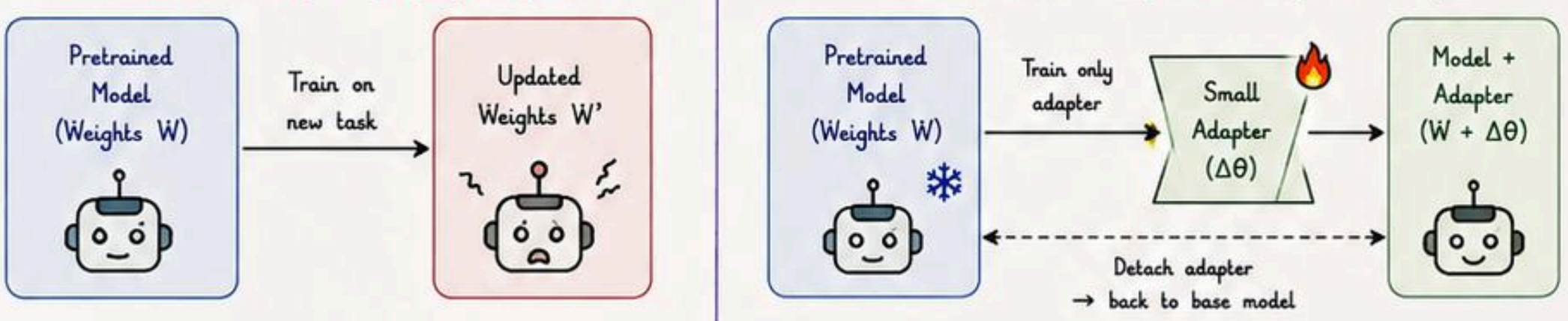
Because the original weights are frozen. In full fine-tuning, gradient descent modifies the very parameters that encode the model's general knowledge, so that knowledge can be degraded. PEFT methods, including LoRA, adapters, prefix tuning and prompt tuning, add a small number of new trainable parameters and update only those.

The pretrained knowledge is physically unchanged, so the worst case is that the adapter learns to override behaviour, not that the underlying capability is destroyed. Adapters can also be detached at inference, returning you instantly to base behaviour.

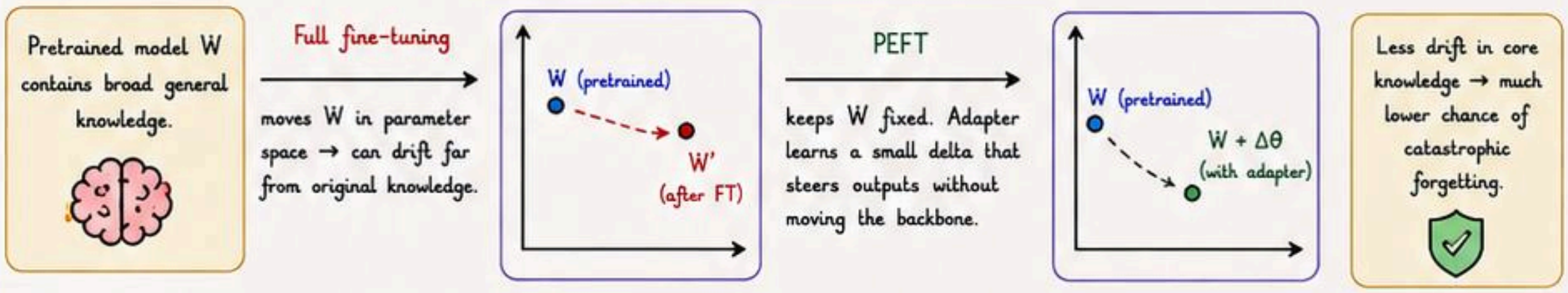
Remember

* Catastrophic forgetting happens when training on a new task reduces performance on previous tasks.
* Full fine-tuning updates all weights → risk of damaging general knowledge.
* PEFT updates only a small set of new parameters.
* Base model weights stay frozen and intact.
* Adapters can be removed → original behaviour comes back.

Full Fine-Tuning vs PEFT (Why forgetting happens less)



How PEFT reduces forgetting (intuition)



Interview edge:

* Be precise rather than absolute.
* PEFT dramatically reduces forgetting; it does not mathematically eliminate it.
* A strong adapter can still steer outputs away from general competence.
* Candidates who claim it fully prevents forgetting get pushed on and lose ground.
* Say: "substantially mitigates, and here is why it is not zero."

Rule of thumb

Freeze the past,
learn the delta,
keep the knowledge.



Section 4: Fine-Tuning, Efficiency and Prompting

* 35. What is model distillation, and how is it applied to LLMs?

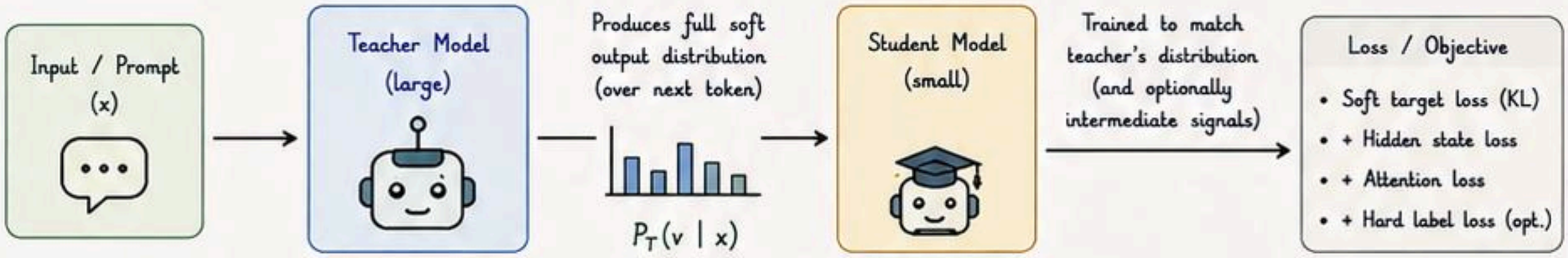
Distillation trains a smaller student model to reproduce the behaviour of a larger teacher. Rather than learning only from hard labels, the student matches the teacher's full soft output distribution, which carries much richer information about how the teacher weighs alternatives, and sometimes also matches intermediate hidden states or attention maps.

The result is a model that is far cheaper to serve while retaining a large share of the teacher's quality on the target domain.

Remember

* Student learns to mimic teacher behaviour.
* Soft targets > hard labels (more information).
* Can match outputs, hidden states, or attention.
* Goal: smaller, faster, cheaper with minimal quality loss.
* Teacher licence terms may restrict use of outputs for training competing models.

The core idea



Student learns what the teacher knows about all possible next steps, not just the final answer.

How it is applied to LLMs

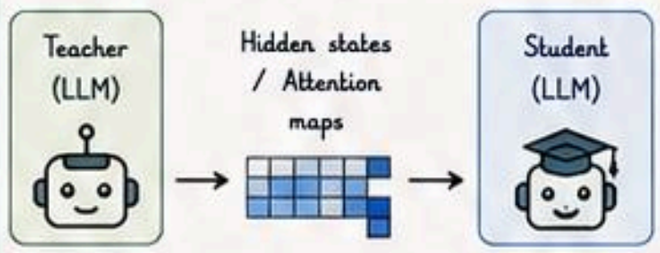
1. Standard (Logit) Distillation



KL divergence between teacher and student next-token distributions at every position.

$$L_{KD} = D_{KL}(P_T \parallel P_S)$$

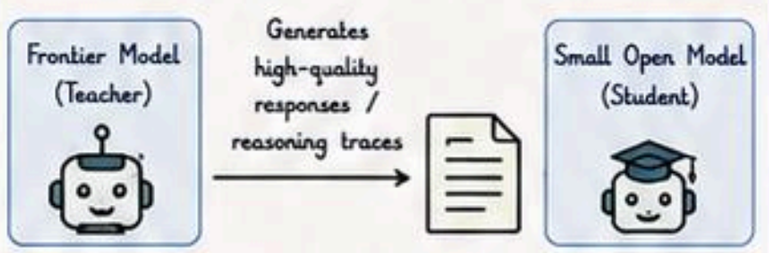
2. Intermediate Representation Matching



Student is trained to match hidden states or attention patterns at one or more layers.

$$L_{rep} = \| H_T - H_S \|_2^2 \text{ (+ attention loss)}$$

3. Sequence-Level (Modern LLM Distillation)

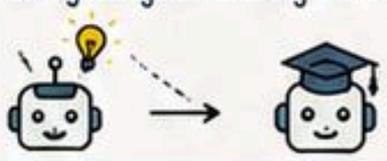


Use teacher to create a high-quality synthetic dataset. Fine-tune the small model on these sequences.

Examples: Llama 3.1 → distilled to 3B/7B models, Qwen-Max → distilled to Qwen-2.5 1.5B / 7B.

Why it works

- Soft distributions encode similarity between options.
- Teacher reveals "dark knowledge" (relative preferences).
- Student generalizes better than with hard labels.
- Intermediate signals guide learning more deeply.



Benefits

- Much smaller model → lower latency & cost.
- Higher throughput, fits on limited hardware.
- Retains most of the teacher's quality.
- Reduces dependency on expensive APIs.
- Enables on-device / on-prem deployments.



Caveats / Things to watch

- Student can inherit teacher's biases or mistakes.
- Quality limited by teacher & training data.
- Over-regularization → student too similar, not flexible.
- Licence/ToS may prohibit training on outputs.

Always check licence terms before collecting data from a hosted model.

Interview edge:

* Name sequence-level distillation: generate high-quality responses/traces from a frontier model and fine-tune a small open model.
* Most compact production models in Indian startups are built this way.
* It turns an expensive per-call API dependency into a cheap self-hosted model.
* Licence terms on the teacher matter → some providers restrict distillation.

Rule of thumb

Distillation trades a little capacity for a lot of efficiency. Right teacher + right data = powerful small model.



Section 4: Fine-Tuning, Efficiency and Prompting

* 36. What is overfitting, and how can it be prevented?

Overfitting is when a model learns patterns specific to the training set, including noise, so training loss keeps falling while performance on unseen data stagnates or worsens.

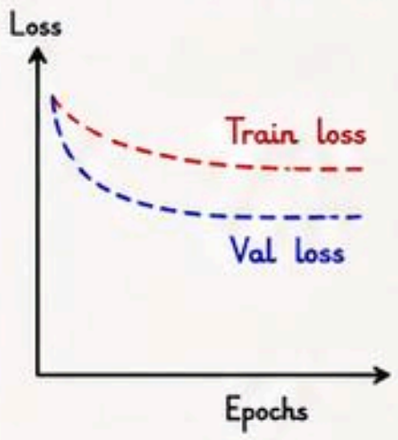
Standard prevention includes more and more diverse data, dropout, weight decay, early stopping on a held-out validation set, cross-validation and data augmentation.

Remember

- Fits noise in training data, not the true signal.
- Training loss ↓ but validation/test performance stagnates or gets worse.
- Goal: good generalization, not perfect memorization.
- More data + regularization + early stopping = better generalization.

Underfitting vs Overfitting (intuition)

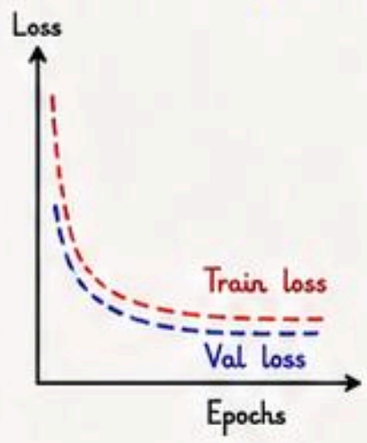
1. Underfitting (high bias)



Symptoms
- Both losses high
- Model too simple
- Can't capture pattern

Fix
Increase capacity, train longer, add better features.

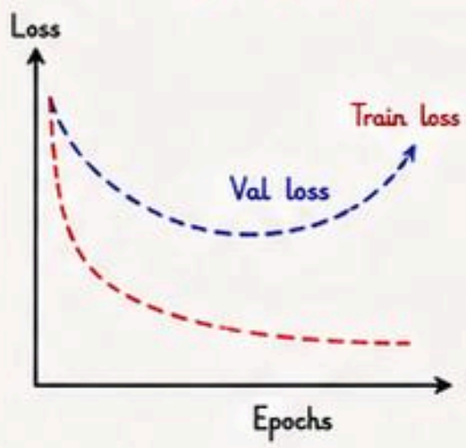
2. Good fit (sweet spot)



Symptoms
- Both losses low and close
- Good generalization

Fix
Maintain + monitor.

3. Overfitting (high variance)



Symptoms
- Train loss very low
- Val loss stops improving / rises
- Memorizes noise

Fix
More data, regularize, early stop, simplify.

Overfitting in LLM fine-tuning (what it looks like)

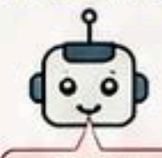
Real-world signs
- ⊗ Parrots training examples verbatim.
- ⊗ Loses format flexibility (answers only in your style).
- ⊗ Overly confident even when question is out of scope.
- ⊗ Struggles with rephrased or unseen prompts.
- ⊗ Better train loss but no real-world improvement.

Seen during training

Q: Explain API rate limiting in 3 bullet points.

Answer (in training data)
- Point A
- Point B
- Point C

→ Model (overfit)



- Point A
- Point B
- Point C
(Parrots!)

--------→ Better behaviour (generalized)



- Limit requests over time window.
- Protects backend from overload.
- Can return 429 with Retry-After header.

Practical controls for LLM fine-tuning

1. Keep epochs low



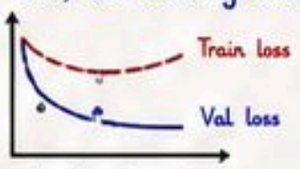
Often 1-3 epochs are enough.

2. Hold out eval set before training



Never touch it until the end.

3. Watch validation loss, not training loss



Early stop at the best val point.

4. Keep LoRA rank modest



Too high rank → more capacity to overfit.

5. Evaluate on OOD (out-of-distribution) prompts



Different topics, styles, languages, formats.

6. Regularize
- ✓ Dropout
- ✓ Weight decay
- ✓ Label smoothing
- ✓ Data augmentation

Pretraining-scale version: memorization



- ⊗ Model can memorize rare sequences from training data.
- ⊗ Leads to privacy leaks and PII exposure.
- ⊗ Riskier when data contains secrets, code, or user content.
- ⊗ Mitigation: data filtering, deduplication, access control, privacy checks, and responsible data governance.

Why it matters
- ✓ Better generalization → reliable in the real world.
- ✓ Avoids embarrassing failures and policy violations.
- ✓ Protects user data and company reputation.
- ✓ Saves compute by avoiding unnecessary re-training.



Interview edge:
- * Translate to LLMs: look for parroting, rigid style, and over-confidence.
- * Concrete levers: low epochs, held-out eval, watch val loss, modest LoRA rank.
- * Test on OOD prompts to confirm general ability survived.
- * Remember memorization at pretraining scale → privacy & licence implications.

Rule of thumb
Good models generalize. Overfit models just remember.



Section 4: Fine-Tuning, Efficiency and Prompting

* 37. What are Generative and Discriminative models?

A discriminative model learns the conditional probability of a label given the input, drawing decision boundaries between classes.

A generative model learns the joint distribution of inputs and labels, or the distribution of the data itself, which means it can produce new samples.

Logistic regression, SVMs and BERT classifiers are discriminative.

Naive Bayes, GANs, diffusion models and LLMs are generative.

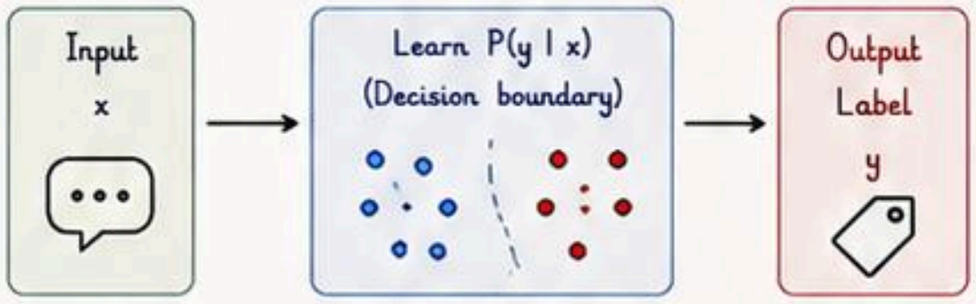
Remember

* Discriminative: learn P(y | x) → focus on boundary.
* Generative: learn P(x, y) or P(x) → model the data.
* Generative models can sample / generate.
* Discriminative models classify / predict labels.
* Generative models usually need more data and compute.

Core difference at a glance

Discriminative Model

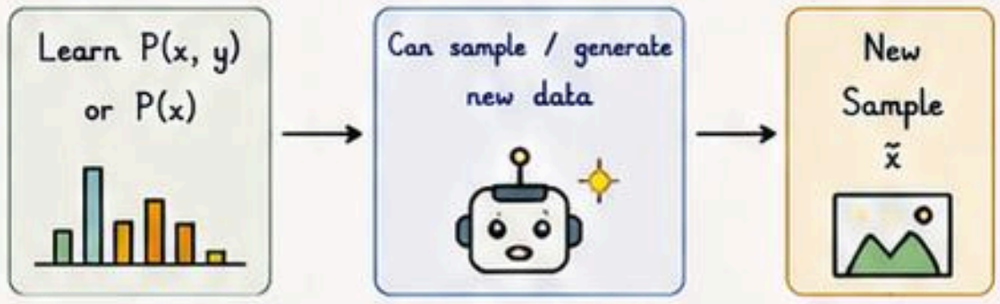
(answers: which bucket does this belong to?)



Goal: draw the best boundary between classes.
Focus: separate classes correctly.

Generative Model

(answers: what would a plausible example look like?)

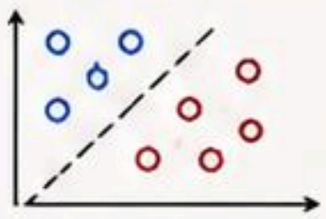


Goal: model the entire data distribution.
Focus: understand how data is produced.

Examples

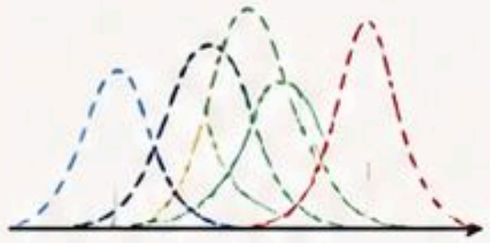
Discriminative models

- Logistic Regression
- SVMs
- BERT Classifier
- RoBERTa / DeBERTa (fine-tuned for classification)



Generative models (classic)

- Naive Bayes
- Gaussian Mixture Models
- HMMs



Generative models (modern)

- GANs (generate images, text, etc.)
- Diffusion Models (DALL-E, Stable Diffusion, etc.)
- LLMs (GPT, Llama, Mistral, etc.)



Typical tasks

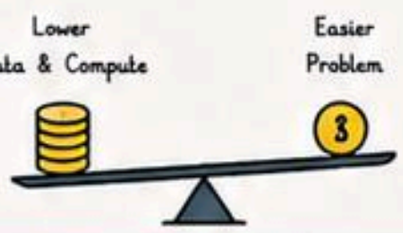
Discriminative:
- Sentiment classification
- Spam detection
- NER / intent detection
- Topic classification

Generative:
- Text / image generation
- Data augmentation
- Summarization
- Creative writing

Practical consequences

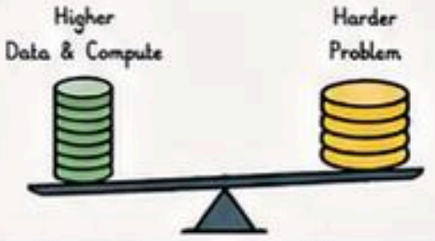
Discriminative models

- Need less data to train well.
- Cheaper to train.
- Faster to converge.
- Optimized for prediction accuracy.
- Do not generate new data.



Generative models

- Need far more data.
- Much higher compute.
- Slower to train.
- Can model uncertainty & diversity.
- Can generate new, plausible samples.



Modelling an entire distribution is a much harder problem than separating a few classes.

Interview edge:

* Discriminative → "which bucket?" (classification).
* Generative → "what would a plausible example look like?" (generation).
* Generative models are more powerful but costlier to train.
* More data + more compute + more patience.

Rule of thumb

Use discriminative models when your goal is prediction.
Use generative models when your goal is creation.



Section 4: Fine-Tuning, Efficiency and Prompting

* 38. Difference between Discriminative AI and Generative AI?

Discriminative AI predicts, scores and classifies against defined outcomes: fraud or not fraud, churn risk, credit score, defect detection. Generative AI produces new artefacts: text, code, images, audio, structured documents.

Discriminative outputs are typically evaluated with precision, recall and AUC, and are comparatively easy to validate. Generative outputs often have no single correct answer, which makes evaluation, guardrails and human review the hard part of the project.

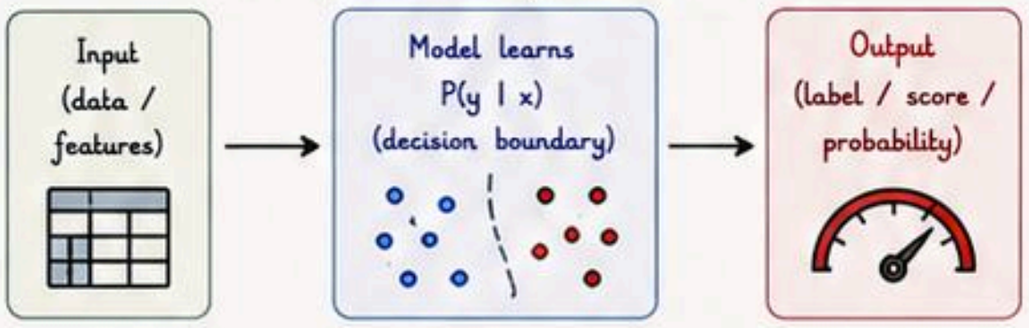
Remember

* Discriminative AI → classify / predict / score.
* Generative AI → create / produce / generate.
* Discriminative = "which bucket?"
* Generative = "what should we create?"
* Most enterprise systems use both together.

Discriminative AI vs Generative AI at a glance

Discriminative AI (Predictive)

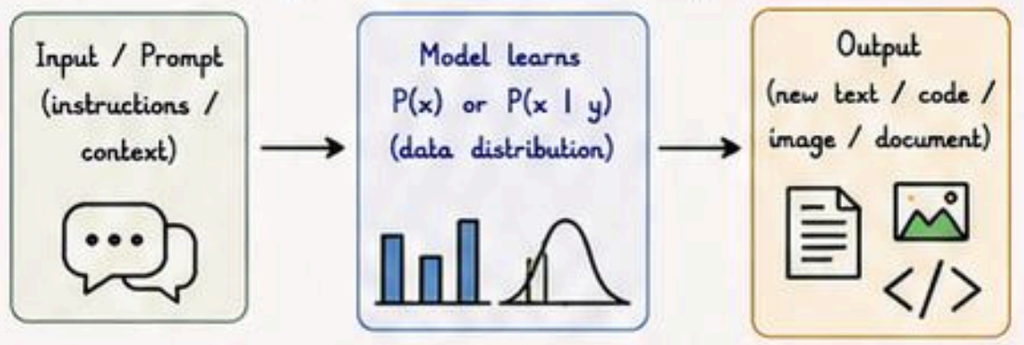
Focus: predict the right label / decision



Goal: choose the correct bucket.
Example: fraud or not fraud.

Generative AI (Creative)

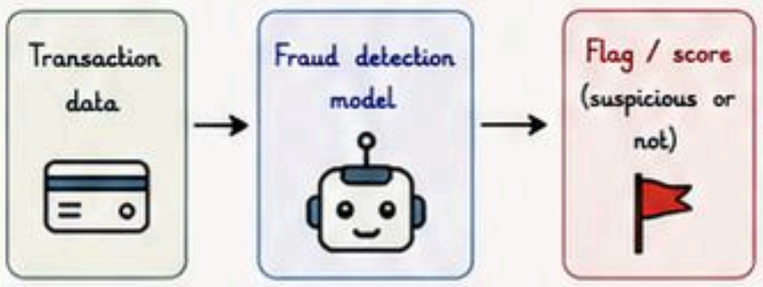
Focus: generate new content / artefacts



Goal: create something plausible.
Example: investigation summary, email, report.

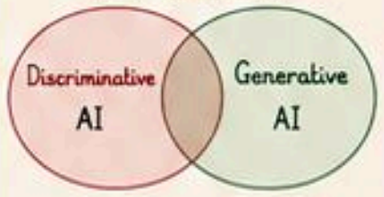
Business view (BFSI example)

Discriminative AI (flags & decides)



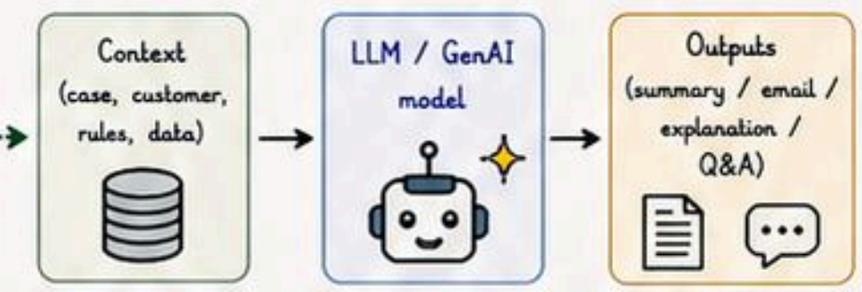
Example: model flags a suspicious transaction.

Combined system (what most enterprises run in 2026)



Predict → Generate → Act

Generative AI (explains & creates)



Example: model drafts investigation summary, explains the flag to customer, answers analyst questions.

Key differences

Aspect	Discriminative AI	Generative AI
Purpose	Predict, classify, score	Generate, create, produce
Output	Label, probability, score	Text, code, image, audio, doc
Typical tasks	Fraud detection, churn, credit scoring, defect detection	Summarization, content creation, code gen, report drafting
Evaluation	Precision, Recall, F1, AUC, Confusion matrix	Human eval, factuality, relevance, fluency, safety, guardrails
Correct answer	Usually one right answer	Often no single correct answer
Data & compute	Less data, less compute	More data, more compute
Validation	Easier and objective	Harder and more subjective

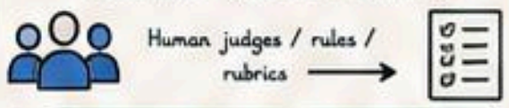
Evaluation contrast

Discriminative (easy to validate)

Prediction ✓ = Actual ✓

Use metrics: Precision, Recall, F1, AUC

Generative (hard to validate)



Use metrics: Quality, Relevance, Faithfulness, Safety, Coherence

Why it matters

- Discriminative models are reliable for decisions and SLAs.
- Generative models unlock new user experiences and productivity.
- Generative requires guardrails, human review and continuous eval.
- The best products combine both for accuracy + experience.

Interview edge:

* In BFSI: detect the suspicious transaction (discriminative) → draft the investigation summary, explain to customer, answer follow-ups (generative).
* In Healthcare: detect high-risk patient (discriminative) → generate discharge summary, patient instructions, doctor notes (generative).
* In E-commerce: predict return risk (discriminative) → generate refund email, product description, support reply (generative).

Rule of thumb

Use discriminative for decisions with clear ground truth. Use generative for creation where humans judge quality.

Section 4: Fine-Tuning, Efficiency and Prompting

* 39. How does prompt engineering influence the output of LLMs?

Mechanically, the prompt is the conditioning context for next-token prediction, so every token you add shifts the probability distribution over everything the model produces afterwards.

That is why role framing, explicit constraints, provided context, worked examples and a specified output schema change results so sharply.

Good prompting is largely about removing ambiguity the model would otherwise resolve by guessing.

Remember

- LLMs predict the next token given all previous tokens.
- The prompt = the context that conditions predictions.
- Small prompt changes → big output differences.
- More structure & clarity → less guessing → more reliable outputs.
- Ambiguity in → ambiguity out.

How the prompt shapes the model's output (intuition)

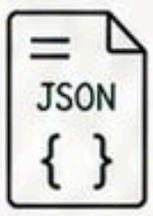


Change the context → change the probabilities → change every token that follows.

Prompting techniques that actually matter in production

1. Specify output format explicitly

Use JSON schema or structured-output mode.



- ✓ Machine-readable
- ✓ Validatable
- ✓ Reliable

Don't leave format to chance.

2. Stable order for instructions & context

Put instructions first, then context, examples, then question.

1. Role / Instructions
2. Long Context
3. Examples (few-shot)
4. User Question

Stable order → better cache hits → lower cost & latency.

3. Use few-shot examples that cover edge cases

Show the model the weird stuff, not just the easy path.



Good coverage:

- ✓ Typical case
- ✓ Edge / rare case
- ✓ Negative case

Better examples → better generalization.

4. State what to do when answer is not in context

Force the model to say "I don't know" instead of hallucinating.

If the answer is not in the provided context, respond with: "I don't know."



Honesty > confidence.

5. Version prompts & evaluate like code

Prompts change behaviour. Treat them like code: version, test, review.

git

- ✓ Version control
- ✓ Evaluation suite
- ✓ Regression tests

Untested prompt change = untested code change.

What changes the output the most?

- Role framing (e.g., "You are a senior risk analyst").
- Clear, explicit constraints.
- High-quality, relevant context.
- Few-shot examples (especially edge cases).
- Strict output schema.
- Temperature and decoding settings (also matter!).



Bad vs Good prompt (example)

Bad (vague)

Explain churn risk.
Context: {long text}
Answer:



→

Good (structured)

Role: You are a churn analyst.
Context: {long text}
Task: Identify top 3 churn reasons with supporting evidence.
Format: Return JSON with fields {reason, evidence, score}.



Reliability over cleverness

- Clever prompts impress once.
- Reliable prompts ship every day.
- The goal is boring, repeatable, correct outputs at scale.



Interview edge:

- Prompt engineering is about reliability and control, not trick phrases.
- Structure your prompt for clarity, determinism and evaluation.
- Treat prompts like code: version, test, measure, iterate.
- Good prompts reduce hallucinations, improve quality, and lower total cost.

Rule of thumb

Be explicit.
Be structured.
Be testable.



Section 4: Fine-Tuning, Efficiency and Prompting

✱ 40. What is Chain-of-Thought (CoT) prompting?

Chain-of-thought prompting asks the model to produce intermediate reasoning steps before its final answer, rather than jumping straight to a conclusion.

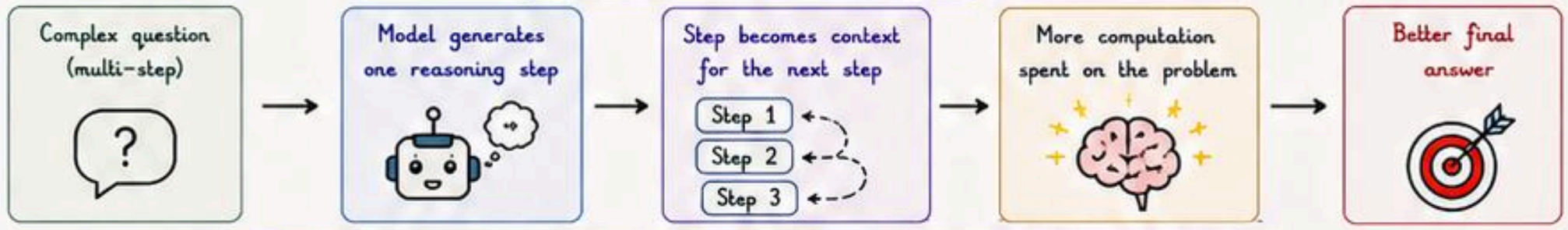
It measurably improves performance on multi-step arithmetic, logic and structured planning tasks, because each generated step becomes context the model can condition on, effectively giving it more computation to spend on the problem.

Zero-shot CoT achieves much of this with a simple instruction to reason step by step; few-shot CoT supplies worked examples.

Remember

- ✱ CoT = show your working, then give the answer.
- ✱ Each step → more context → better decisions.
- ✱ Zero-shot CoT = just ask to reason step by step.
- ✱ Few-shot CoT = give worked examples.
- ✱ Great for math, logic, planning, troubleshooting.

Why CoT works (intuition)



CoT lets the model think out loud, so every thought becomes useful context.

CoT in action (example)

Zero-shot CoT

Instruction: "Let's think step by step."

Q: A train travels 120 km in 2 hours. It then travels 150 km in 3 hours. What is the average speed over the whole journey?

A:
① Total distance = 120 + 150 = 270 km
② Total time = 2 + 3 = 5 hours
③ Average speed = 270 / 5 = 54 km/h

Answer: 54 km/h ✓

Few-shot CoT

Instruction: "Here are examples. Now solve."

Example 1: ... (worked steps) ...
Example 2: ... (worked steps) ...
Q: 17 × 24 = ?
A:
① 17 × 24 = 17 × (20 + 4)
② = 17 × 20 + 17 × 4
③ = 340 + 68 = 408

Answer: 408 ✓

Direct answer (no CoT)

Instruction: "Answer directly."

Q: A train travels 120 km in 2 hours. It then travels 150 km in 3 hours. What is the average speed over the whole journey?

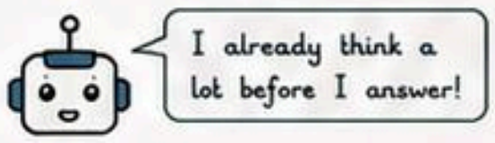
A: 54 km/h

(Short, but accuracy is often lower on harder problems.) ✗

Interview edge: what actually matters in 2026

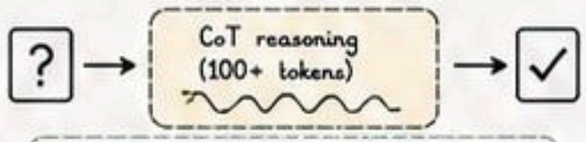
1. The landscape moved

- ✓ Modern reasoning models already perform extended internal deliberation by default.
- ✓ Manually adding "think step by step" adds less than it did in 2023.
- ✓ It can sometimes hurt, by forcing verbose or off-track reasoning.



2. Production cost: think before you think

- ✓ Reasoning tokens are billed tokens.
- ✓ More tokens → higher cost.
- ✓ More tokens → higher latency.
- ✓ For high-volume, low-complexity calls, CoT is often the wrong trade.



Use CoT where accuracy gain > cost.

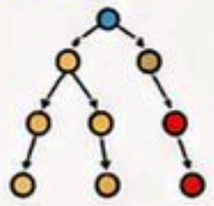
3. Know the family of techniques

CoT (single path)



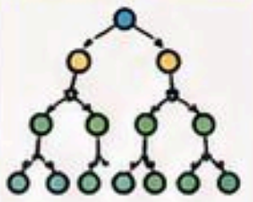
One reasoning path → final answer.

Self-Consistency (many paths)



Majority vote final answer.

Tree-of-Thought (branch & search)



Explore branches, evaluate, pick best.

When to use CoT

- ✓ Multi-step math / logic
- ✓ Complex planning
- ✓ Troubleshooting
- ✓ Legal / policy reasoning
- ✓ Low-stakes, high-accuracy tasks
- ✗ Simple lookups / extraction



Good CoT prompt checklist

- ☑ State the goal clearly.
- ☑ Ask to reason step by step.
- ☑ Define the output format.
- ☑ Provide examples that cover edge cases.
- ☑ State what to do if the answer is not available.

Bad CoT prompt smells

- ✗ Vague instructions.
- ✗ No stop condition ("don't ramble").
- ✗ No output schema.
- ✗ Examples too easy / not representative.
- ✗ Unversioned, untested prompts.



Interview edge:

- ✱ CoT helps the model allocate more computation to hard problems.
- ✱ Use it deliberately: accuracy gain must beat cost (tokens + latency).
- ✱ Know the relatives: self-consistency (vote across paths), tree-of-thought (search branches).
- ✱ Version prompts, test changes, and measure accuracy vs cost.

Rule of thumb

Use CoT when the problem is hard enough to justify the extra tokens. Otherwise, keep it simple and fast.



Section 5: RAG, Architecture Scale and Production Reality

* 41. What are the key steps in the Retrieval-Augmented Generation (RAG) pipeline?

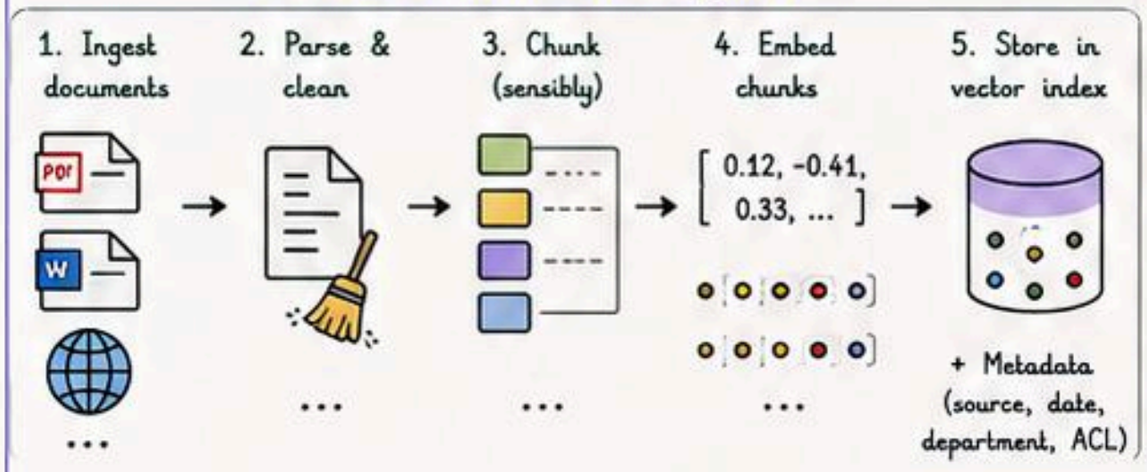
There are two phases. Indexing, done offline: ingest source documents, parse and clean them, chunk them sensibly, generate embeddings for each chunk with an embedding model, and store the vectors plus metadata in a vector index.

Then querying, at runtime: embed the user query, retrieve the most similar chunks, optionally rerank them with a cross-encoder, assemble the retrieved context into the prompt alongside instructions, generate the answer, and return it with citations back to the source chunks.

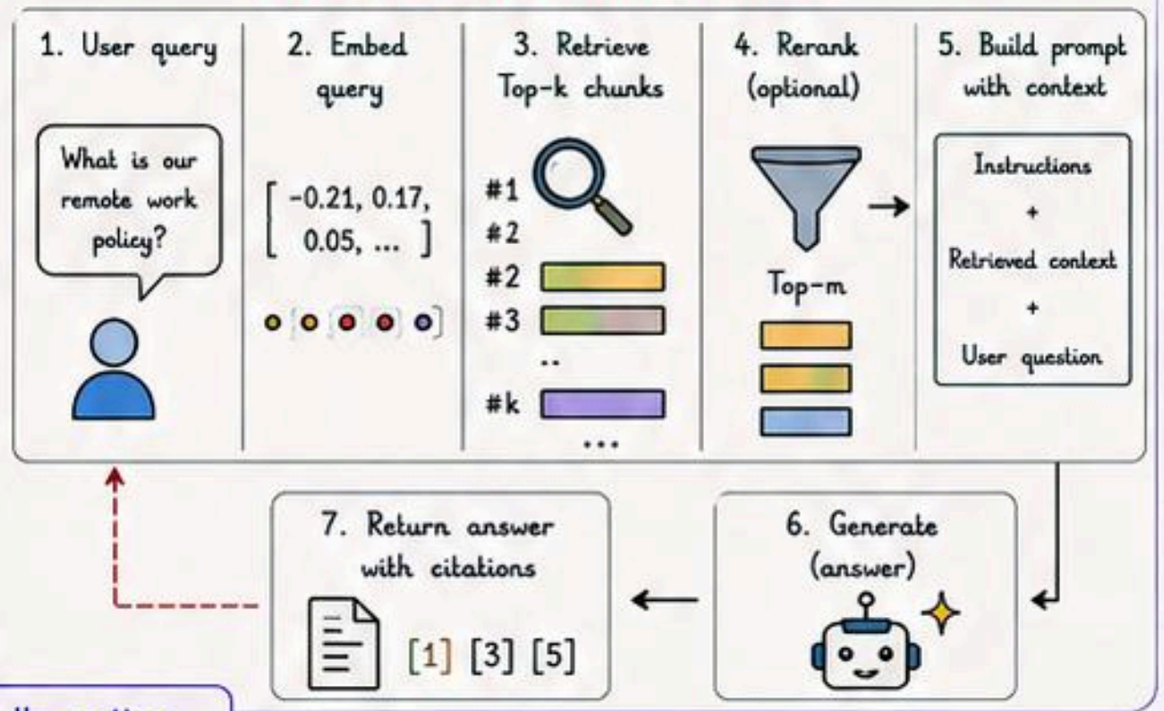
Remember

- RAG = Retrieve relevant context → Augment prompt → Generate.
- Indexing is offline (build once, update periodically).
- Querying is online (happens for every user request).
- Always return citations to build trust.
- Most failures come from retrieval, not generation.

PHASE 1: INDEXING (offline)



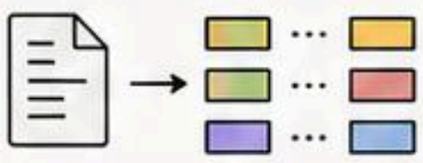
PHASE 2: QUERYING (runtime)



Interview edge: what actually matters

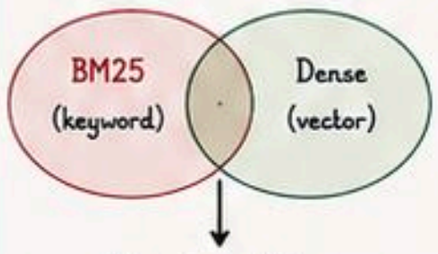
1. Chunking is make-or-break

- Naive fixed-size splitting severs tables, clauses, code blocks.
- Use semantic chunking with overlap.
- Respect structure: headings, sections, lists, tables.
- Keep chunks self-contained and meaningful.



2. Hybrid retrieval is essential

- Pure vector search misses exact identifiers and rare terms.
- Combine BM25 keyword search with dense vectors.
- Typical approach: retrieve from both → merge → dedupe.



3. Recall first, precision later

- Retrieval recall is usually the bottleneck, not the model.
- Over-retrieve (e.g., top 50–100) then rerank.
- Rerank with a cross-encoder for relevance.



4. Metadata filtering matters

- Filter by date, department, product, region, entitlement.
- Pre-filter before retrieval when possible.
- Keeps answers relevant and access-controlled.



5. Evaluate retrieval separately

- You can't fix what you don't measure.
- Measure retrieval quality apart from answer quality.
- Use both offline eval and spot-checks.

Retrieval metrics
- Context Precision @k
- Context Recall @k
- MRR / nDCG

Generation metrics
- Faithfulness
- Answer Relevance
- Helpfulness

Example: Chunking matters (policy document)

Bad (naive fixed split)

> ... Employees must get manager approval before working remotely. The request must be made in writing ...
>
> ... writing ... Employees must log hours daily in the HR system ...

Better (structure-aware)

> Section: Remote Work Approval
> - Employees must get manager approval
> - The request must be made in writing
>
> Section: Time Logging
> - Employees must log hours daily
> - in the HR system

Hybrid retrieval example

Query: "leave encashment rules"

BM25 results	Vector results
1. Leave Policy.pdf	1. Leave Policy.pdf (p. 12)
2. HR Circular 2024-01	2. Leave Encashment Guide
3. Payroll FAQ	3. HR Benefits Handbook

→ Merged + deduped candidates → Rerank → Top 5 used in prompt

What a good answer returns

Answer: Employees can encash up to 60 days of earned leave on separation, subject to manager approval ...

Sources:
[1] Leave Policy.pdf (p. 12)
[3] HR Circular 2024-01
[5] Benefits Handbook (p. 45)

Interview edge:

- Chunk well. Retrieve broadly, Rerank sharply. Filter by metadata.
- Test retrieval, not just answers.
- Citations build trust and help users verify.
- Bad retrieval can make a great model look wrong.

Rule of thumb

RAG quality = Retrieval quality × Context relevance × Model quality.



Section 5: RAG, Architecture Scale and Production Reality

✱ 42. How does knowledge graph integration help LLMs?

A knowledge graph stores entities and the explicit typed relationships between them, which gives the model something vector similarity cannot: precise, structured, multi-hop facts.

Vector retrieval finds text that resembles your query, but a question like "which suppliers to our Pune plant are also customers of our competitor" requires traversing relationships, not matching semantics.

Graphs also provide provenance and explainability, since you can show the exact path used, and they make constraints and hierarchies enforceable.

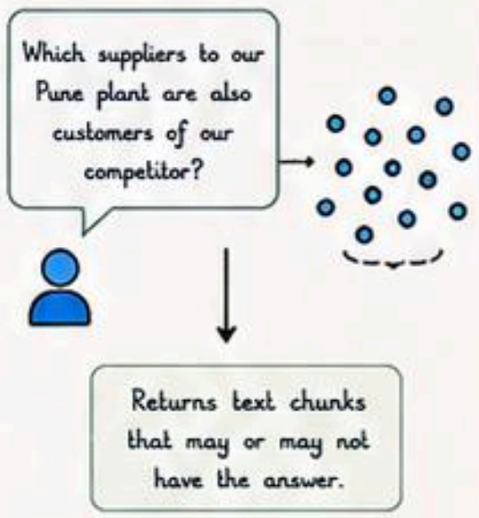
Remember

- ✱ Graphs store entities + relationships.
- ✱ They support multi-hop reasoning.
- ✱ They add provenance, constraints and hierarchies.
- ✱ Vector search = similarity. Graph search = structure.
- ✱ Best when your questions are about relationships.

Vector RAG vs Knowledge Graph RAG

Vector RAG (similarity)

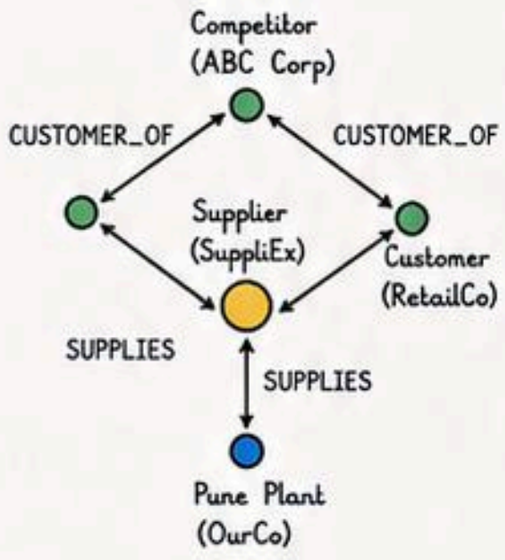
Finds chunks that look similar to the query.



May miss the exact facts or mix unrelated info.

Knowledge Graph RAG (structure)

Traverse relationships to reach the exact answer.



Traverses relationships and returns precise, explainable answer.

How a Graph helps the LLM (intution)

Multi-hop reasoning
Follow chains of typed relationships.

Example:
Plant → Supplier → Customer → Competitor

Provenance & explainability
Show the exact path and sources.

Example path:
OurCo Pune Plant → supplies → SuppliEx → customer_of → ABC Corp

Constraints & hierarchies
Enforce rules, domains and access control.

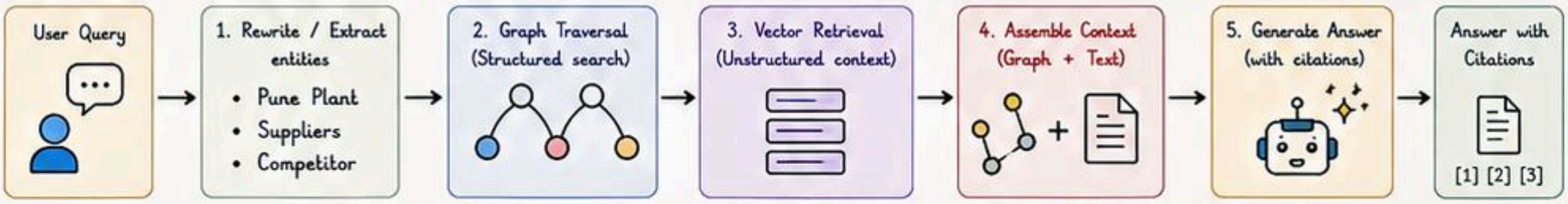
Example:
Only show suppliers active in India under approved categories.

Cleaner context for LLM
Less noise, more precise facts → fewer hallucinations.

Result:
Higher factual accuracy and more consistent answers.

GraphRAG: the modern approach

GraphRAG combines graph traversal with vector retrieval.



Structure gives precision. Text gives breadth. Together they give the best answers.

Why it works (benefits)

- Improves multi-hop and aggregate queries.
- Reduces hallucination (facts come from curated graph).
- Better aggregation (counts, sums, rollups).
- Handles synonyms via graph aliases.
- Great for explainability and compliance.

Honest counterweight (costs)

- Needs ontology design (entities, relations, rules).
- Data extraction and entity linking is hard.
- Requires continuous curation and updates.
- More infra to run (graph DB + retrievers).
- Higher upfront cost and time-to-value.

When to use what?

Scenario	Start with	Add Graph when...
Q&A over docs	Vector RAG	Many relationship questions
Policies / SOPs	Vector RAG	Need constraints & hierarchies
Product / Catalog	Hybrid	Cross-entity navigation needed
Enterprise Ops	Hybrid	Multi-hop critical to decision
Highly dynamic data	Vector RAG	Graph cost > benefit

Interview edge:

- ✱ Know where RAG breaks: chunking, retrieval recall, filtering, and evaluation.
- ✱ Hybrid retrieval (BM25 + vectors) is usually necessary.
- ✱ Over-retrieve → rerank → filter by metadata.
- ✱ Measure: context precision/recall, answer faithfulness, not just model accuracy.

Rule of thumb

Use GraphRAG when answers depend on relationships. Use Vector RAG when answers live in the text. Combine both when you can.

Section 5: RAG, Architecture Scale and Production Reality

* 43. How does Gemini's architecture improve training efficiency and stability compared to other multimodal LLMs?

Based on what Google has published, three things stand out.

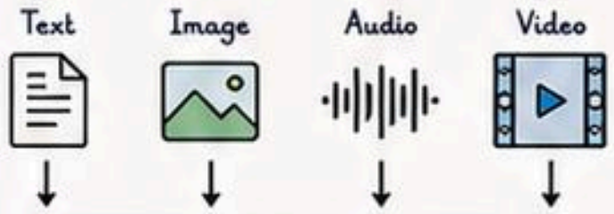
① Natively multimodal pretraining
② Sparse Mixture-of-Experts (MoE)
③ Training at scale on Google TPU with stability engineering

Remember

* Natively multimodal from the start.
* MoE gives more parameters, same compute per token.
* TPU + reliability engineering = fewer interruptions.
* Don't over-claim: not all internals are public.

① Natively multimodal pretraining

Gemini is pretrained jointly on interleaved text, image, audio and video. The model learns a shared representation space across modalities from the beginning.

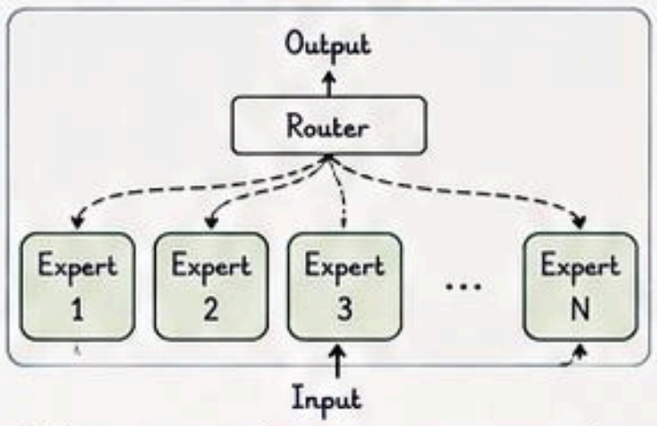


✓ Better cross-modal reasoning
✓ No post-hoc alignment of separate encoders
✓ Fewer interfaces to maintain and fine-tune

② Sparse Mixture-of-Experts (MoE)

Many Gemini models use sparse MoE layers. Total parameter count can be large, but only a small subset (experts) is activated per token, keeping compute per token bounded.

MoE layer (example)



✓ More capacity without linear compute growth
✓ Experts specialize, improving quality
✓ Compute cost ⇝ constant per token

③ TPU pods + stability engineering

Gemini training ran on Google's TPU pods with a stack engineered for large-scale stability and fast recovery from failures.

Massive parallelism
Tens of thousands of TPU chips working in parallel.

Reliability engineering
Fast detection, automatic restart and recovery from hardware or software faults.

Less downtime
More stable runs over weeks, less wasted compute.

Why this improves training efficiency and stability

Better learning signal	More capacity, same compute	Fewer failures, faster recovery	Higher utilization	Better outcomes
Joint multimodal data gives richer supervision and stronger representations.	MoE increases model size without increasing FLOPs per token.	Reliability stack reduces interruptions and keeps accelerators productive.	Less downtime → more useful training steps per wall-clock hour.	Higher quality models, faster iteration and lower total cost per run.

What Google has said vs. what is not public

What has been published

✓ Gemini is natively multimodal from pretraining.
✓ Gemini models (e.g., 1.5 Pro, 1.5 Flash) use Mixture-of-Experts.
✓ Training used large-scale TPU pods.
✓ Google has published technical reports describing high-level techniques, training data at a high level, and safety practices.

What has not been published (as of now)

✗ Exact layer counts, hidden sizes, and full architecture diagrams.
✗ Number of experts, expert sizes, routing details.
✗ Exact dataset composition and proportions.
✗ All optimization tricks, code, and system internals.

Interview edge: the right way to answer



✓ Stick to what the technical reports and Google have stated.
✓ Explain the principlers and why they lead to better efficiency and stability.
✓ Call out the limits of public information.
✓ Show integrity: "Here's what we know, here's what we don't."



Key Takeaways

* Natively multimodal pretraining → better cross-modal understanding.
* MoE → more parameters, same compute per token.
* TPU + reliability engineering → stable, efficient training at massive scale.
* Know the public facts, respect what is not published.

Rule of thumb

Architectures are only one part of the story. Data quality, training scale, and engineering reliability determine results.



Section 5: RAG, Architecture Scale and Production Reality

* 44. How does Mixture of Experts (MoE) improve scalability?

A dense model uses every parameter for every token. An MoE model replaces feed-forward blocks with a set of parallel expert networks plus a lightweight router that selects a small number of experts (often 1 or 2) for each token.

Total parameters (and therefore total capacity) grow substantially while the compute spent per token stays close to that of a much smaller dense model. That decouples model capacity from inference cost → the core scaling win.

Remember

* Dense model = use all parameters for every token.
* MoE = route each token to only a few experts.
* More capacity, same (or similar) compute per token.
* Decouples model capacity from inference cost.
* 1–2 experts per token is common in practice.

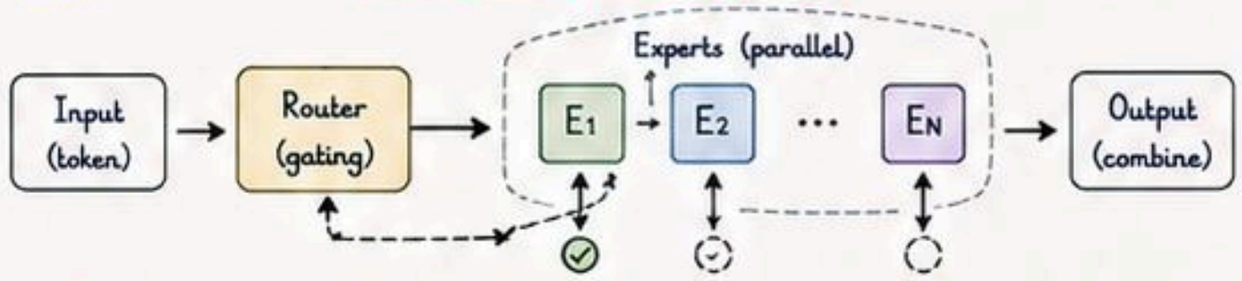
vs.

Dense Model (for each token)



- Every token uses the entire FFN.
- Compute per token grows with model size.
- Capacity and cost are tightly coupled.

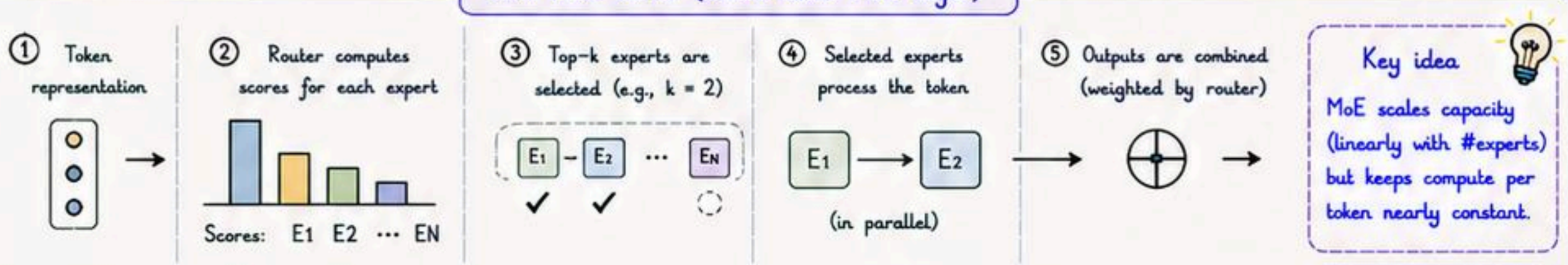
MoE Model (for each token)



- Router selects a small number (k) of experts, e.g., k = 1 or 2.
- Only selected experts run; others are skipped.
- Huge total capacity with similar compute per token.

Selected for this token Not selected (skipped)

How MoE works (inside each MoE layer)



Key idea

MoE scales capacity (linearly with #experts) but keeps compute per token nearly constant.

The costs and challenges (interview edge)

1. High memory footprint

All experts must be loaded in memory even if only a few run.

VRAM usage ≈ total parameters, not active parameters.

2. Router can collapse

Router may send most tokens to a few "favourite" experts.

Needs auxiliary load-balancing losses during training to keep experts well-utilized.

3. Harder to serve

Expert parallelism across devices introduces:

- Communication overhead
- Uneven batch utilization and straggler issues

4. Trickier fine-tuning

Routing patterns can shift during fine-tuning.

Requires careful LR, regularization and often more data.

LoRA/QLoRA support is improving but still harder than dense models.

5. More moving parts

More hyperparameters (k, #experts, capacity factor, aux loss weight).

More things can go wrong!

When MoE is a great trade

- You need much more capacity but have a fixed compute budget.
- Large-scale pretraining where scaling laws demand capacity.
- Workloads are diverse; experts can specialize.
- You can invest in serving infrastructure and operability.



When it might not be worth it

- You are memory-constrained.
- Model is small-medium scale.
- Workload is simple or highly uniform.
- You need the simplest serving stack possible.



Interview edge:

* MoE decouples capacity from compute → the core scalability win.
* But memory, routing, load balancing and serving are real costs.
* Always evaluate end-to-end: quality, latency, VRAM, and ops complexity.
* Know your workload: not every problem needs hundreds of experts!

Rule of thumb

MoE shines when you need 10x more capacity without 10x more compute → and you can afford the engineering complexity.



Section 5: RAG, Architecture Scale and Production Reality

* 45. What is zero-shot learning in LLMs?

Zero-shot learning means the model performs a task from an instruction alone, with no examples in the prompt and no task-specific training.

It works because instruction tuning taught the model to follow directions in general, and because pretraining exposed it to enough varied text that many tasks are already latent in its representations.

Remember

- No examples in the prompt.
- No task-specific training.
- Just an instruction and (optionally) some context.
- Relies on instruction tuning + broad pretraining.
- Start simple, escalate only when needed.

Zero-shot in action (just an instruction)

Summarization



Instruction:
"Summarize the following text in two sentences."

Input:
<long article here>

Model output:
(two-sentence summary)

Classification



Instruction:
"Classify the sentiment of the following review as Positive, Neutral or Negative."

Input:
"The product is okay, but delivery was late."

Model output:
Neutral

Translation



Instruction:
"Translate the following English sentence into French."

Input:
"Customer trust is earned."

Model output:
"La confiance des clients se mérite."

Information extraction



Instruction:
(Extract the invoice number, date and total amount from the text below. Return JSON."

Input:
<invoice text >

Model output:
{"invoice_number": "INV-1023", "date": "2024-07-05", "total": 1250.00}

The decision rule: how to escalate

1. Start with zero-shot

- Fastest to build
- Fewest tokens
- Lowest latency
- Surprisingly capable

Use when:
The task is clear, the output format is simple, and quality is acceptable.

2. Move to few-shot

Ex 1
Ex 2
...
Ex n

- Adds examples in the prompt
- Improves format consistency
- Covers tricky / edge cases

Use when:
Zero-shot fails on edge cases, or the model misunderstands the desired format.

3. Move to fine-tuning

- Task-specific training
- Most reliable
- Higher cost and effort

Use when:
Few-shot still misses your quality bar, or the prompt (with examples) is too long and expensive to run.

Escalation guide

Approach	When it works well	Common failure mode	What to try next	Cost / effort
Zero-shot	Clear instructions, common tasks, good training coverage	Wrong format, misses edge cases, inconsistent detail	Add a few examples (few-shot) Clarify constraints in the prompt	Lowest (Few tokens, fastest)
Few-shot	When examples disambiguate or set the right pattern	Still fails on hard cases, complex reasoning gaps	Add more / better examples Or fine-tune for the task	Medium (More tokens, longer prompt)
Fine-tuning	When you need highest reliability or custom behavior	Overfitting, data cost, maintenance overhead	Improve data, regularize, monitor and evaluate	Highest (Train time + infra + ops)

Interview edge:

- Zero-shot is the default starting point — always test it first.
- Escalate only with evidence, not because "fine-tuning is better".
- Choose the cheapest approach that reliably meets the quality bar.
- Track quality, latency, and token cost as you iterate.



Rule of thumb

Zero-shot → Few-shot → Fine-tune
Only move right when the previous step fails your quality bar or costs too much.



Section 5: RAG, Architecture Scale and Production Reality

✱ 46. What is few-shot learning in LLMs and why is it useful?

Few-shot learning means we put a few input-output examples in the prompt. The model infers the pattern and applies it to the new input. No model weights change—this is in-context learning, not training.

It is useful because it:

- ✓ Teaches format and conventions instantly
- ✓ Needs no training infrastructure
- ✓ Can be edited in seconds
- ✓ Works well with a handful of labelled examples

Remember

- ✱ Examples live in the prompt, not in the weights.
- ✱ Model adapts during the forward pass only.
- ✱ Quality of examples > quantity of examples.
- ✱ Every example is tokens on every call.

Few-shot prompting in action

Task:

Classify the sentiment of the review as Positive, Neutral or Negative.

Examples (in the prompt)	Input (review)	Output (label)
Example 1	I love the battery life and the camera quality is excellent!	Positive
Example 2	The product is okay, does the job. Nothing special.	Neutral
Example 3	Terrible customer service and the app keeps crashing.	Negative

→ New input (not seen in examples): Delivery was on time and the product works as expected, but the packaging was poor. → Model output: Neutral

The model infers the pattern from the examples and applies it to the new input.

Why is it useful?

Instant adaptation
Get good results in minutes, not weeks.

No training needed
No GPUs, no data pipeline, no fine-tuning.

Easy to iterate
Change examples or instructions in seconds.

Works with small data
Effective when you have a handful of labelled examples, not thousands.

The craft: how to do few-shot well

1. **Choose the right examples**
   Cover hard, ambiguous and edge cases, not just the obvious ones.
2. **Keep it balanced**
   Balance label distribution; ordering and skew bias the model.
3. **Keep formatting identical**
   The model copies structure—be consistent in delimiters, casing, spacing and schema.
4. **Show the full reasoning (if needed)**
   If you want step-by-step or a specific format, show it in the examples.
5. **Don't overdo it**
   Too many examples dilute the signal and waste tokens.

The limits of few-shot

Token cost on every call
Each example is tokens every single time. At scale, this gets expensive.

Context window pressure
Examples compete with retrieved context and user input.

Inconsistency at the edge
Rare or very complex cases may still fail.

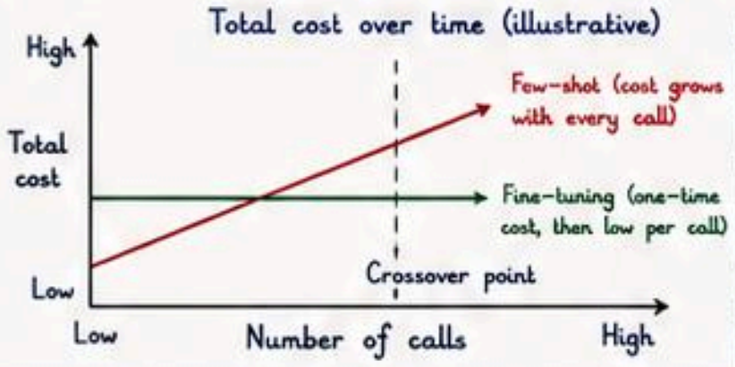
Not a replacement for learning
If the task needs deep domain knowledge, relationships or generalization—fine-tune.

When does few-shot become more expensive than fine-tuning?

Few-shot (in-context)

Examples in every prompt → Cost repeats on every call

Total cost over time (illustrative)



Fine-tuning

One-time training cost → Low cost per subsequent call

Factors that shift the crossover

- Number and length of examples
- Average prompt size
- Model pricing (input/output tokens)
- Call volume and frequency
- Cost of training and infra

Below the crossover: use few-shot. Above the crossover: fine-tune.

Interview edge:

- ✱ Start with zero-shot.
- ✱ Add few-shot when format or edge cases fail.
- ✱ Fine-tune only when few-shot cannot meet the quality bar or costs more at scale.
- ✱ Think in total cost and latency, not just accuracy.

Rule of thumb

Examples teach.
Tokens cost.
Training amortizes.
Pick the cheapest path that meets the quality bar.

Section 5: RAG, Architecture Scale and Production Reality

* 47. What is a context window?

The context window is the maximum number of tokens a model can attend to in a single forward pass.

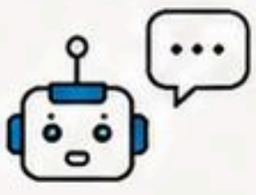
It covers everything the model can "see": instructions, history, retrieved content, tools, and the answer it generates.

Exceed it and something must be dropped, summarized, or truncated.

Remember

- Context window = the model's working memory.
- More room, but not infinite attention or perfect recall.
- You still need to curate what goes in.
- Every token in the window costs time and money.

What counts toward the context window?

System prompt	User instructions	Conversation history	Retrieved documents (Chunks)	Tool definitions & schemas	Tool outputs (Results)	Assistant response (Generated tokens)
						

All of the above competes for the same budget (measured in tokens).

Marketing number vs. useful number

Marketing number (the max window)

- The advertised maximum (e.g., 1M tokens).
- The hard limit of the model.
- Great for fitting more information.

Useful number (the effective window)

- The range where quality actually stays high.
- Degrades as content gets buried in the middle.
- Depends on task, model, and placement.

Bigger windows don't mean the model reasons equally well across all of it.

Why large context is not a silver bullet

Lost-in-the-middle problem
Retrieval accuracy and reasoning quality drop for information buried in the middle of very long inputs.

Higher cost and latency
Processing time and cost increase roughly with input length. Attention cost is quadratic in sequence length ($O(n^2)$).

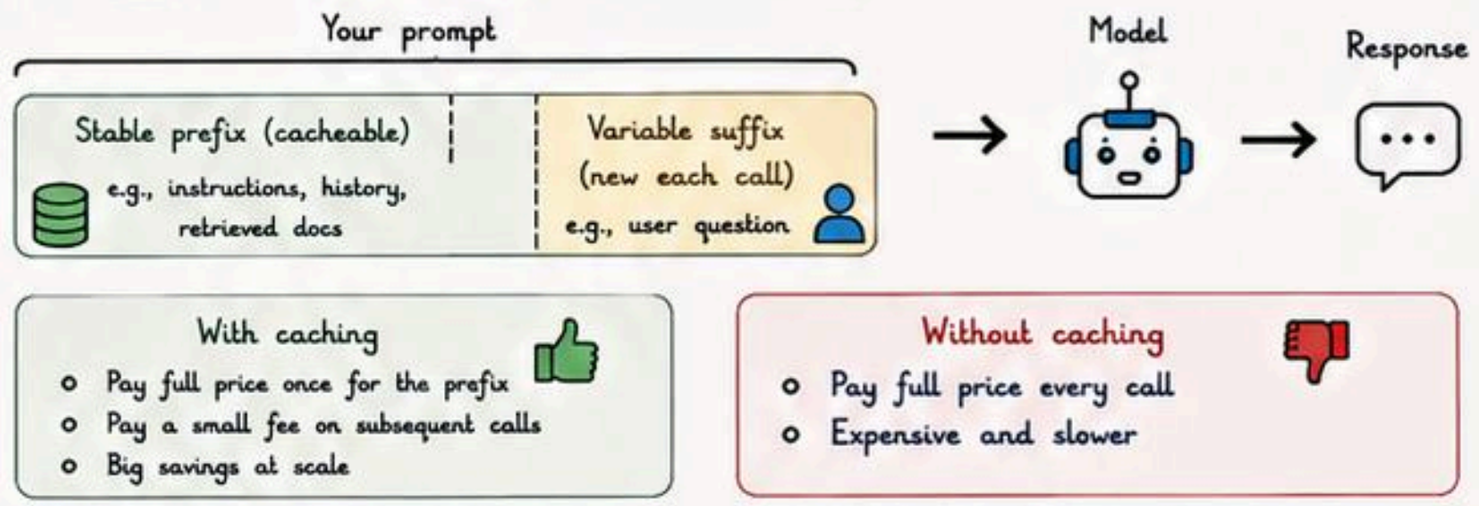
Does not replace RAG
Passing an entire corpus is not feasible or effective. RAG still selects the right few chunks.

Diminishing returns
Beyond a point, more context adds noise and distraction, not better answers.

Prompt caching: the single most effective cost lever

Prompt caching lets the provider cache the computation for a long, stable prefix of your prompt.

- Reused across calls
- Dramatically cheaper
- Lower latency
- Ideal for chat history, instructions, and retrieved chunks that don't change



With caching
- Pay full price once for the prefix
- Pay a small fee on subsequent calls
- Big savings at scale

Without caching
- Pay full price every call
- Expensive and slower

Interview edge:

- Separate the marketing number from the useful number.
- Large context changes the trade-off, it doesn't remove the need for RAG.
- Cache aggressively, measure savings.
- Always design for what to include, what to summarize, and what to drop.

Rule of thumb
Fill your context window with curated, high-signal content. Not more.
The best answer is not in the longest prompt.

Section 5: RAG, Architecture Scale and Production Reality

* 48. What is a hyperparameter?

A hyperparameter is a configuration value you set before training rather than something the model learns from data.

Weights and biases are learned by the model.
Hyperparameters control how the model learns, fine-tunes, generates and how an entire system like RAG behaves.

Remember

* Set before training or inference; not learned.
* Control the process, not the model's knowledge.
* Huge impact on quality, stability, speed and cost.
* Must be tuned, versioned and documented.

Where hyperparameters show up in the stack

1. Model training (from scratch)



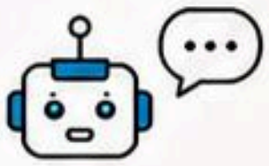
- Learning rate (and schedule)
- Batch size
- Number of epochs
- Optimizer (AdamW, SGD, etc.)
- Weight decay (L2)
- Dropout rate
- Gradient clipping
- Warmup steps
- Label smoothing
- Architecture choices (layers, hidden size, heads)

2. Fine-tuning (LoRA / PEFT)



- LoRA rank (r)
- LoRA alpha (scaling)
- LoRA dropout
- Target modules (e.g., q_proj, v_proj)
- Bias (trainable or not)
- Learning rate
- Batch size
- Max sequence length
- Number of epochs
- Adapter merge strategy

3. Inference / Decoding (generation)



- Temperature
- Top-p (nucleus sampling)
- Top-k
- Max output tokens
- Presence penalty
- Frequency penalty
- Repetition penalty
- Stop sequences
- Beam width (if used)
- Seed (for reproducibility)

4. Retrieval (RAG)



- Chunk size
- Chunk overlap
- Embedding model
- Top-k retrieved chunks
- Similarity threshold
- Reranker model
- Reranker threshold
- Metadata filters (e.g., date, dept, access)
- Hybrid weights (BM25 vs dense)

5. System / Ops



- Prompt template version
- Context window budget
- Caching strategy
- Rate limits / concurrency
- Max retries
- Timeouts
- Tool selection strategy
- Compression / summarization settings
- Cost controls (budgets)

Parameters vs. Hyperparameters

Parameters (learned)



- Weights and biases
- Learned from data
- Updated by gradient descent
- Define what the model knows

Hyperparameters (set by you)



- Configuration choices
- Set before training/inference
- Do not change via gradient
- Define how the model learns or is used

Why hyperparameters matter so much



Small changes can make or break performance.



Affect quality, stability, speed and cost.



Control trade-offs (e.g., quality vs latency vs cost).



The biggest gains often come from tuning not bigger models.

The tuning mindset

- Start with sensible defaults.
- Change one thing at a time.
- Use a validation set or eval suite.
- Measure, log and compare.
- Version every experiment.
- Choose the simplest setting that meets the bar.

Define goal → (e.g., accuracy, faithfulness, latency, cost)



Choose search space → (ranges, choices, constraints)



Run experiments → (grid, random, Bayesian, etc.)



Evaluate → (metric, human eval, cost & latency)



Pick winner → (and document why)



Lock & monitor → (revisit when data or goals change)



Common hyperparameter failure modes

- Leaving everything at defaults.
- Tuning on the test set (data leakage).
- Chasing metrics without understanding the trade-off.
- Not controlling randomness (no seed).
- Changing too many things at once.
- Not logging experiments or configs.
- Over-tuning to the validation set.
- Ignoring cost and latency impact.

Rule of thumb

Good defaults get you started.
Good experiments get you results.
Good engineering makes them repeatable, cheap and safe in production.



Interview edge:

* Hyperparameters exist at every layer: training, fine-tuning, inference, retrieval and ops.
* Most teams under-invest in RAG and decoding hyperparameters.
* Treat hyperparameter tuning as a product capability, not a one-off task.
* Document, version and justify every setting you ship.



One line to remember

Hyperparameters don't teach the model- they decide how the model learns, what it sees, and how it speaks.

Section 5: RAG, Architecture Scale and Production Reality

* 49. An LLM generates offensive or incorrect outputs. How do you handle it?

Treat it as an incident, not a prompt-tweaking exercise.

Assume the model will fail. Our job is to detect, contain, fix and learn—then make sure it doesn't silently regress.

Remember

* Incorrect output ≠ Offensive output (different root causes).
* You cannot fix what you cannot reproduce.
* Guardrails belong on both input and output.
* Add failures to the eval suite so they cannot regress.

The incident response sequence

1. Contain

Disable the affected feature or fall back to a safe response path if the impact is user-facing.

2. Reproduce & log

Capture the exact prompt, context, model version, parameters, timestamps and output.
Without this, you can't reproduce or fix.

3. Diagnose the layer at fault

Is it retrieval, prompt/policy, model behavior, safety filtering, schema validation, or something else?

4. Fix in layers

Apply controls at multiple layers so one weak link doesn't break the system.

5. Prevent & learn

Add to eval suite, red-team regularly, monitor in prod and document what happened and what we changed.

Diagnose: common root causes

Incorrect (factual) outputs

- Missing or irrelevant retrieved context (low recall).
- Wrong or outdated context (bad data / stale index).
- Model answers from parametric memory instead of saying "I don't know".
- Ambiguous question without clarifying follow-up.
- Instructions don't require grounding or citations.

Offensive / harmful outputs

- Gaps in input or output moderation and safety policies.
- Jailbreaks, prompt injection, or role confusion.
- Untrusted content treated as instructions.
- Weak system prompt or conflicting instructions.
- Model lacks refusal / de-escalation patterns.

Fix in layers: defense-in-depth

1. Retrieve & ground answers

- Use high-recall retrieval (hybrid + metadata filters).
- Over-retrieve, then rerank.
- Require citations in the answer.

2. Instruct to abstain

- If the context does not contain the answer, say "I don't know".
- Reward abstention in evaluation.

3. Moderation & safety (input + output)

- Classify input for policy violations.
- Classify output for harm//offense.
- Block, redact or rewrite as needed.

4. Prompt & data hardening

- Strong system prompt with clear boundaries.
- Treat all retrieved and user content as untrusted data.
- No direct execution of content.

5. Constrain outputs

- Use schemas / JSON mode / regex.
- Validate and retry on failure.
- Never trust free-text for critical actions.

6. Human in the loop

- Human review for high-risk categories (financial, legal, medical, HR).
- Escalate when confidence is low.

7. Fallbacks & safe responses

- Safe default replies.
- Offer clarifying questions.
- Provide help links or escalation paths.

Prevent & learn continuously

Eval suite (regression tests)

Add the failing case (and variants) to the evaluation suite. Run on every prompt, model or data change.

Red-teaming

Run structured red-teaming regularly. Use both automated checks and human adversaries.

Monitoring in production

- Track safety and quality metrics.
- Alert on spikes in policy violations, abstentions, errors.

Versioning & audit trail

Version prompts, indexes, models, parameters and data. Keep an audit trail for every change.

Escalation & disclosure

Document escalation owners, timelines and communication plan for users and stakeholders.

Interview edge:

* I would add it to the eval suite so it cannot silently regress.
* Guardrails belong on both input and output.

Don't blame the model. Design the system to fail safely and to improve every time it fails.

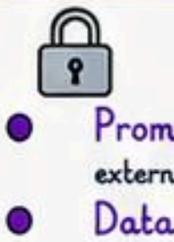
Rule of thumb

Detect early. Block unsafe.
Be honest when unsure.
Learn and iterate.

Section 5: RAG, Architecture Scale and Production Reality

* 50. What are common challenges of using LLMs?

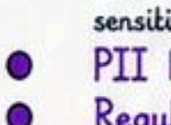
LLMs are powerful but come with real-world trade-offs. Challenges show up across accuracy, engineering, security, compliance and people.



The pattern: LLM projects fail on evaluation, cost control and security—almost never on model capability.

Remember

* No model is perfect.
* Assume failure modes and design for safety.
* Measure, monitor and iterate.
* Your system is more than the model.



1. Accuracy and trust

- **Hallucination:** Inherent to probabilistic generation; fluency ≠ truth.
- **Stale knowledge:** Parametric memory has a training cutoff and may be outdated.
- **Overconfidence:** Models sound sure even when wrong.
- **Poor calibration:** Rarely expresses uncertainty well.
- **Wrong source usage:** Answers may ignore retrieved context or mix sources.



Mitigate with RAG, citations, abstention rules, eval suites and human review.



2. Engineering and cost

- **Token-based cost:** Scales with input + output tokens; easy to overspend.
- **Latency:** Long context and complex tasks increase response time.
- **Rate limits & outages:** Providers throttle or go down.
- **Context limits:** Force chunking, summarization and trade-offs.
- **Non-determinism:** Same prompt ≠ same output; traditional tests don't work.



Mitigate with caching, guardrails, retries, backoff, eval suites (LLM-as-judge) and cost budgets.



3. Security and compliance

- **Prompt injection:** No complete solution—treat all external content as untrusted.
- **Data leakage:** Prompts and outputs may expose sensitive data to third-party APIs.
- **PII handling:** Risk of exposure or memorization.
- **Regulatory compliance (India):** DPDP Act governs transfer of personal data outside India. Many enterprises prefer self-hosted or in-region deployments.
- **Sector rules:** RBI, IRDAI, SEBI, Health Data Rules, etc. add additional constraints.



Mitigate with input/output filtering, data minimization, DLP, encryption, audit logs and compliance reviews.



4. Organisational challenges

- **Model churn:** Versions change, behaviour shifts, or models get deprecated.
- **Vendor lock-in:** Provider-specific features and data formats increase switching cost.
- **Bias & fairness:** Training data bias can create legal and reputational risk (hiring, lending, insurance).
- **Skills gap:** Demo is easy; production needs different engineering, eval and ops skills.
- **Defining success:** Most pilots fail due to unclear metrics and value definition.



Mitigate with abstraction layers, open standards, training, and clear KPIs tied to business outcomes.

What it means in practice



Measure everything
Define metrics (factuality, helpfulness, safety, cost, latency) and track continuously.



Design for failure
Fallbacks, retries, human in-the-loop and circuit breakers are not optional.



Control cost
Set budgets, monitor tokens in real time, use caching, compress and summarize.



Secure by default
Least-privilege, data minimization, red-teaming and ongoing monitoring.



Iterate with feedback
Collect user feedback, add hard cases to eval suite, ship improvements.

Interview edge:

* Name the pattern, not just the list.
* LLM projects fail on evaluation, cost control and security—almost never on model capability.
* Build systems that fail safely and improve continuously.

Rule of thumb
Assume the model will be wrong, or misused. Your job is to contain the damage, learn fast and ship what is safe and useful.

